

Machine & Deep Learning pour l'Ingénierie des Données

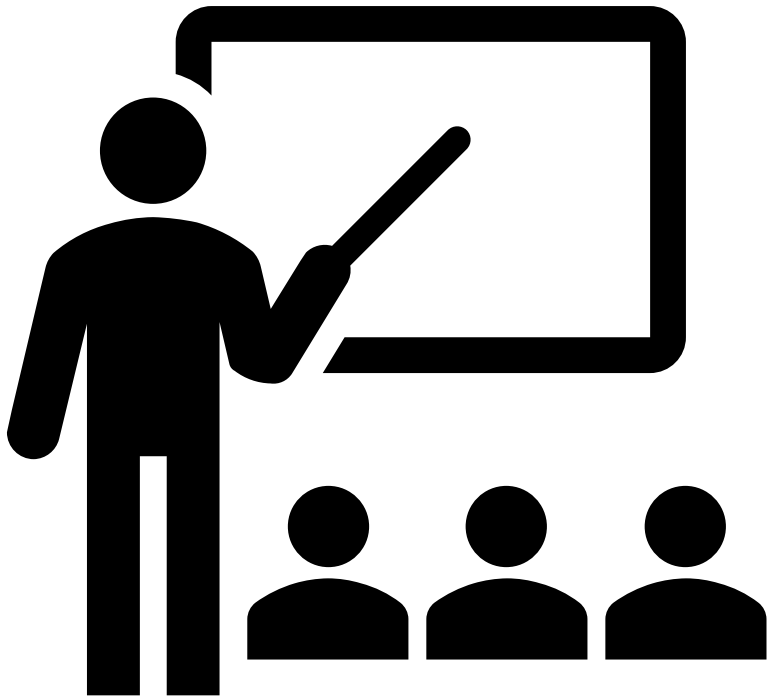
Philip Tchatchoua

2025-2026

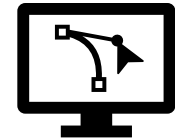


Mastère Data Engineering & IA

Contenu du cours



Fondamentaux du
Machine & Deep
Learning



Extraction, préparation,
visualisation et
communication des
données



Modélisation
supervisée et non-
supervisée en Machine
Learning



Modélisation
supervisée et avancée
en Deep Learning

IV. Modélisation supervisée et avancée en Deep Learning



Comprendre et implémenter des architectures profondes adaptées aux différents types de données

4.1

**Réseaux de neurones
artificiels**

4.2

**CNN et vision par
ordinateur**

4.3

**RNN et traitement des
données séquentielles**

4.4

**Métriques d'évaluation
des performances**

4.5

**Optimisation et
régularisation**

4.6

**IA générative : GAN,
VAE**

4.7

**Attention, Transformers,
RAG & LLM**

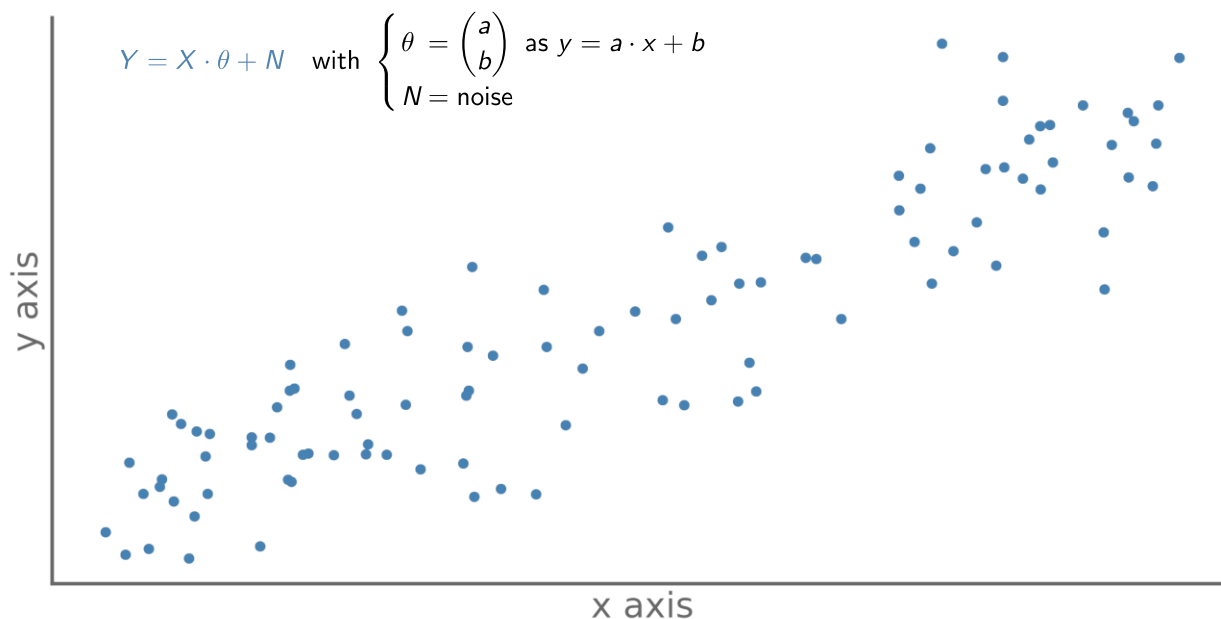
4.8

**Modélisation
multimodale**

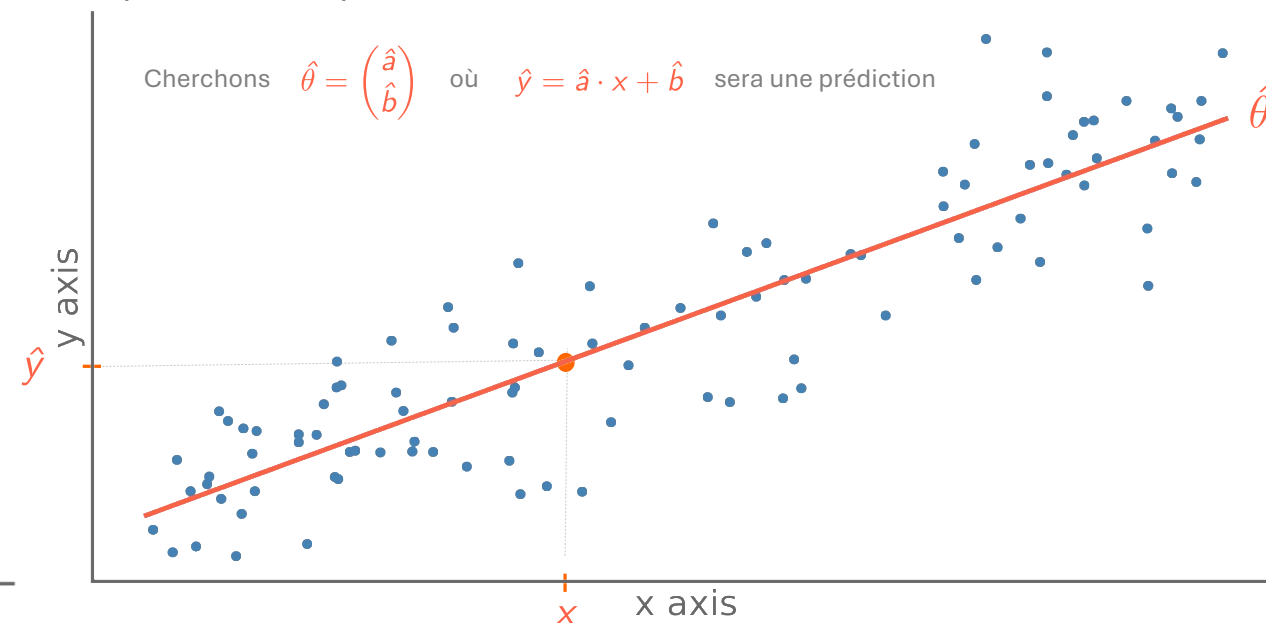
Régression linéaire

En statistique, la régression linéaire est un modèle qui estime la relation linéaire entre une réponse scalaire et une ou plusieurs variables explicatives.

Nous avons un phénomène pour lequel nous avons des observations.

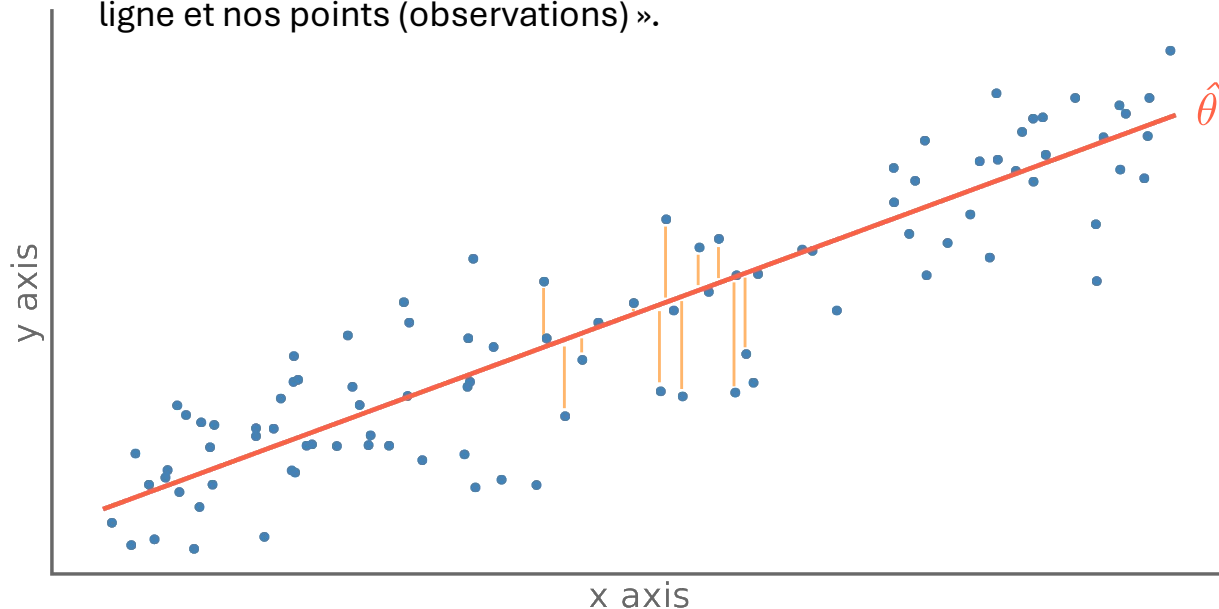


Nous recherchons une ligne droite qui passe « au plus près » de nos points d'observations.



Régression linéaire

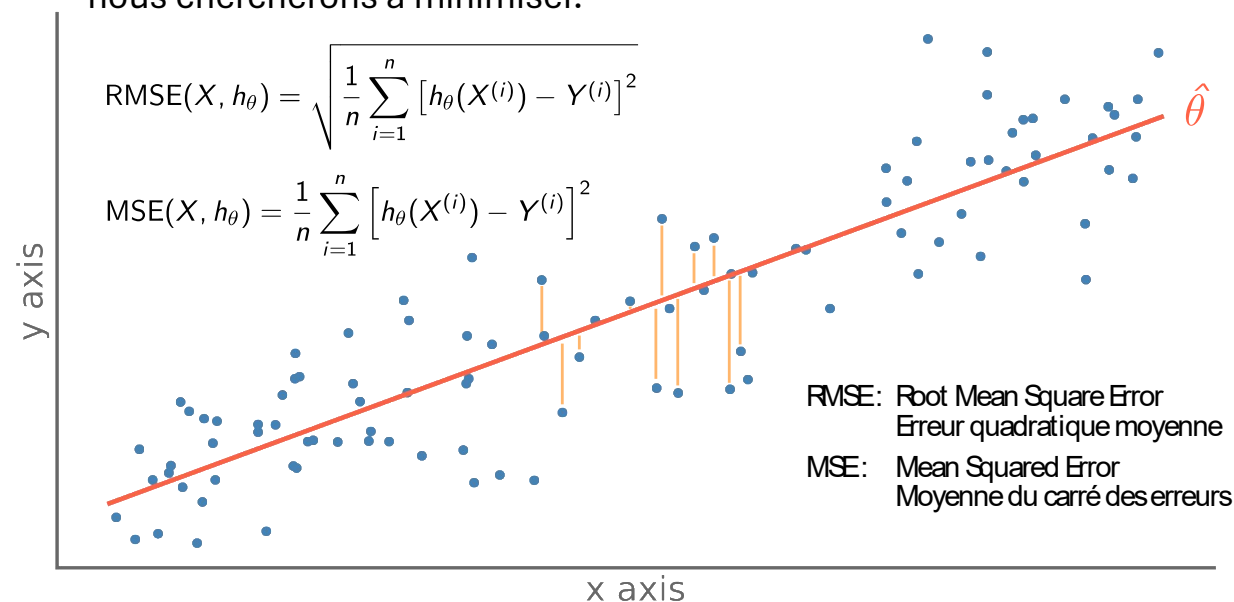
« Au plus près » signifie « minimiser la distance entre la ligne et nos points (observations) ».



Pour cela, nous utiliserons une « fonction de perte », que nous chercherons à minimiser.

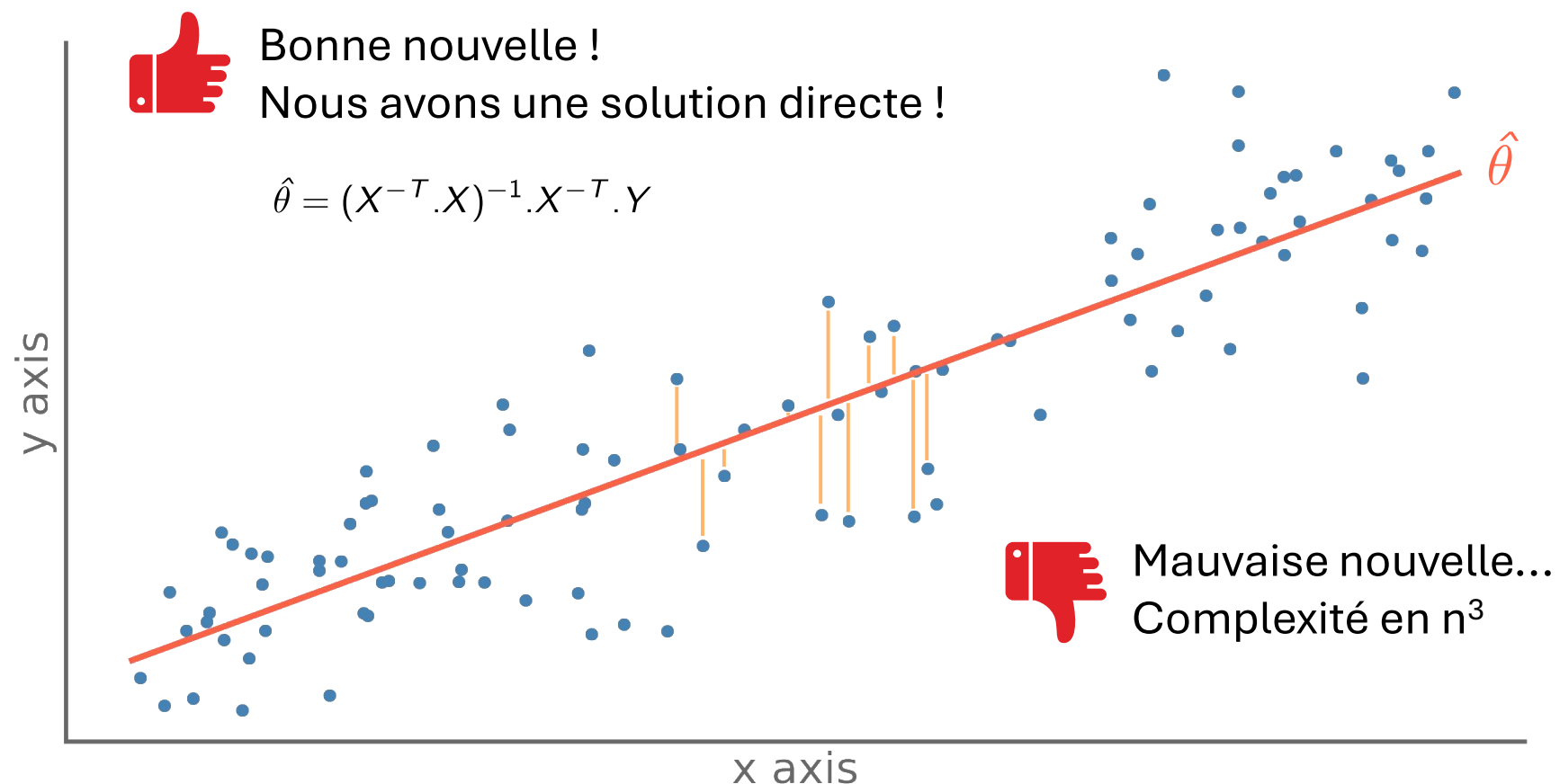
$$\text{RMSE}(X, h_{\theta}) = \sqrt{\frac{1}{n} \sum_{i=1}^n [h_{\theta}(X^{(i)}) - Y^{(i)}]^2}$$

$$\text{MSE}(X, h_{\theta}) = \frac{1}{n} \sum_{i=1}^n [h_{\theta}(X^{(i)}) - Y^{(i)}]^2$$



La fonction de perte ou de coût (*loss function*) est la quantification de l'écart entre les prévisions du modèle et les observations réelles.

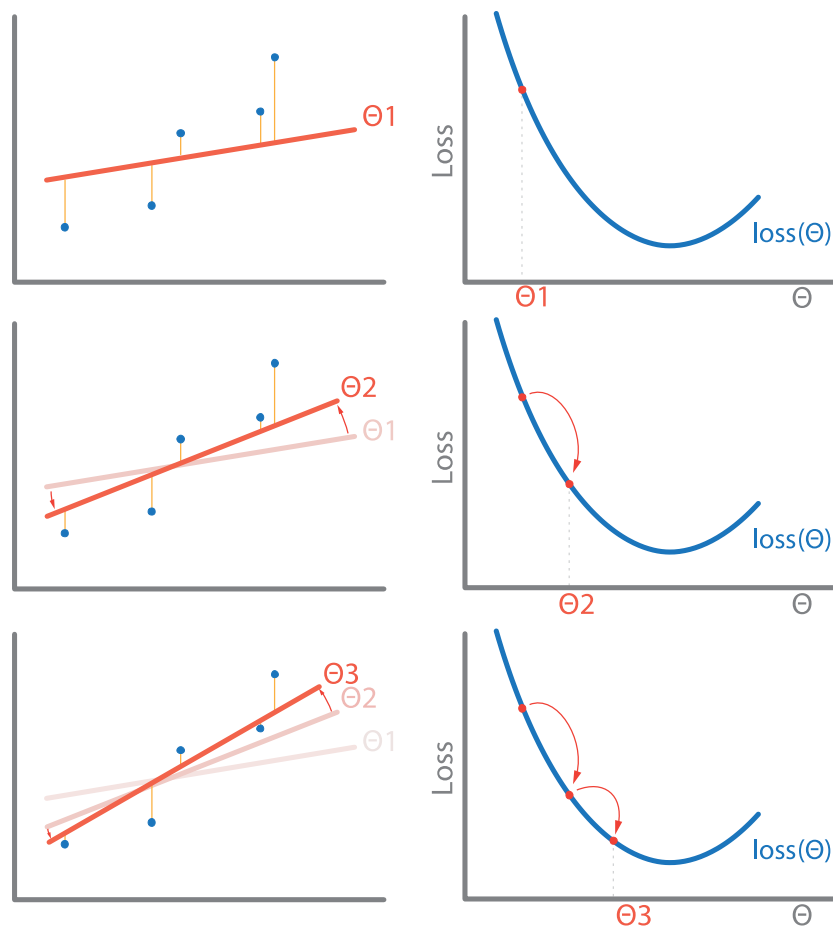
Régression linéaire



Complexité algorithmique proportionnelle au cube de la taille de l'entrée → algorithme coûteux en temps et en ressources de calcul

Descente du gradient

L'algorithme du gradient, aussi appelé **descente de gradient**, est un algorithme d'optimisation différentiable. L'objectif de la méthode de descente de gradient est de trouver un **minimum d'une fonction de plusieurs variables le plus rapidement possible**.



Nous allons rechercher de manière itérative la meilleure position de notre ligne, en faisant varier ses paramètres (Θ).



Mais comment pouvons-nous faire varier efficacement nos paramètres (Θ) ?

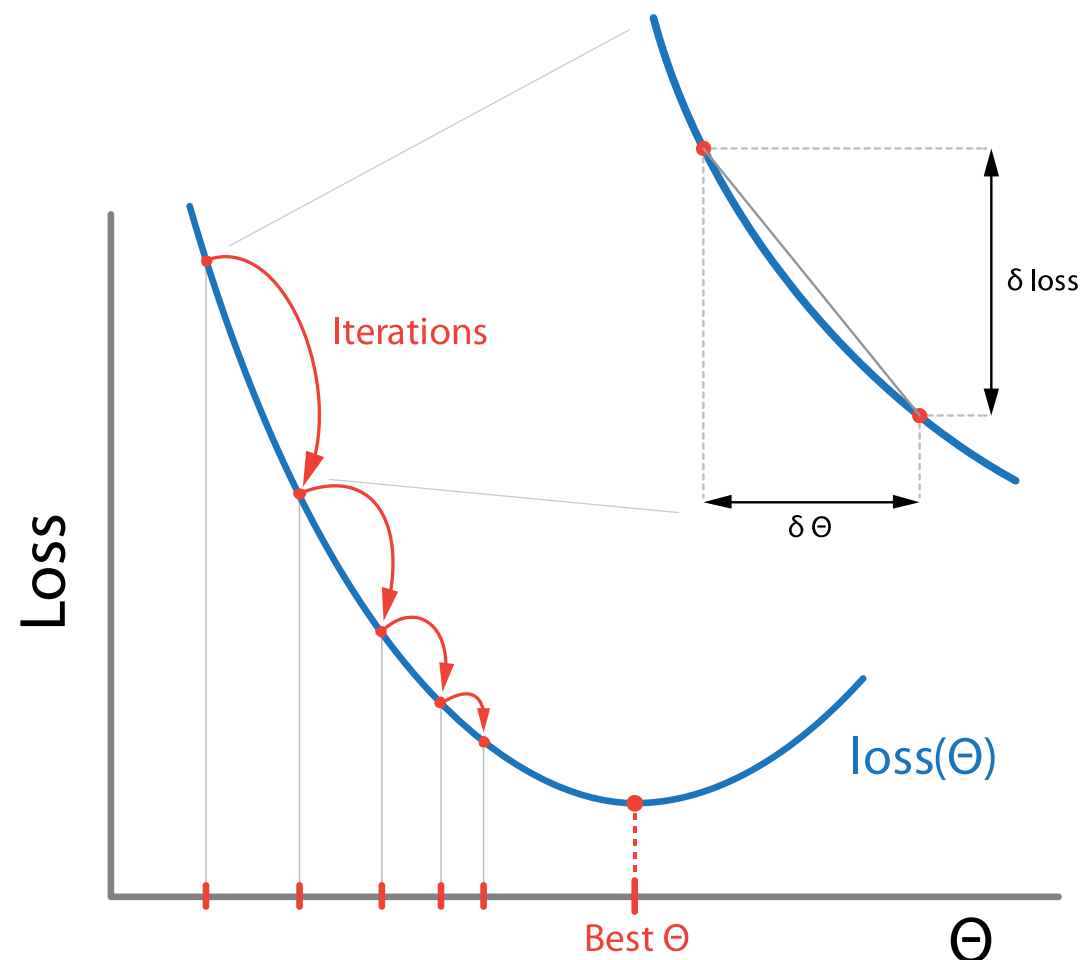
Note :

Les fonctions de perte pourraient être :

$$\text{RMSE}(X, h_{\theta}) = \sqrt{\frac{1}{n} \sum_{i=1}^n [h_{\theta}(X^{(i)}) - Y^{(i)}]^2}$$

$$\text{MSE}(X, h_{\theta}) = \frac{1}{n} \sum_{i=1}^n [h_{\theta}(X^{(i)}) - Y^{(i)}]^2$$

Descente du gradient



En changeant Θ de $\delta \Theta$
 Nous améliorons la perte (Θ) de $\delta loss$

Le gradient est la pente que nous allons suivre pour minimiser notre fonction de perte.

$$\text{gradient} = \frac{\delta loss}{\delta \theta}$$

Une solution itérative est : $\theta \leftarrow \theta - \eta \cdot \frac{\delta loss}{\delta \theta}$

où η est le taux d'apprentissage

Ce processus est appelé descente de gradient et la fonction itérative utilisée pour optimiser la descente est la fonction d'optimisation.

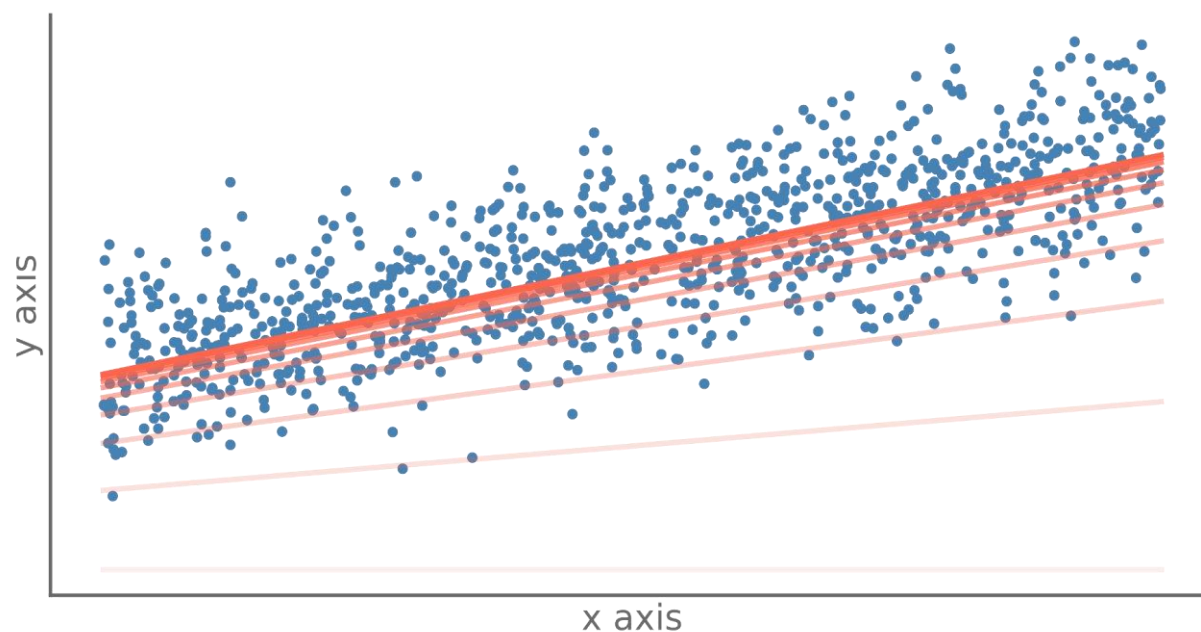
Descente du gradient

$$MSE(X, h_{\theta}) = \frac{1}{n} \sum_{i=1}^n [h_{\theta}(X^{(i)}) - Y^{(i)}]^2$$

$$\nabla_{\theta} MSE(\Theta) = \begin{bmatrix} \frac{\partial}{\partial \theta_0} MSE(\Theta) \\ \frac{\partial}{\partial \theta_1} MSE(\Theta) \\ \vdots \\ \frac{\partial}{\partial \theta_n} MSE(\Theta) \end{bmatrix} = \frac{2}{m} X^T \cdot (X \cdot \Theta - Y)$$

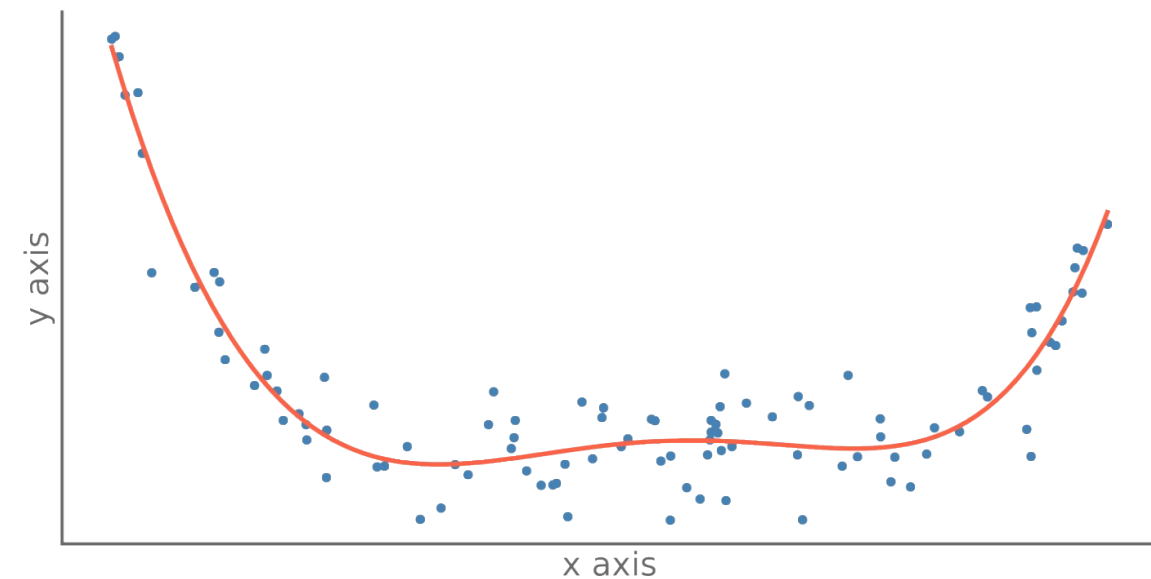
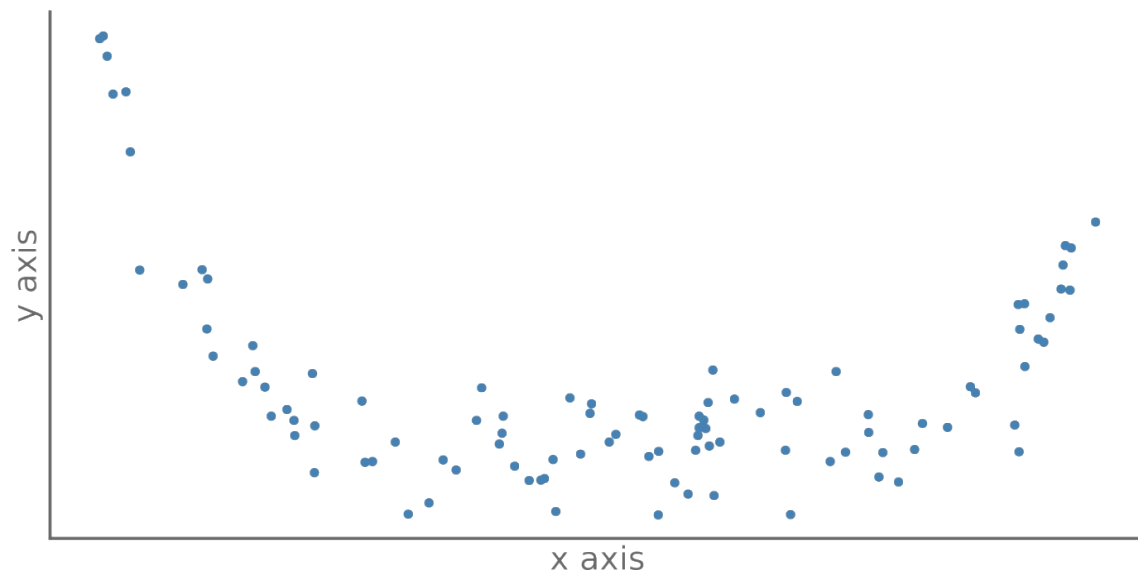
La solution itérative est : $\Theta \leftarrow \Theta - \eta \cdot \nabla_{\theta} MSE(\Theta)$

où η est le taux d'apprentissage



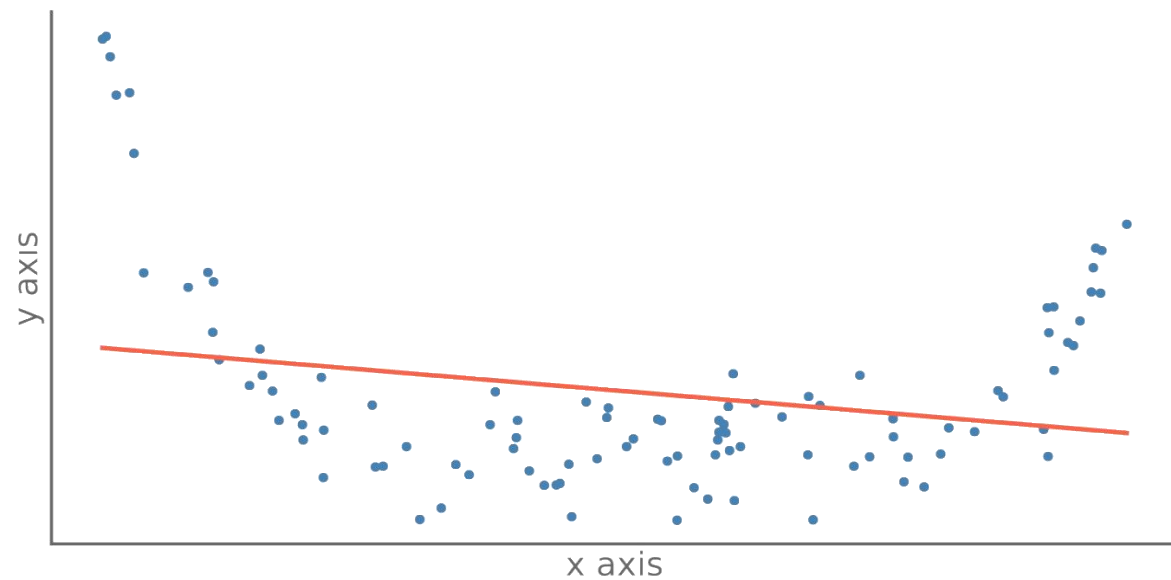
#i	Loss	Gradient		Theta	
0	+12.481	-6.777	-1.732	-3.388	+0.000
20	+4.653	-4.066	-1.039	-2.033	+0.346
40	+1.835	-2.440	-0.624	-1.220	+0.554
60	+0.821	-1.464	-0.374	-0.732	+0.679
80	+0.455	-0.878	-0.224	-0.439	+0.754
100	+0.324	-0.527	-0.135	-0.263	+0.799
120	+0.277	-0.316	-0.081	-0.158	+0.826
140	+0.260	-0.190	-0.048	-0.095	+0.842
160	+0.253	-0.114	-0.029	-0.057	+0.851
180	+0.251	-0.068	-0.017	-0.034	+0.857
200	+0.250	-0.041	-0.010	-0.020	+0.861

Régression polynomiale

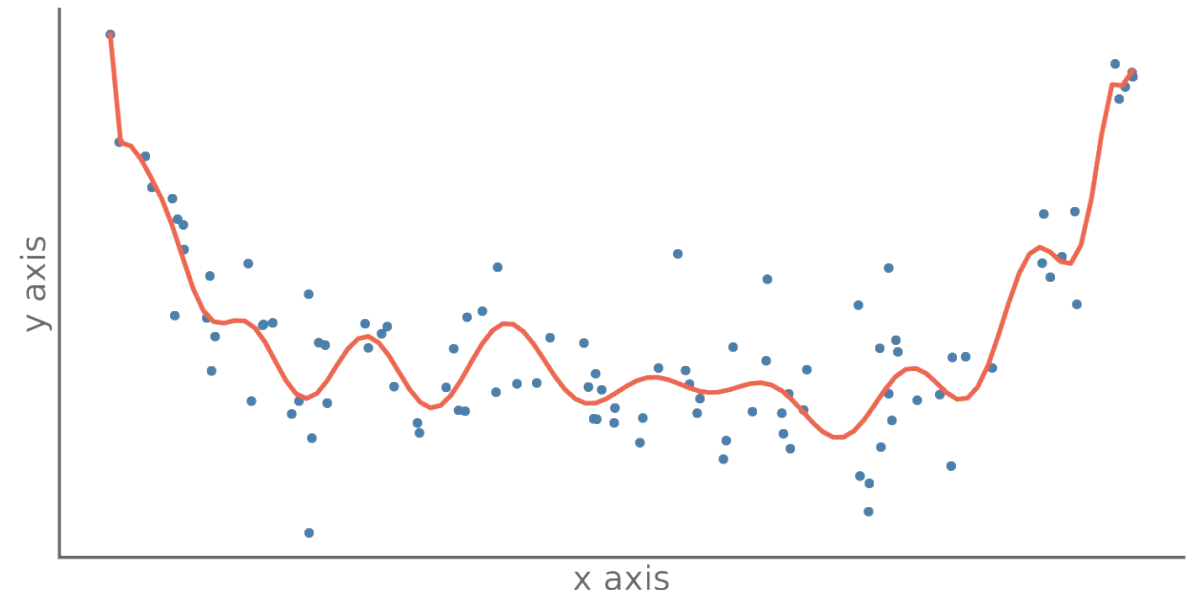


$$P_n(x) = a_0 + a_1 \cdot x + a_2 \cdot x^2 + \cdots + a_n \cdot x^n = \sum_{i=0}^n a_i \cdot x^i$$

Régression polynomiale



Sous-apprentissage

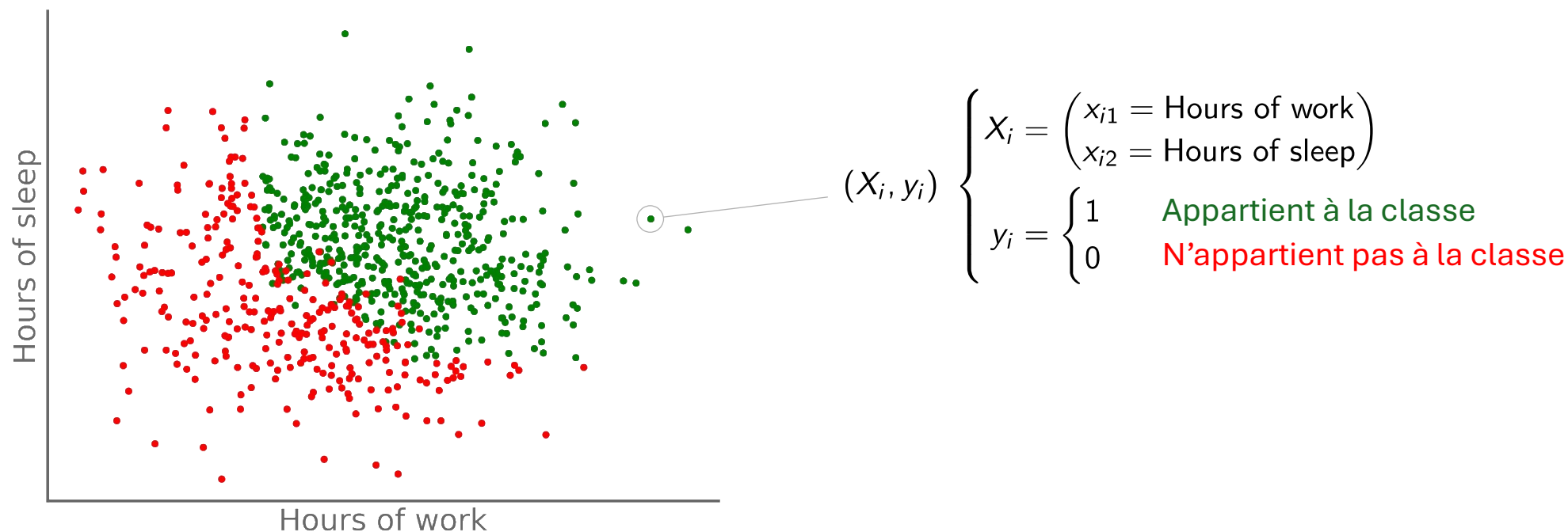


Sur-apprentissage

Régression logistique

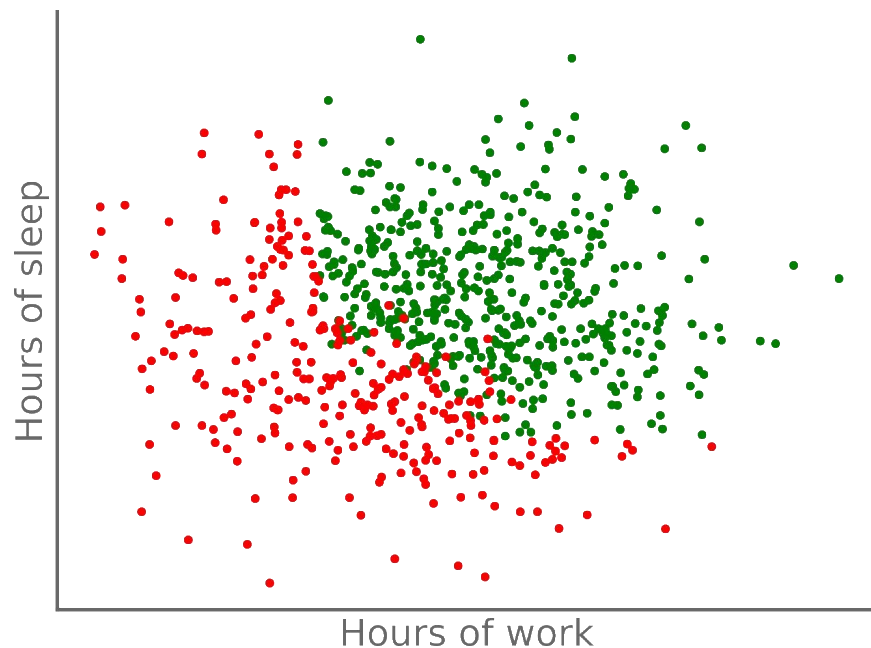
En statistiques, la régression logistique (ou modèle logit) vise à fournir une probabilité d'appartenir à une classe. C'est un modèle de régression principalement **utilisé en classification binaire**.

Dataset : X Observations
y Classe



Régression logistique

Dataset : X Observations
y Classe

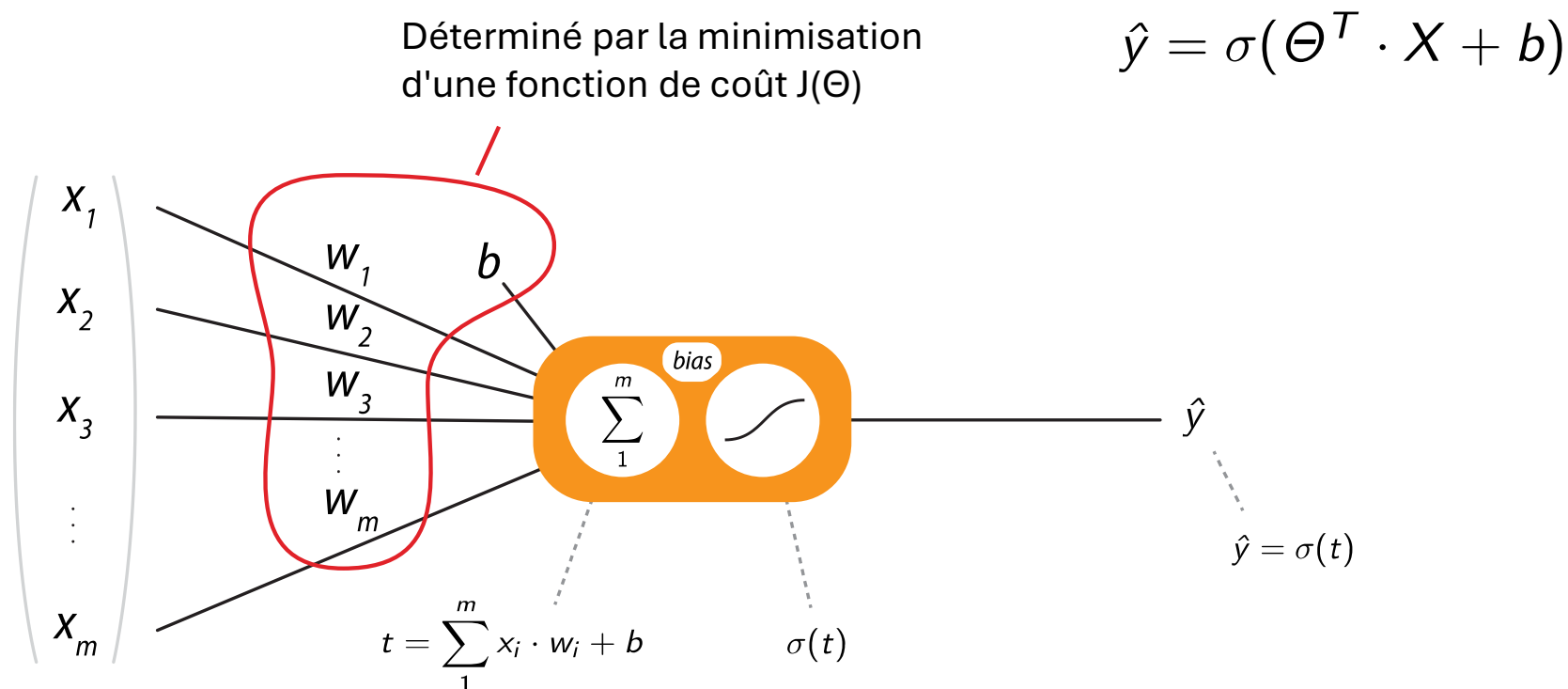


Objectif : Prédire la classe x donnée, nous voulons prédire y
$$y_{pred} = f(x)$$

où f est une fonction linéaire



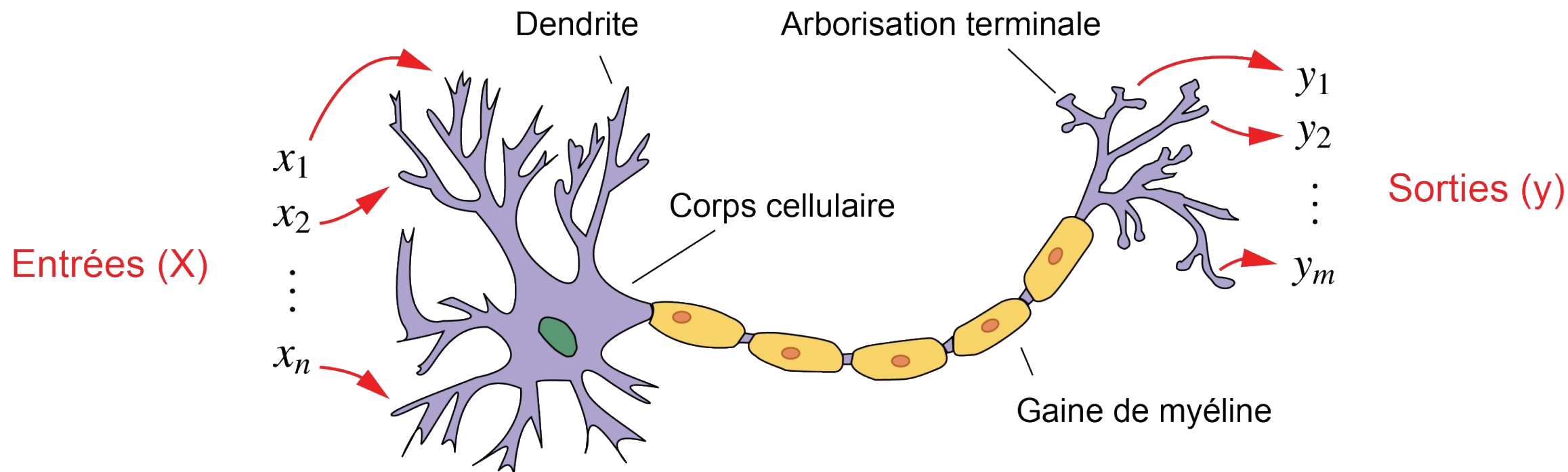
Régression logistique



Entrée	Biais/Poids	Fonction d'activation	Sortie
X	Θ, b	$\sigma(t)$	$\hat{y}$

C'est un « **neurone artificiel** » ! On a donc un réseau neuronal de... 1 neurone !

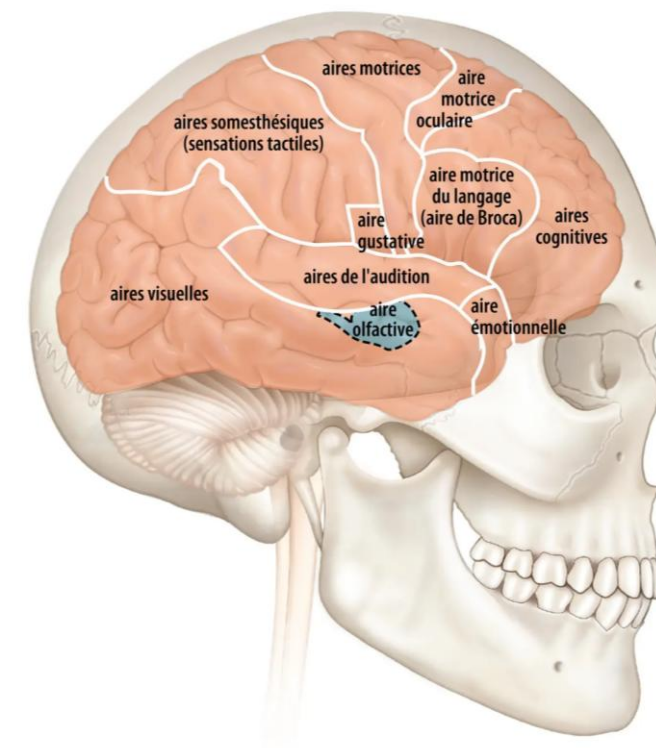
Neurone biologique



Un neurone est une cellule capable de transmettre des informations à d'autres neurones au travers de ses différentes connexions (synapses).

Cerveau humain : un modèle biologique extrême

- Nombre de neurones : ≈ 100 milliards (10^{11}).
- Nombre moyen de connexions par neurone (synapses) : $\approx 10\,000$.
- Nombre total de synapses : $\approx 100\,000$ milliards (10^{14}).
- Densité du cortex : ≈ 1 milliard de connexions par mm^3 .
- Performance énergétique : Le cerveau fonctionne à ≈ 20 W.



Cerveau humain : un modèle biologique extrême

CERVEAU HUMAIN vs. RÉSEAU DE NEURONES ARTIFICIEL



Nombre de neurones

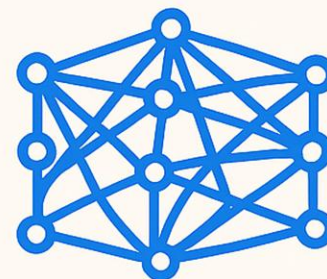
100 milliards

Connexions par neurone

10 000

Volume du cerveau

$\approx 1\,200\text{ cm}^3$



Nombre de neurones

3 000 milliards

Connexions par neurone

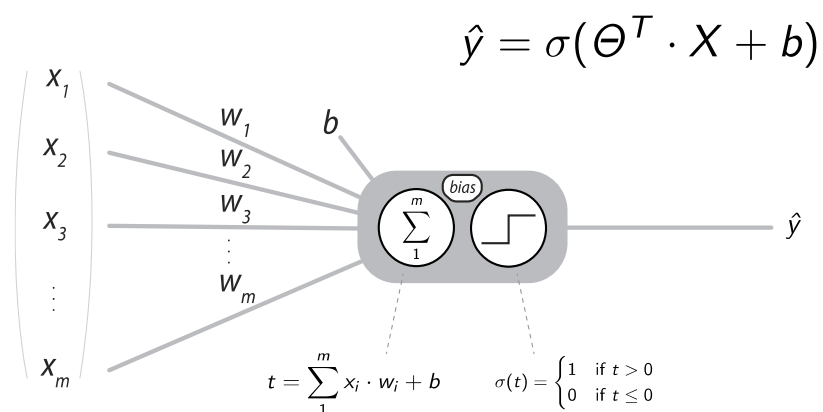
1000

Volume du modèle IA

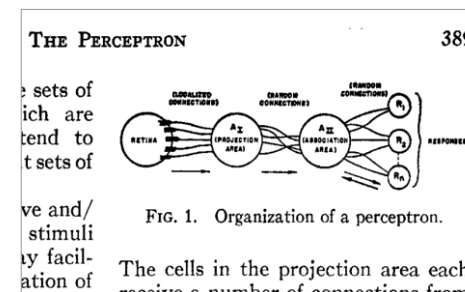
$\geq 30\,000\text{ cm}^3$

Neurone artificiel : Perceptron

L'élément de base des réseaux de neurones est l'algorithme du perceptron, qui **imite le fonctionnement du neurone biologique** et permet d'effectuer tâches très diverses.



Classifieur linéaire et binaire



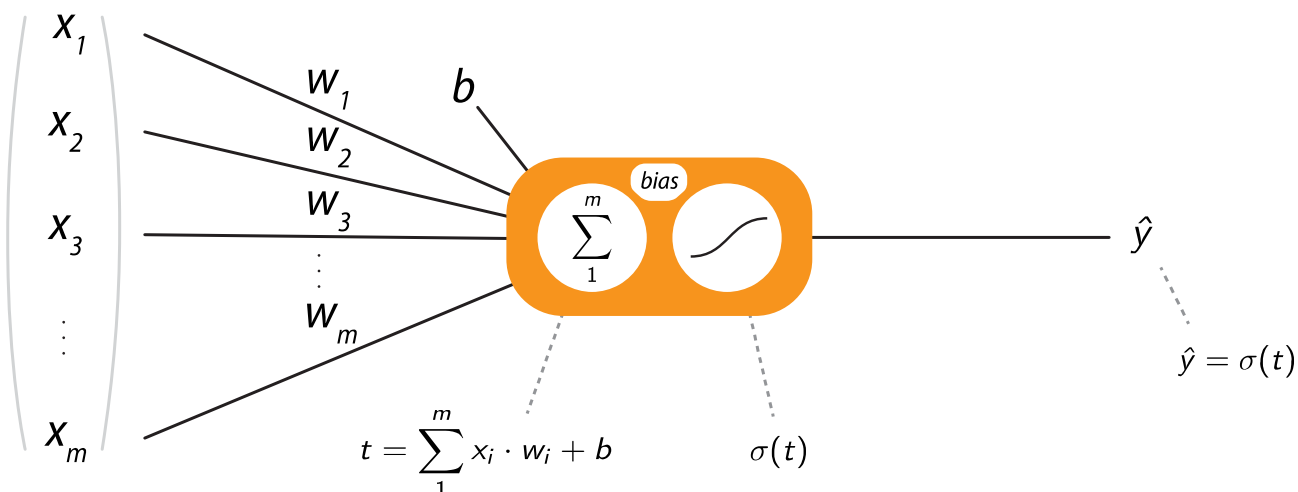
Perceptron
Frank Rosenblatt
1958



F. Rosenblatt, 1958

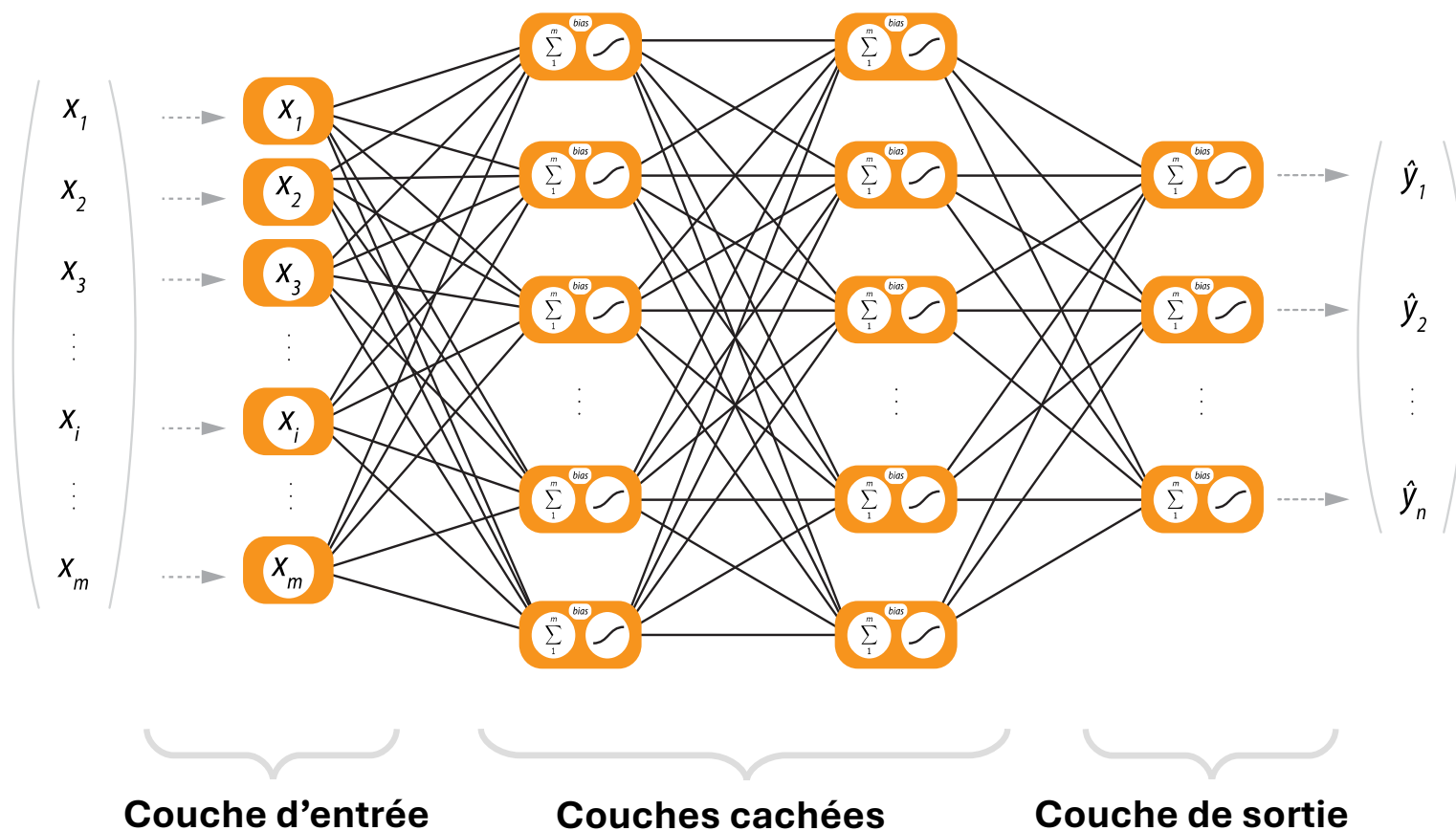
Neurone artificiel : Perceptron

- Le neurone artificiel est un automate doté d'une **fonction de transfert** (vecteur-à-scalaire) qui transforme ses entrées en sortie selon des règles précises.
 - Exemple : un neurone somme ses entrées, compare la somme résultante à une valeur seuil, et émet une sortie appropriée.
- Les paramètres importants sont les **coefficients synaptiques (biais + poids)**, et la **façon de les ajuster (fonction de perte / coût)**
 - Apprendre revient à déterminer les coefficients synaptiques les plus adaptés pour les exemples présentés



Deep Neural Networks (DNN)

Un **Deep Neural Network (DNN)**, ou réseau neuronal profond, est un modèle d'intelligence artificielle inspiré du fonctionnement du cerveau humain. Il est constitué de plusieurs couches de neurones artificiels connectés entre eux.



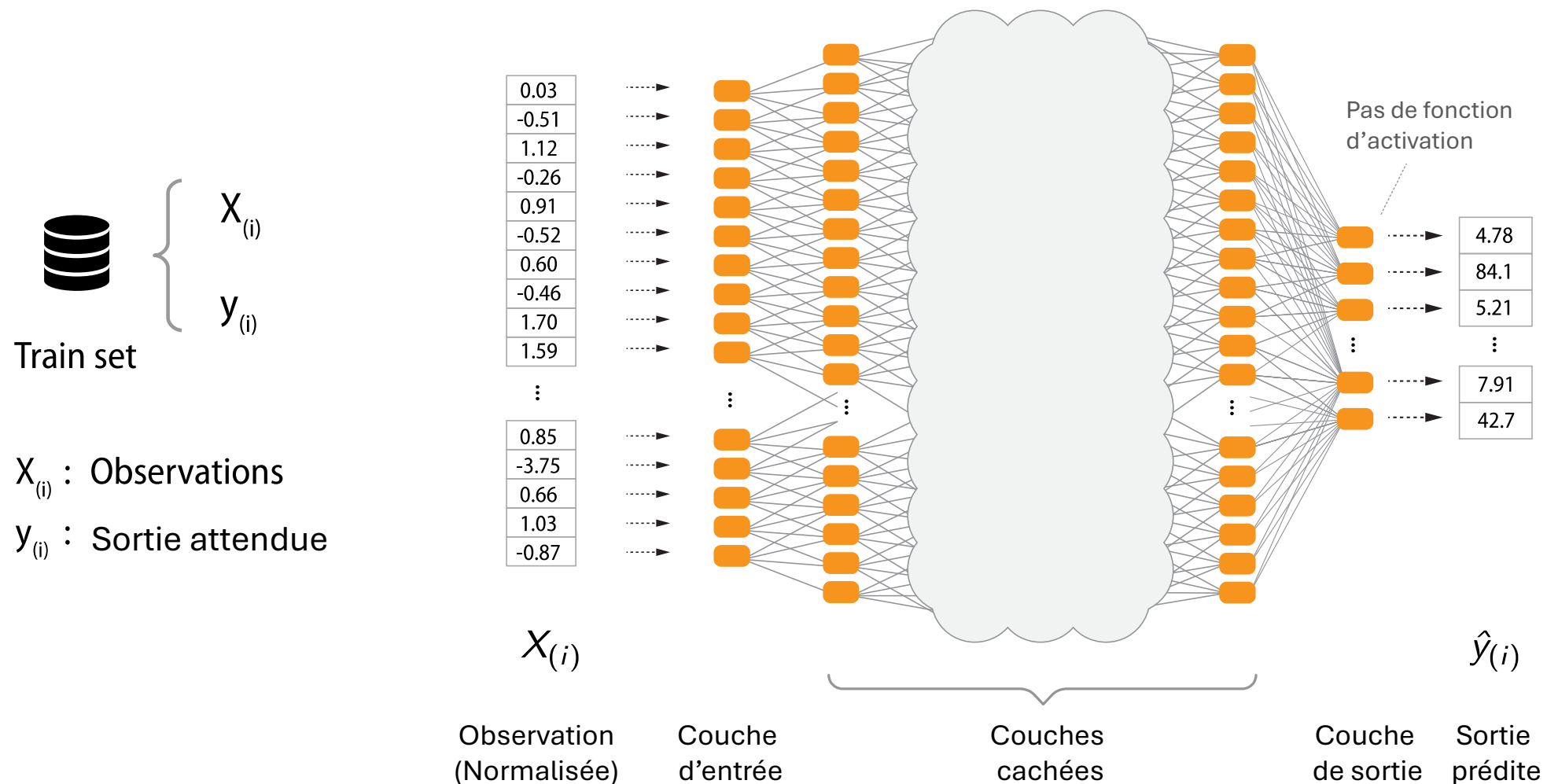
Deep Neural Networks (DNN)

Pourquoi le Deep Learning (DL) ?

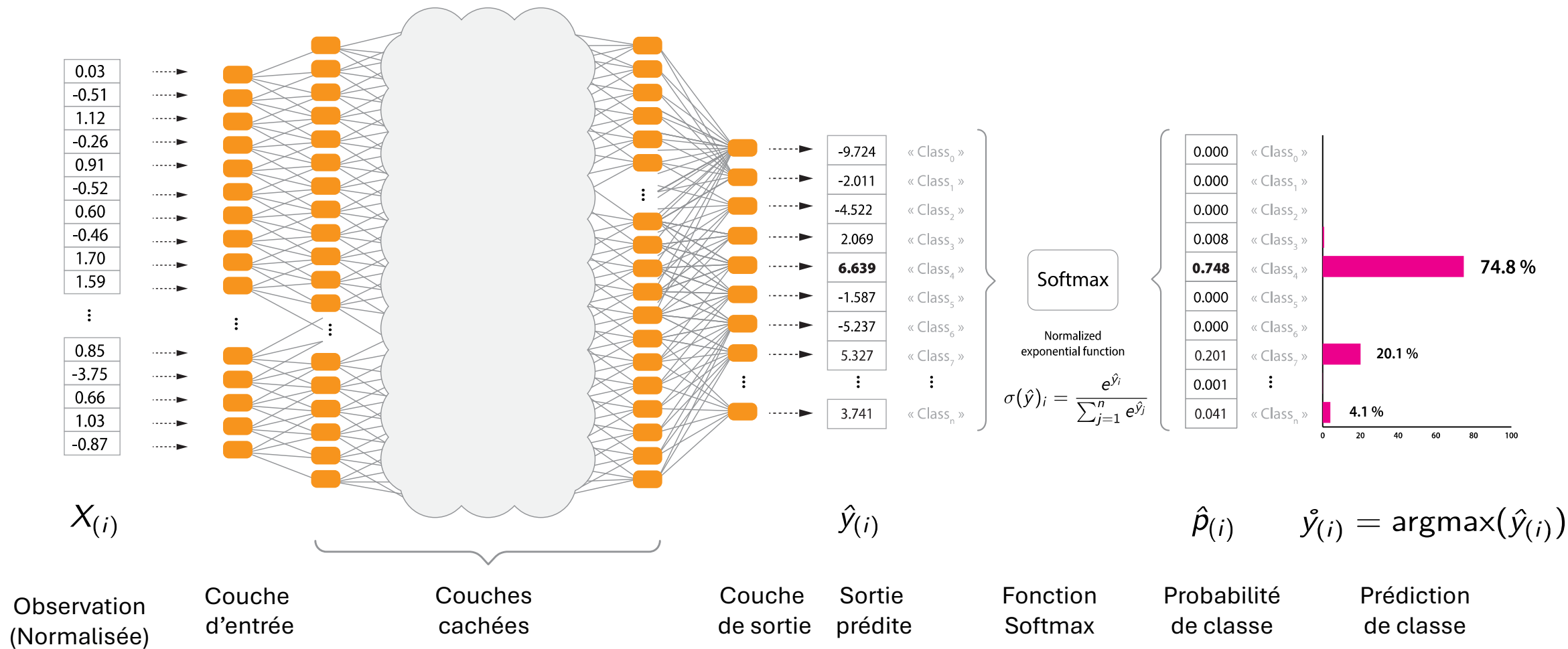
1. **Tâches complexes** : Les réseaux de neurones excellent pour les problèmes où les relations entre les données sont non linéaires ou trop complexes pour être définies manuellement (reconnaissance vocale, vision par ordinateur, traduction automatique, etc.). Certaines tâches, comme la reconnaissance d'objets dans des images ou le traitement du langage naturel, sont presque impossibles à résoudre efficacement sans réseaux de neurones.
2. **Apprentissage de haut niveau** : Les réseaux neuronaux profonds apprennent des représentations abstraites des données, ce qui leur permet de capturer des relations cachées.
3. **Grands volumes de données** : Plus les données sont nombreuses, plus les réseaux de neurones sont efficaces, grâce à leur capacité à apprendre automatiquement des caractéristiques pertinentes.

Deep Neural Networks (DNN)

Un DNN est un outil puissant qui apprend à résoudre des problèmes complexes en passant les données à travers **plusieurs niveaux d'analyse**.



Deep Neural Networks (DNN)



Deep Neural Networks (DNN)

Plusieurs éléments contribuent à rendre un réseau de neurones efficace dans ses prédictions.

1 Qualité des données

- Données pertinentes
- Nettoyage des données
- Prétraitement (normalisation, encodage augmentation)

2 Conception de l'architecture

- Taille et profondeur du réseau
- Types de couches (CNN, RNN,...)
- Choix de la fonction d'activation

3 Algorithme d'optimisation

- Optimisateurs avancés ?

4 Régularisation

- Éviter le surapprentissage (overfitting)

5 Taille et qualité des données d'entraînement / validation

- Quantité suffisante pour entraîner et tester
- Représentativité dans la division

6 Fonction de perte

- Adaptée à la tâche

7 Hyperparamètres

- Taux d'apprentissage, taille du batch, nombre d'époques

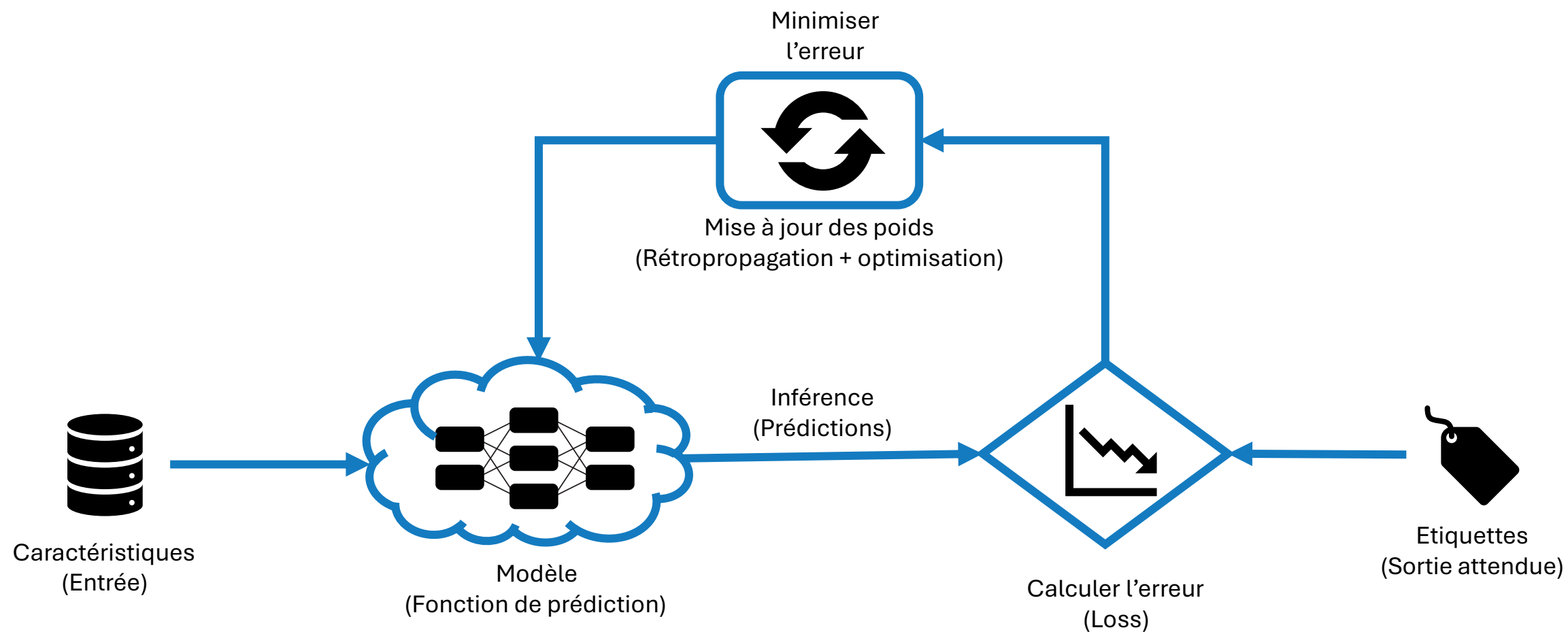
8 Métriques, monitoring et ajustements

- Métriques adaptées à la tâche
- Surveiller la perte et les métriques
- Ajuster les paramètres

9 Matériel et temps d'entraînement

- Parallélisation des calculs (GPU, TPU)

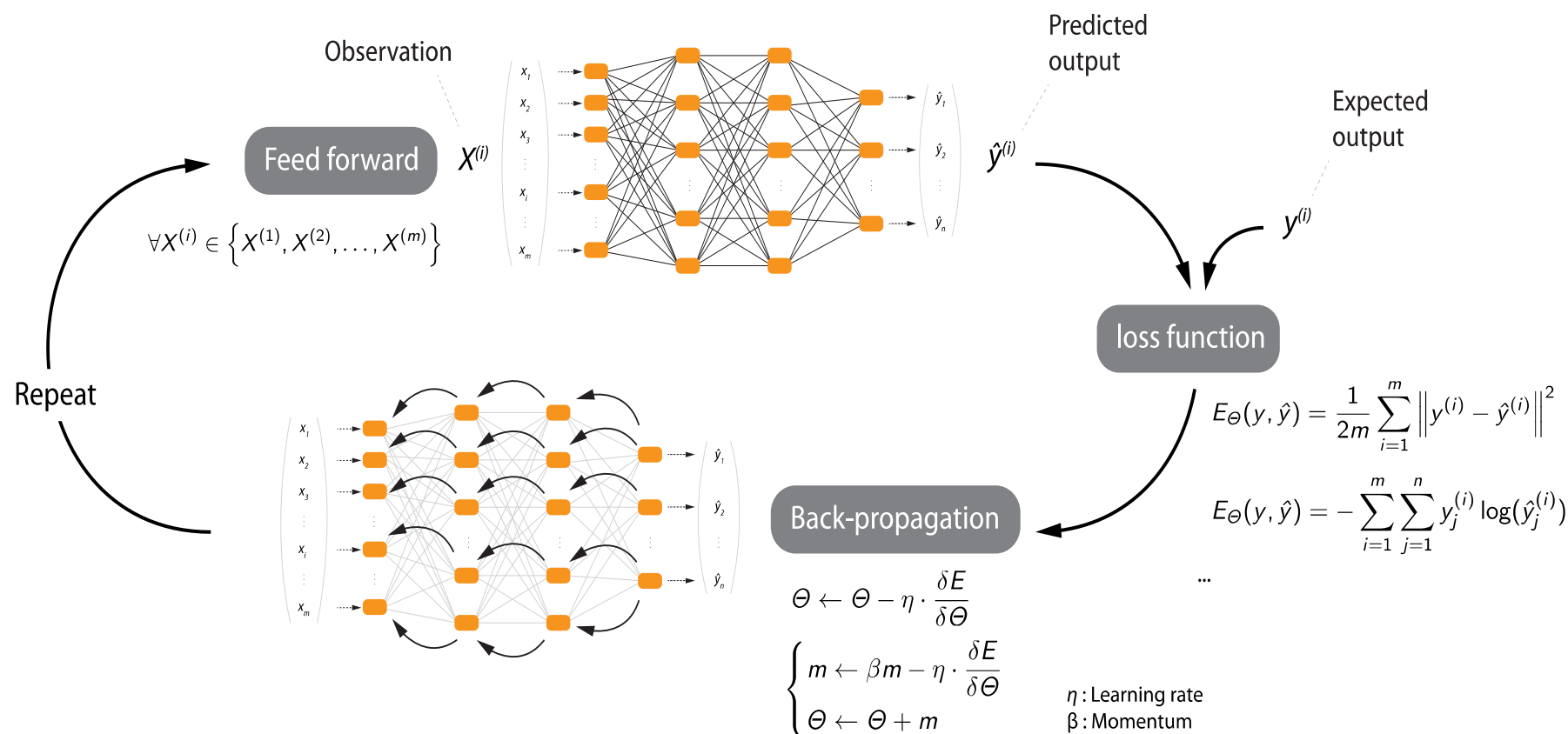
Processus d'apprentissage - général



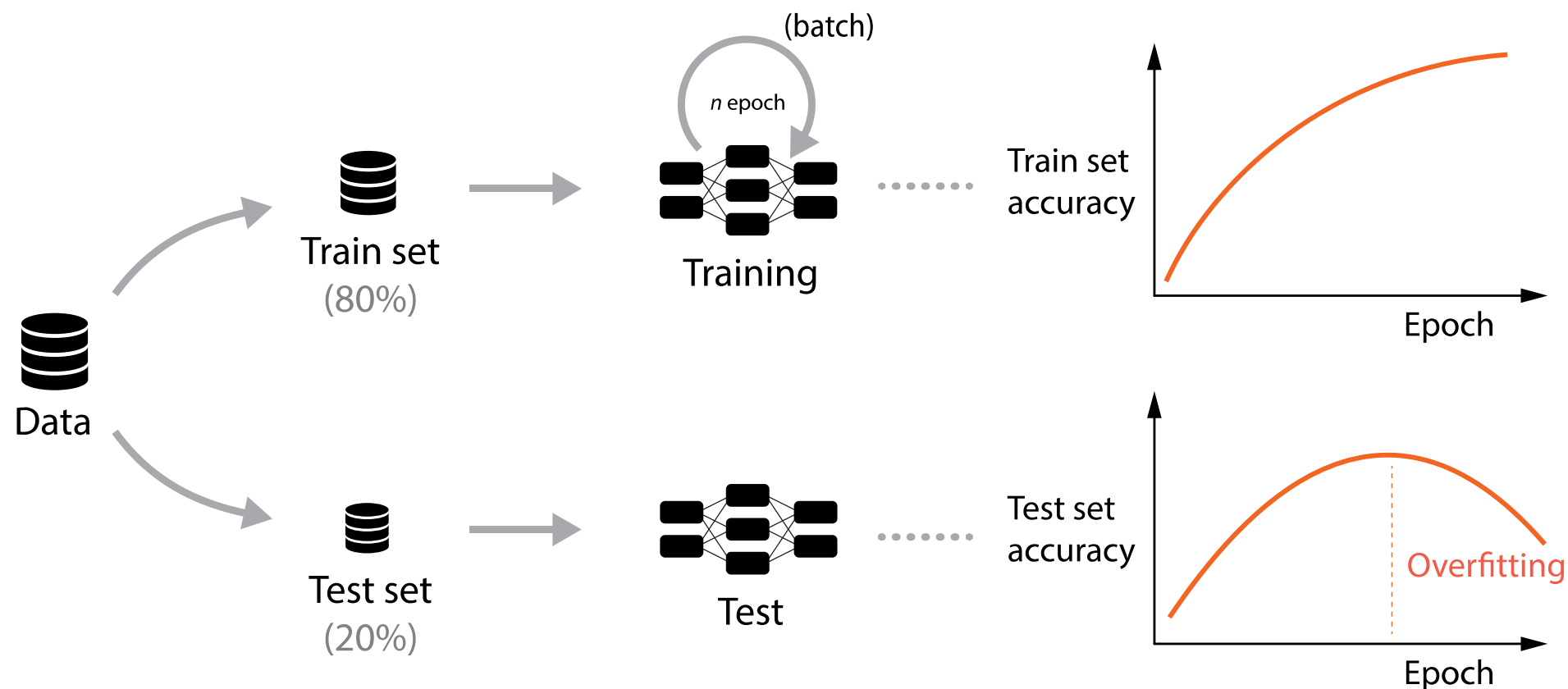
Processus d'apprentissage - général

La back-propagation (rétropropagation) du gradient est une **méthode utilisée pour entraîner les DNN**.

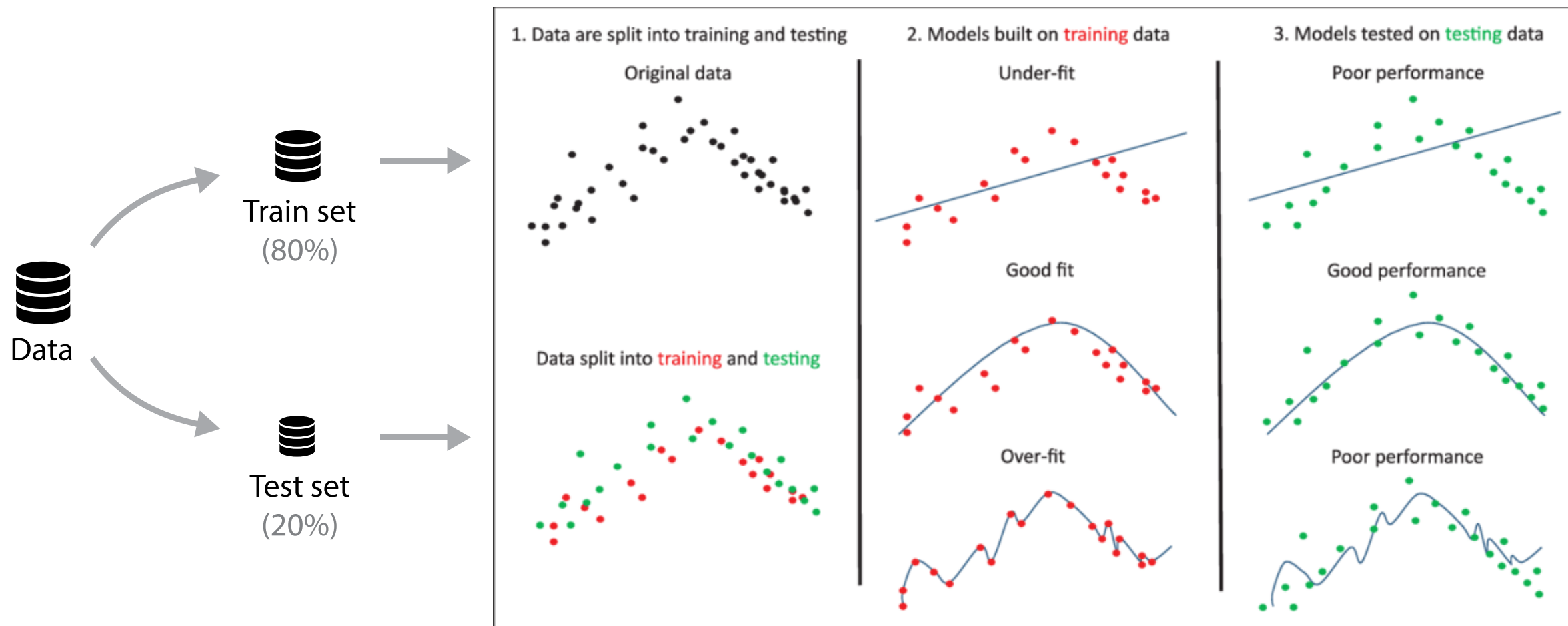
- **Ajuste les poids du réseau** en calculant les erreurs et en "propageant" ces erreurs en sens inverse dans le réseau.
- L'objectif étant de minimiser l'erreur en ajustant progressivement les paramètres du réseau à l'aide d'une méthode d'optimisation comme la descente de gradient.



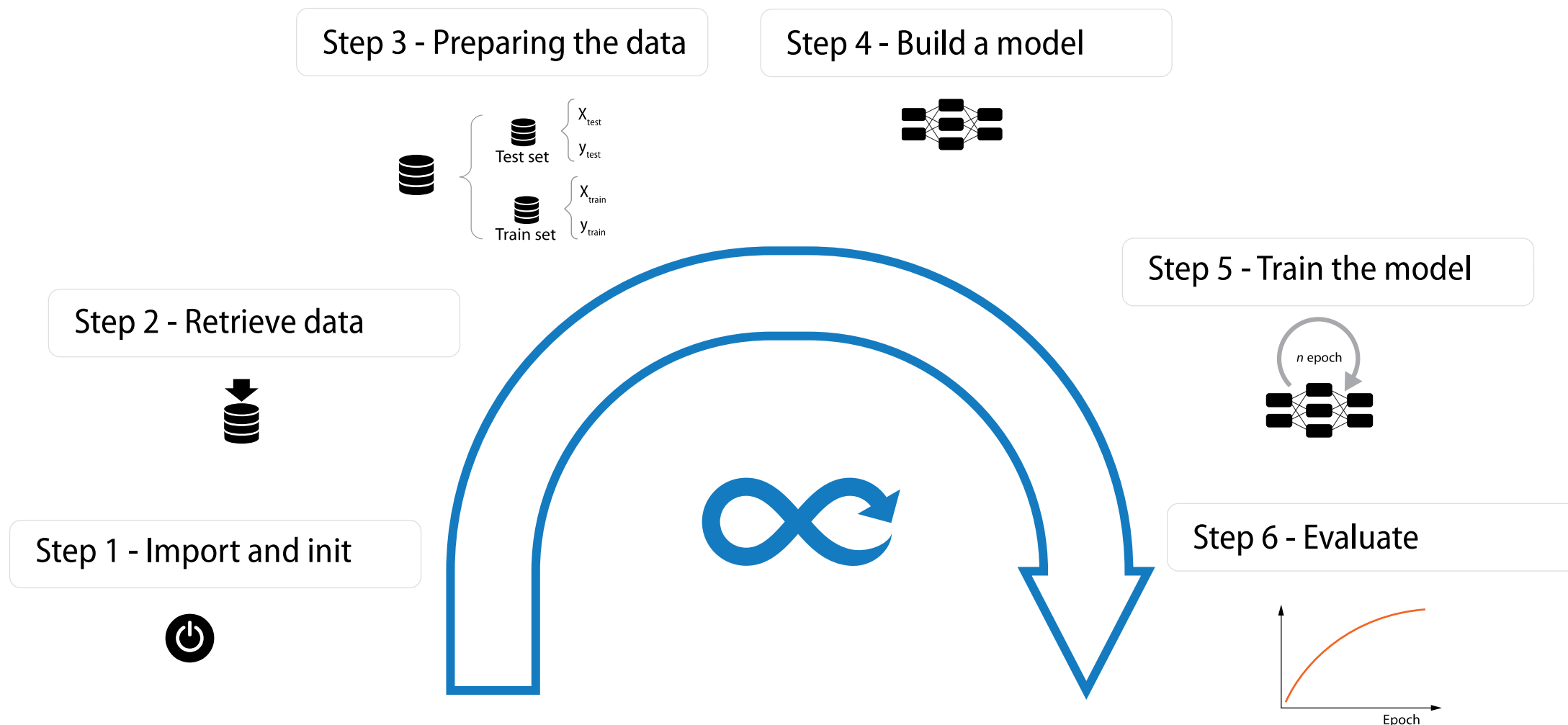
Processus d'apprentissage - général



Processus d'apprentissage - général

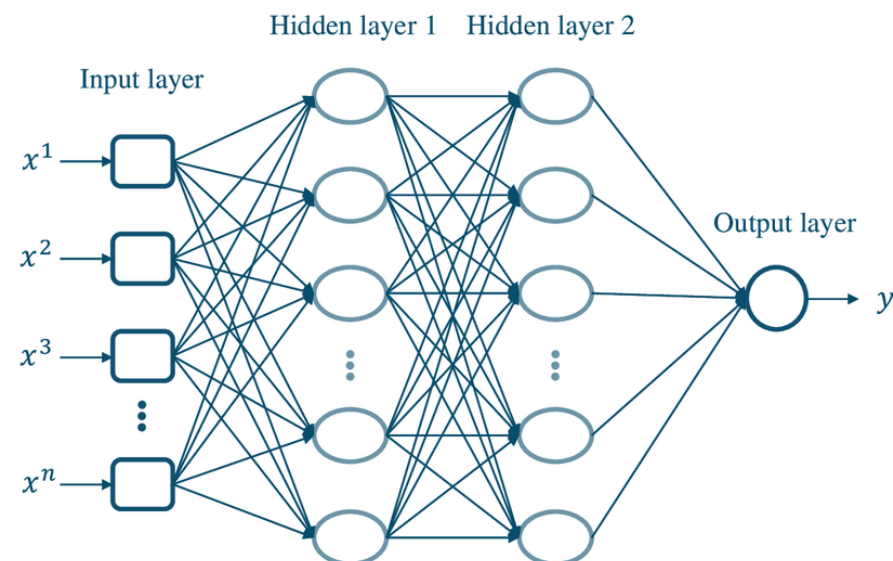


Processus général



Perceptron multi-couche (MLP)

Les MLP sont des réseaux de neurones de base où l'information circule de l'entrée vers la sortie sans retroaction (*feedforward*). Ils sont composés de plusieurs couches entièrement connectées (*fully connected, FC*). Les MLP sont **adaptés aux données structurées où les relations entre les variables sont complexes mais non séquentielles**.

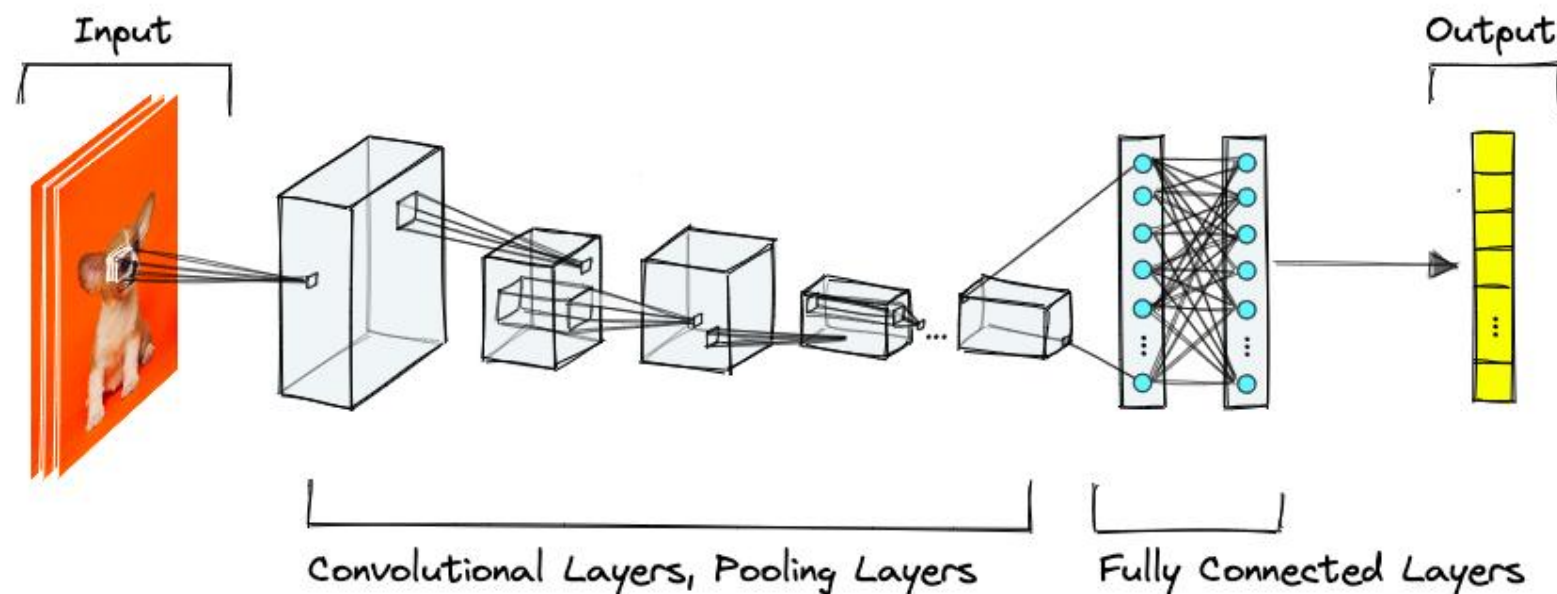


Applications :

- **Classification de données tabulaires** : Par exemple, pour segmenter des clients ou détecter des fraudes en analysant des données structurées.
- **Prédiction de ventes** : Estimer les ventes futures en fonction de données historiques et de facteurs externes.

Réseaux de Neurones Convolutifs (CNN)

Les CNN sont conçus pour **traiter des données ayant une structure en grille**, comme les images. Ils utilisent des filtres pour détecter des motifs locaux. Les CNN excellent dans la **capture des relations spatiales et des motifs visuels**, ce qui les rend idéaux pour l'analyse d'images.

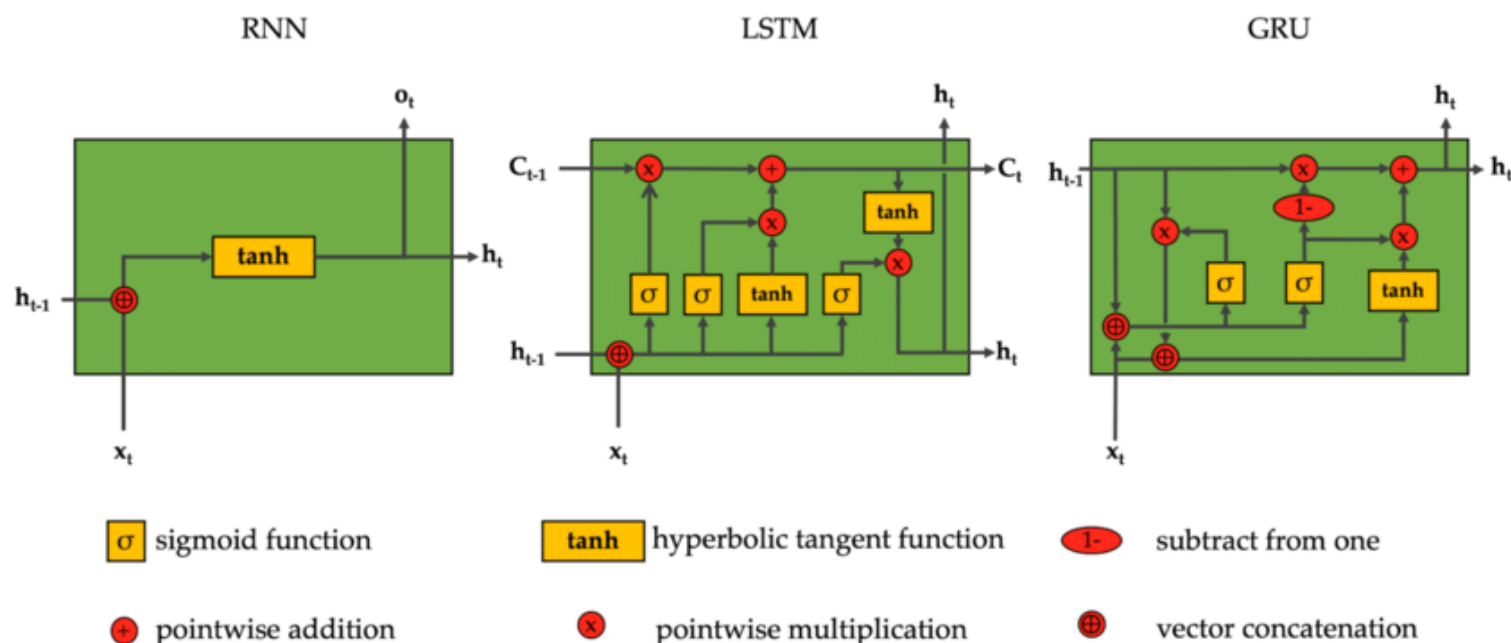


Applications :

- **Reconnaissance d'images** : Identifier des objets ou des personnes dans des photos, utilisé dans la reconnaissance faciale.
- **Imagerie médicale** : Détecter des anomalies ou des maladies à partir de scans médicaux (radios, IRM).

Réseaux de Neurones Récurrents (RNN, LSTM, GRU)

Ces réseaux sont adaptés au **traitement des données séquentielles**, car ils possèdent une mémoire interne pour conserver l'information contextuelle. La capacité à **gérer des dépendances temporelles** fait des RNN, LSTM et GRU des choix appropriés pour les données séquentielles.

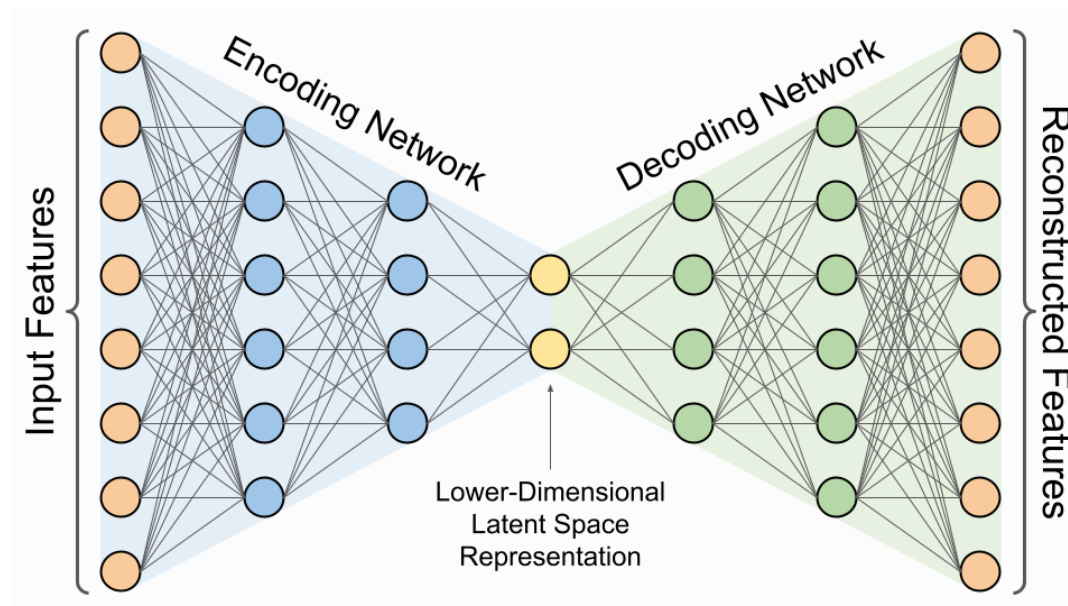


Applications :

- **Reconnaissance vocale** : Des assistants virtuels comme Siri ou Alexa utilisent des LSTM pour transcrire la parole en texte, facilitant l'interaction vocale avec les utilisateurs.
- **Traduction automatique** : Des services tels que Google Traduction emploient des RNN pour convertir du texte d'une langue à une autre, améliorant la communication internationale.

Autoencodeurs

Ces réseaux apprennent à **encoder les données en une représentation compacte** puis à les reconstruire, capturant ainsi les caractéristiques essentielles. Les autoencodeurs sont **efficaces pour extraire les caractéristiques principales des données**, facilitant ainsi l'analyse et la détection d'anomalies.

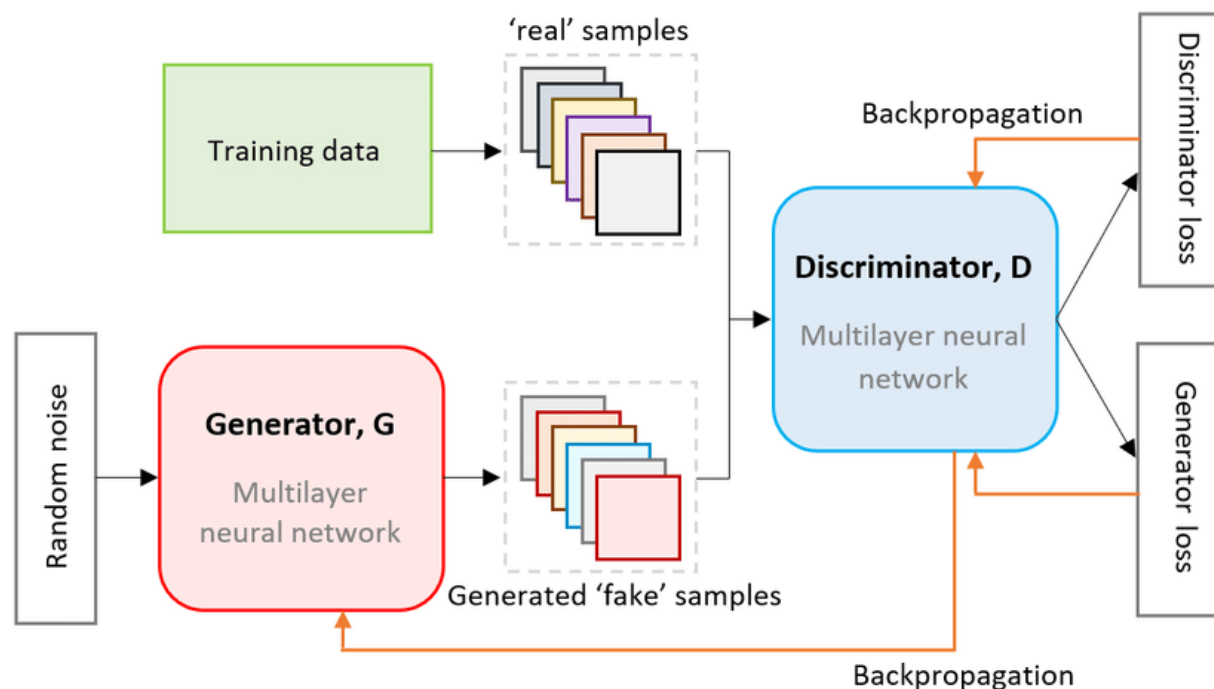


Applications :

- **Réduction de dimensionnalité** : Simplifier les données tout en conservant l'information pertinente, utile pour la compression de données.
- **Détection d'anomalies** : Identifier des données inhabituelles en comparant la reconstruction à l'entrée originale.

Réseaux Antagonistes Génératifs (GAN)

Les GANs consistent en **deux réseaux en compétition** : un générateur qui crée des données synthétiques et un discriminateur qui évalue leur authenticité. La capacité des GANs à **générer des données réalistes** les rend intéressants pour l'enrichissement de données et les applications créatives.

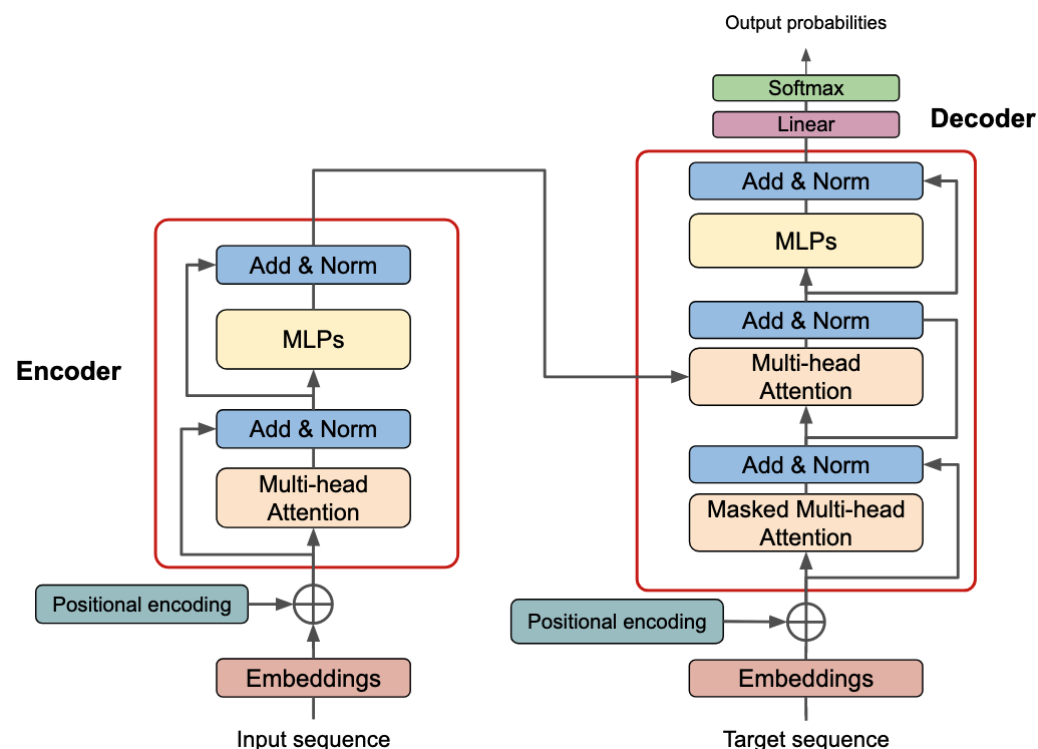


Applications :

- **Génération d'images** : Créer des images réalistes, utilisé dans l'art numérique et la conception.
- **Augmentation de données** : Produire des exemples supplémentaires pour enrichir des ensembles de données limités.

Transformers

Les Transformers utilisent des **mécanismes d'attention pour traiter des séquences de données**, capturant efficacement les relations contextuelles. Les Transformers peuvent **traiter des dépendances à long terme dans les données séquentielles**, surpassant souvent les modèles précédents comme les RNN.



Applications :

- **Traitement du langage naturel (NLP)** : Des modèles comme BERT ou GPT, basés sur les Transformers, sont utilisés pour l'analyse sémantique des textes, aidant à la compréhension contextuelle dans les chatbots ou les systèmes de recommandation.

IV. Modélisation supervisée et avancée en Deep Learning



Comprendre et implémenter des architectures profondes adaptées aux différents types de données

4.1

Réseaux de neurones
artificiels

4.2

CNN et vision par
ordinateur

4.3

RNN et traitement des
données séquentielles

4.4

Métriques d'évaluation
des performances

4.5

Optimisation et
régularisation

4.6

IA générative : GAN,
VAE

4.7

Attention, Transformers,
RAG & LLM

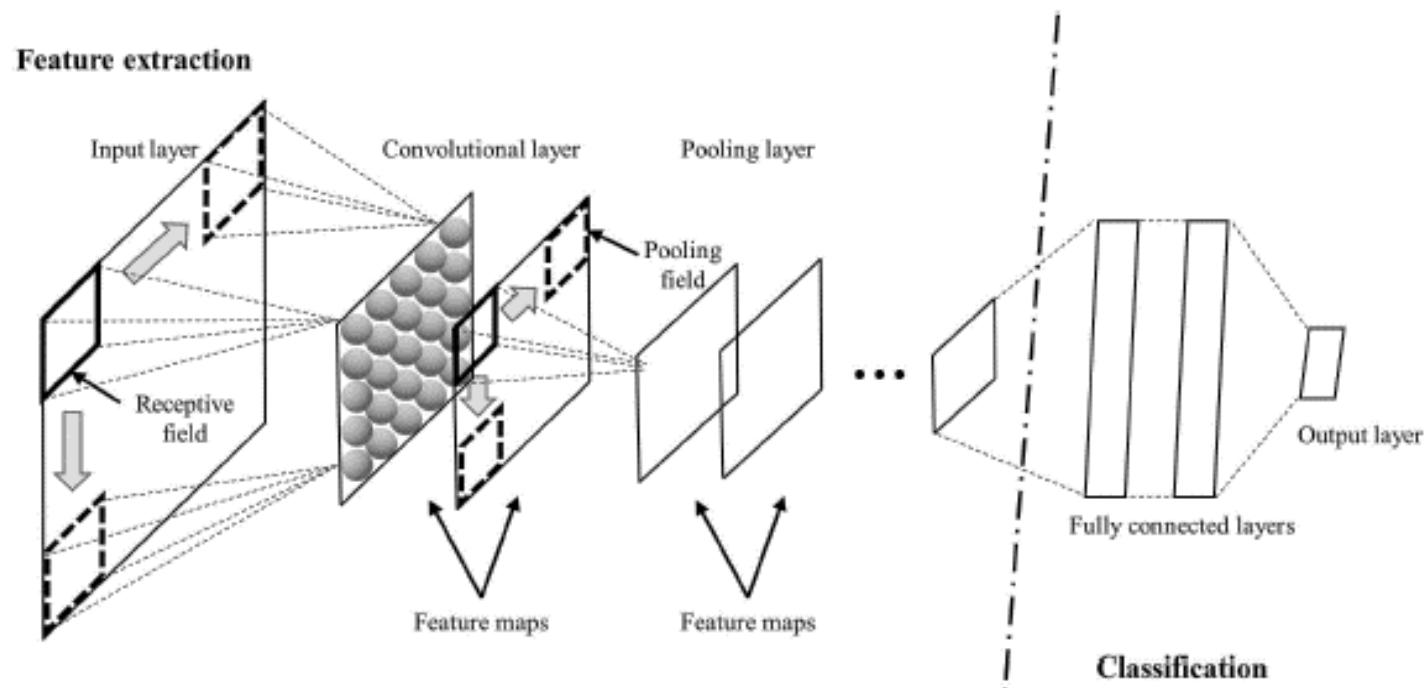
4.8

Modélisation
multimodale

Convolutional Neural Networks (CNN)

Réseau de neurones artificiels à propagation avant, **multicouches** basé sur le **principe de convolution**

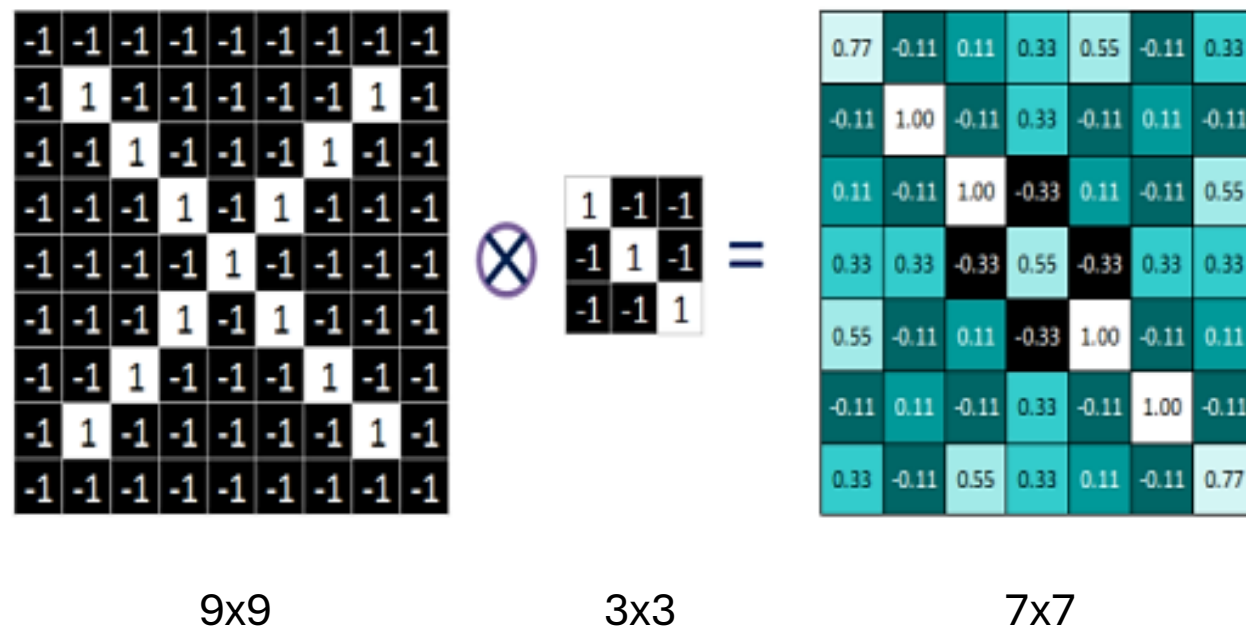
- Apprend à réaliser des tâches de classification directement à partir de données sous forme matricielle telles que images, vidéos, textes ou sons
- Deux caractéristiques principales : utilisation de filtres de convolution (aussi appelé noyau ou kernel) et mise en œuvre du pooling (réduction de taille)



Différentes couches d'un réseau convolutif

Couche de **convolution**

- Fait passer la matrice en entrée par un **ensemble de filtres convolutifs**, chaque filtre activant certaines caractéristiques précises dans l'entrée
- Les filtres sont **initialisés aléatoirement** puis mis à jour par rétropropagation du gradient
- Trois hyperparamètres : dimensions du filtre, le pas (stride) et la marge (zero-padding)



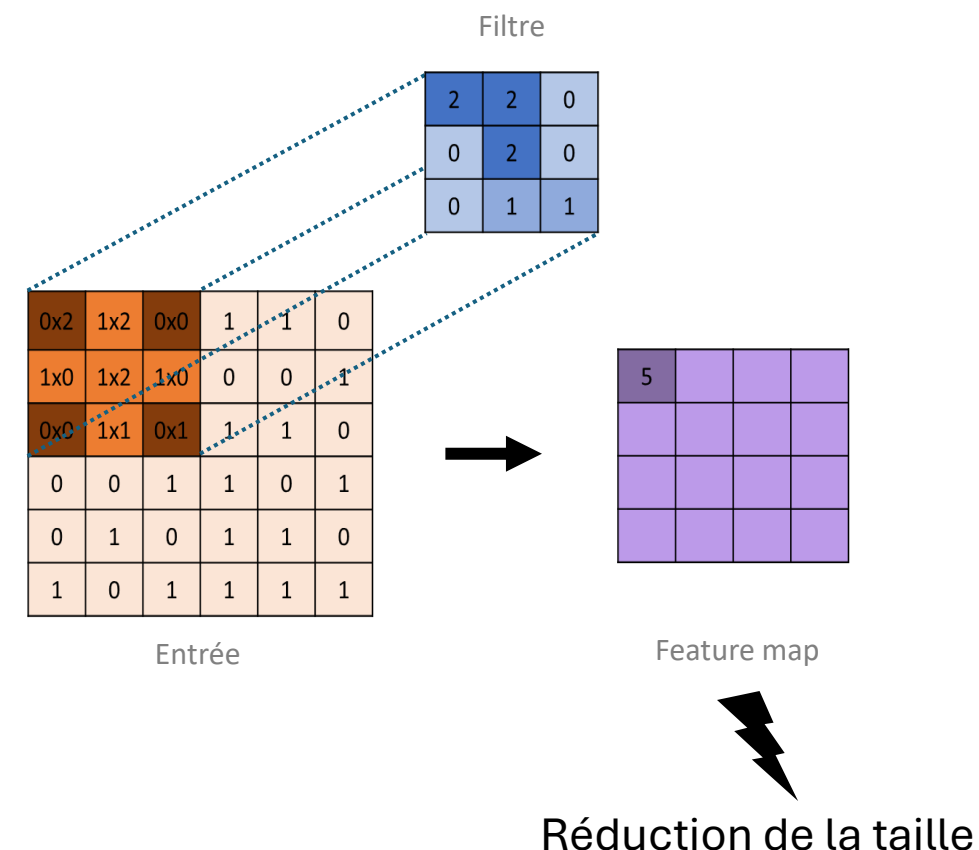
Opération de convolution

Opération permettant d'appliquer un filtre (noyau) de convolution à une image (matrice) pour en extraire certaines caractéristiques

- Le filtre recouvre une partie de la matrice image, effectue un **produit élément par élément des deux blocs superposés**, la somme de ces produits donne le résultat
- Ensuite, le filtre de convolution est décalé progressivement sur l'ensemble de la matrice image

Plusieurs paramètres pour le filtre de convolution :

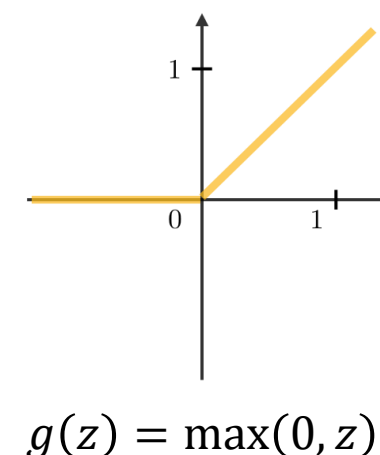
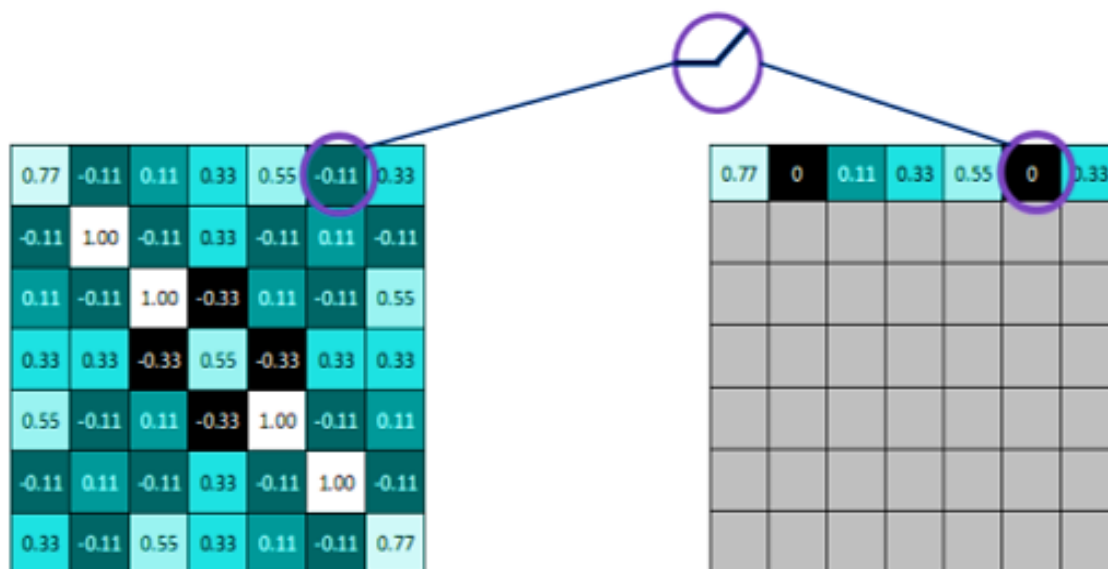
- **Dimensions** : Matrice de taille (M x N)
- **Stride** : Nombre de « cases » par lesquels le filtre se déplace après chaque opération
- **Padding** : technique consistant à ajouter des zéros à chaque côté des frontières de l'entrée



Différentes couches d'un réseau convolutif

Couche d'**activation** : **ReLU (Rectified Linear Unit)**

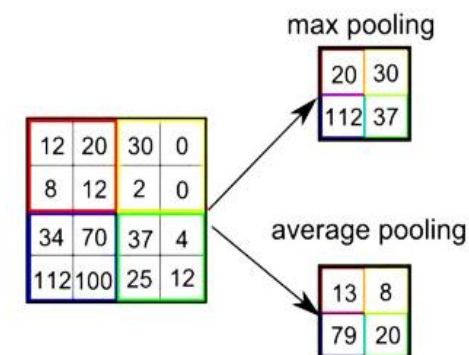
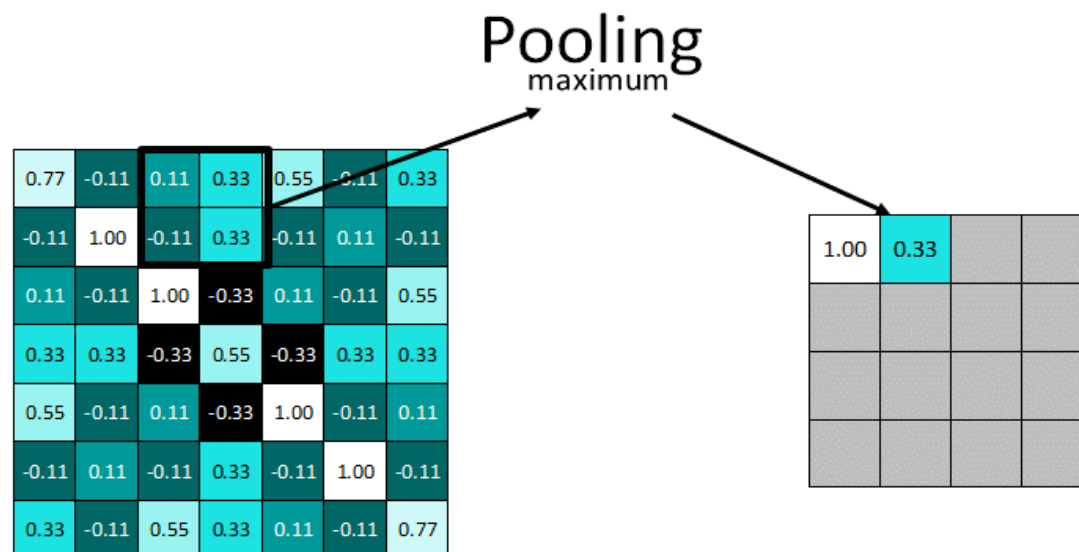
- Fonction d'activation mappant les valeurs négatives à zéro et conservant les valeurs positives
- A pour but d'introduire des complexités **non-linéaires** au réseau
- L'activation ReLU permet **d'accélérer et d'optimiser l'apprentissage** sans faire une différence significative à la généralisation de précision



Différentes couches d'un réseau convolutif

Couche de mise en commun : Pooling

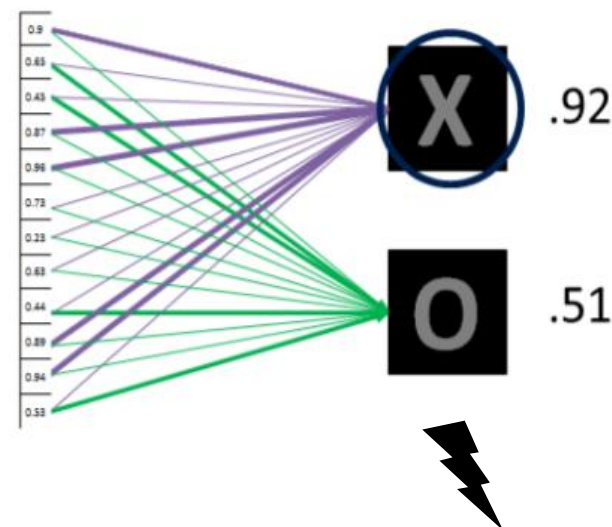
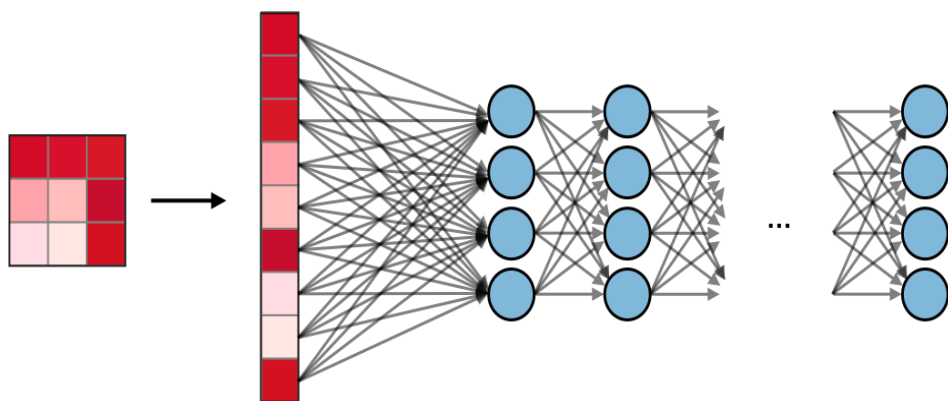
- Effectue une opération de **sous-échantillonnage** réduisant ainsi la quantité de paramètres
- Améliore l'efficacité du réseau et permet d'**éviter le sur-apprentissage**
- Les plus populaires : **max pooling** et **average pooling**
- Un hyperparamètre : forme du pooling (M x N)



Différentes couches d'un réseau convolutif

Couche **entièrement connectée** : **Fully Connected**

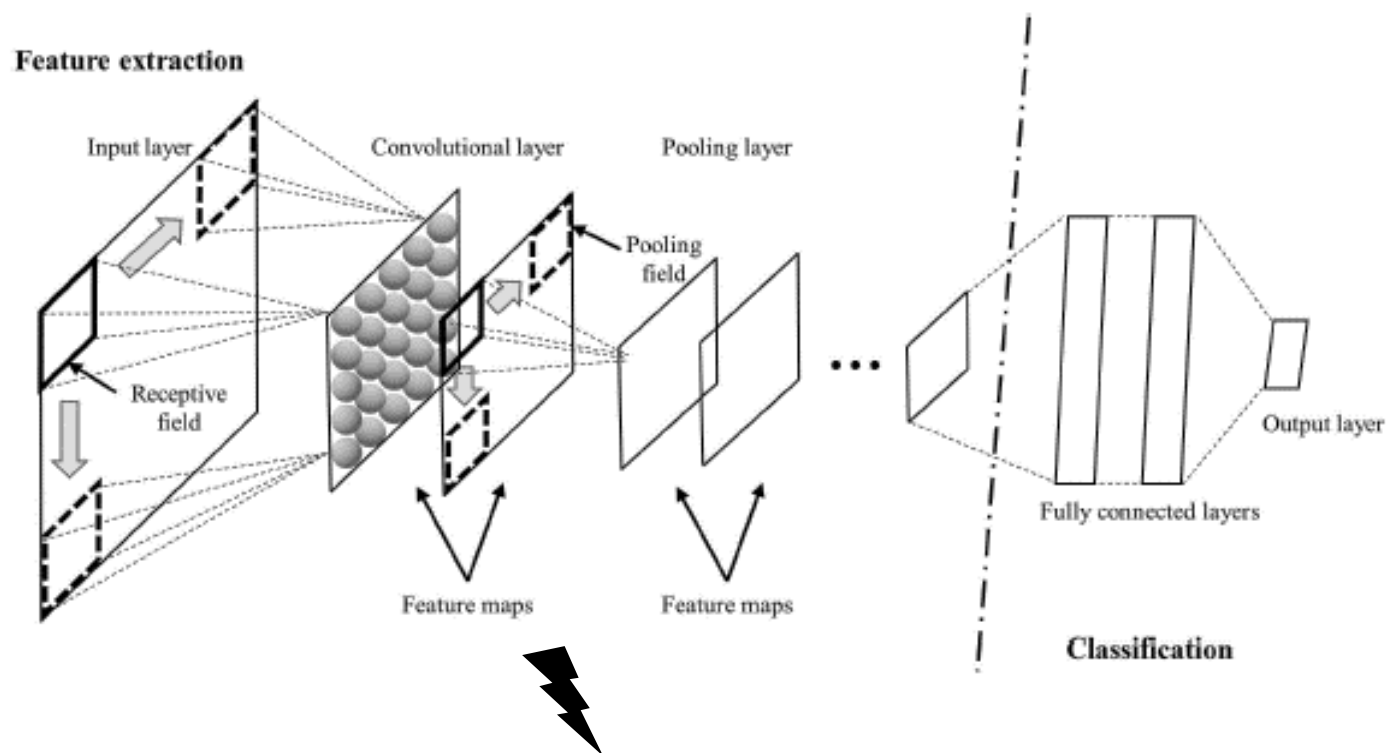
- Présente **en fin d'architecture** et s'applique sur une entrée préalablement aplatie (vecteur)
- Chaque entrée est connectée à tous les neurones (combinaison linéaire) et produit un vecteur en sortie
- A la sortie, une **fonction de perte** spécifie comment l'entraînement du réseau pénalise l'écart entre le signal prévu et réel



Perte par entropie
croisée sigmoïde

Convolutional Neural Networks (CNN)

- Les entrées brutes sont **filtrées**, **rectifiées** et **mises en commun** → crée un ensemble rétréci et filtré par caractéristiques qui deviennent plus grandes et plus complexes, et **les données deviennent plus compactes**
- Les couches inférieures représentent des **caractéristiques simples**
- Les couches supérieures représentent des **caractéristiques beaucoup plus complexes**



Couches répétées plusieurs fois

Apprentissage par rétropropagation du gradient

Pour une sortie différente de celle attendue (**erreur**), l'algorithme est réglé de telle sorte que le modèle change les paramètres (**poids**) chaque fois que la sortie n'est pas celle attendue.

Les poids des connexions synaptiques (valeurs de filtres de convolution) ainsi que les poids des couches entièrement connectées sont automatiquement appris par **rétropropagation des erreurs**

- **Algorithme itératif** basé sur la descente du gradient qui décompose l'erreur grâce au **théorème de dérivation des fonctions composées** la propage de la dernière vers la première couche, couche par couche
- La mise à jour des poids se fait en alternant une **phase forward** (ou montante), dans laquelle les sorties des couches intermédiaires sont mises à jour, et une **phase backward** (ou descendante), dans laquelle le gradient de l'erreur par rapport aux poids d'une couche est calculé à partir du gradient de l'erreur par rapport aux poids d'une couche supérieure

Apprentissage par rétropropagation du gradient

L'apprentissage consiste à **corriger les erreurs selon l'importance** des éléments qui ont justement participé à la réalisation de ces erreurs

- Les poids synaptiques qui contribuent à engendrer une erreur importante se verront modifiés de manière plus significative que les poids qui ont engendré une erreur marginale

$$w_j^{(t+1)} = w_j^{(t)} - \eta \frac{\partial \text{Erreur}(f(x^{(i)}), y^{(i)})}{\partial w_j}$$

L'algorithme d'apprentissage dépend largement de la **fonction d'erreur** (ou de coût - aussi appelé *loss*) mais aussi du **taux d'apprentissage**

- Plusieurs choix selon le problème par exemple l'**erreur quadratique** pour la régression, l'**entropie croisée** pour la classification

Vision par ordinateur : compétition ILSVRC

ImageNet¹ : Base de données : **15 millions d'images** haute résolution et **22.000 catégories**



ImageNet Large Scale Recognition Vision Challenge (ILSVRC) :

- Environ **1.000 images** pour chacune des **1.000 catégories**
- Environ 1.200.000 images d'entraînement
- 50.000 images de validation
- 100.000 images de test

¹ImageNet : https://image-net.org/static_files/files/imagenet_ilsvrc2017_v1.0.pdf
<https://www.image-net.org/download.php>

Réseau de neurones profond (Deep Neural Network)

L'idée des réseaux de neurones profonds est assez simple : **empiler un tas de couches**

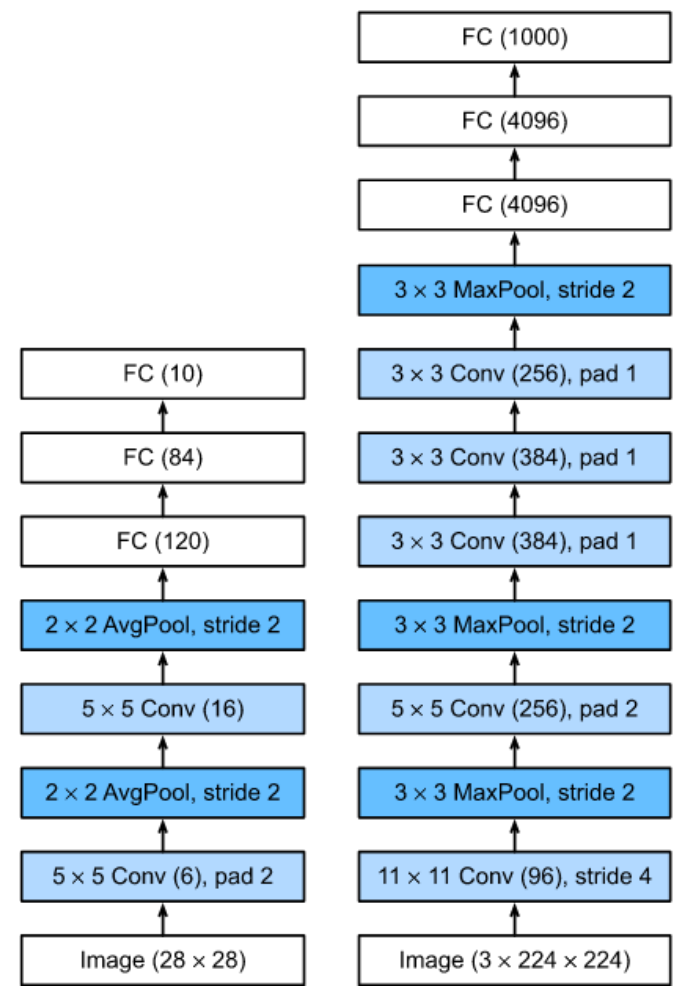
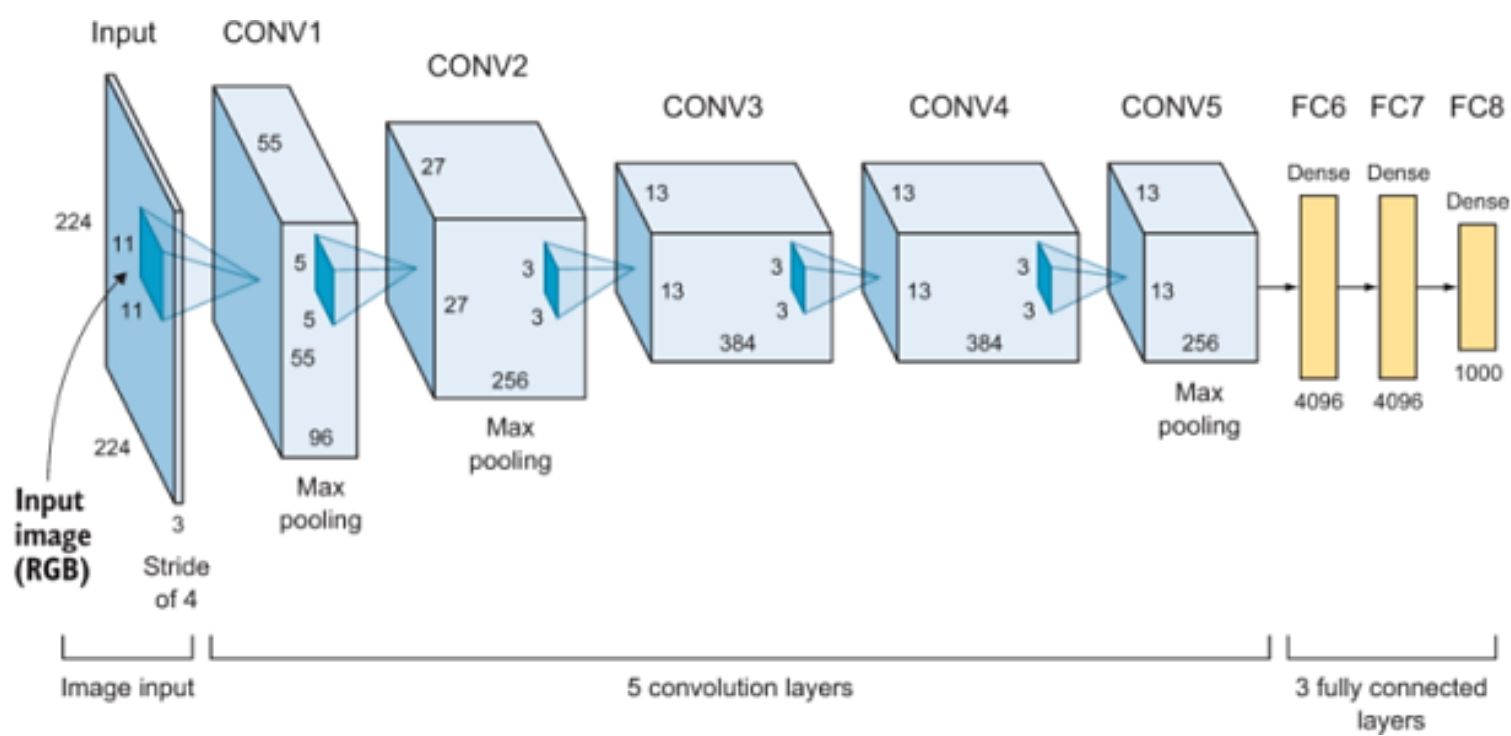
- Un réseau de neurones est composé d'une couche d'entrée, une couche de sortie et au moins une couche cachée entre les deux
- Plus le nombre de couches cachées est élevé, plus un réseau est dit « profond »
- Les **performances** peuvent **varier énormément** selon les architectures et les choix d'hyperparamètres

*La suite correspond aux architectures CNN importantes qui étaient à un moment donné (ou actuellement) les modèles de base sur lesquels de nombreux projets de recherche et systèmes déployés ont été construits. Chacun de ces réseaux a été brièvement une architecture dominante et beaucoup ont été lauréats ou finalistes de la compétition **ILSVRC**, qui sert de baromètre des progrès de l'apprentissage supervisé en vision par ordinateur depuis 2010.*

Ils sont le produit de l'intuition, de quelques connaissances mathématiques et de beaucoup d'essais et d'erreurs. Ces modèles sont présentés dans l'ordre chronologique.

Architectures de CNN : AlexNet

AlexNet¹, qui est un CNN à 8 couches dont 5 couches de convolution (avec ReLU, Max pooling, Dropout), a remporté **ILSVRC 2012** avec une marge phénoménale. Première architecture de réseau de neurone à remporter la compétition.



¹Alex Krizhevsky, Ilya Sutskever, Geoffrey E Hinton, « Imagenet classification with deep convolutional neural networks, » 2012.

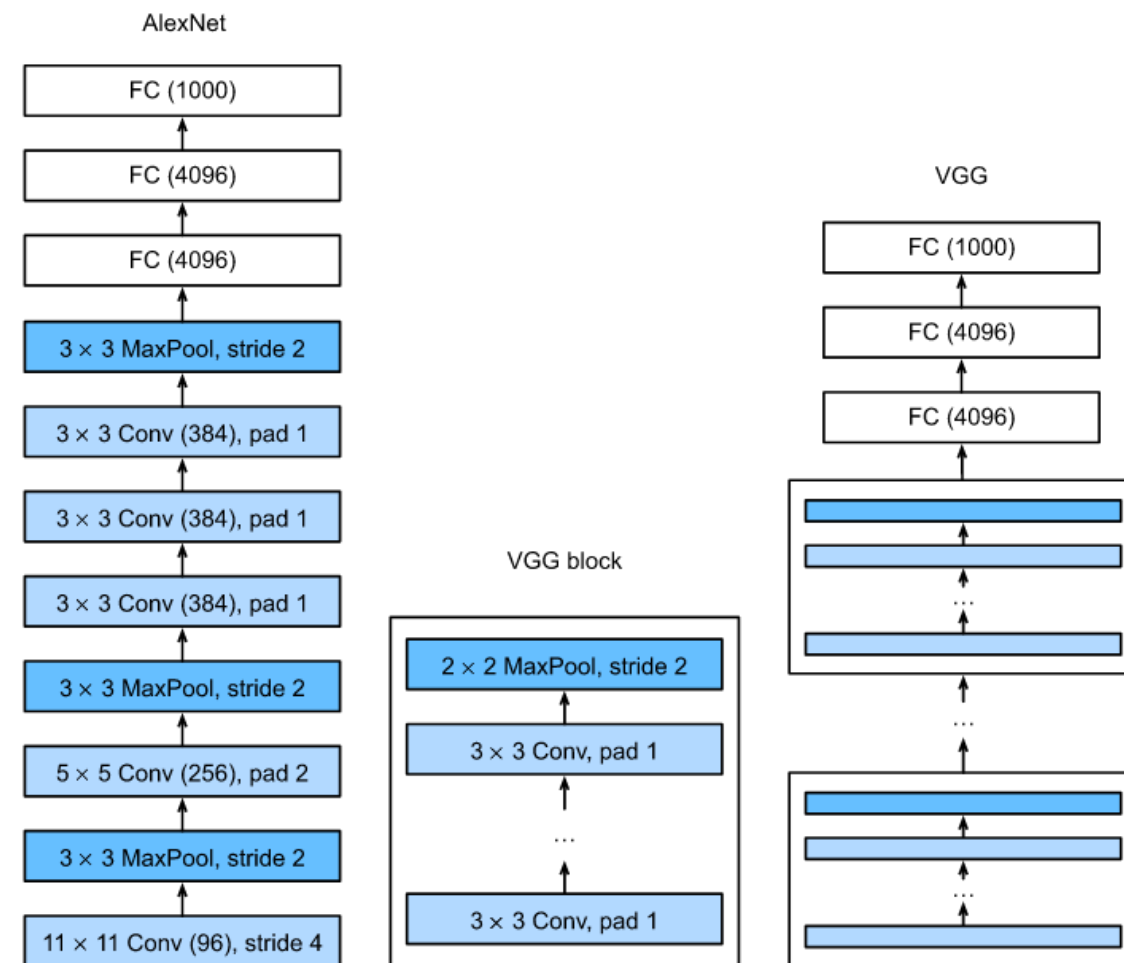
De LeNet (gauche) à AlexNet (droite)

Architectures de CNN : VGGNet

Avec VGGNet¹, la logique derrière les CNN est passée de couches entières de neurones à des **blocs**, répétant des motifs de couches. **Structures répétées**, facile d'implémentation en utilisant des **boucles**.

VGGNet a plusieurs particularités

- Concept de bloc/modules répétitifs : un bloc étant composer d'une séquence de couches convolutives, suivie d'une couche de max pooling **pooling 2x2** avec une **foulée (stride) de 2** (divisant par deux la résolution après chaque bloc) pour le sous-échantillonnage spatial
- Plusieurs **noyaux de petite taille** : les noyaux convolutifs **3x3** ont un **padding de 1** (en gardant la hauteur et la largeur)
- Diminution du nombre de paramètres.



¹ K. Simonyan, & A. Zisserman, « Very deep convolutional networks for large-scale image recognition, » arXiv preprint, 2014. DOI : arXiv:1409.1556.

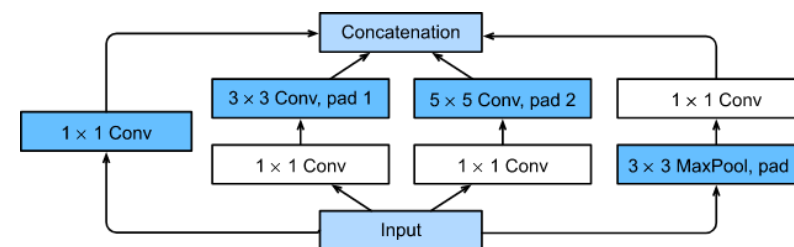
Architectures de CNN : GoogleNet (Inception)

GoogleNet¹ est le gagnant de l'ILSVRC 2014 avec **22 couches**, 12 fois moins de paramètres que AlexNet et de nouveaux concepts : le **module Inception** et le **global average pooling** pour générer ses estimations (et éviter une pile de couches entièrement connectées à la fi).

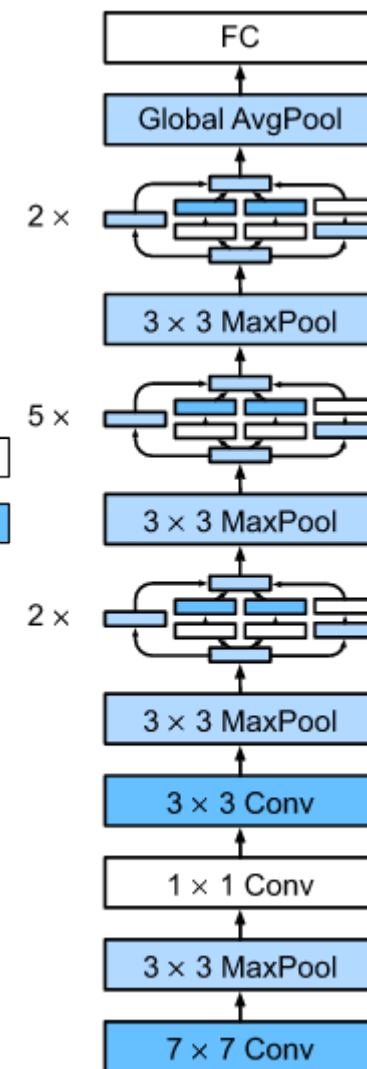
L'un des objectifs est de répondre à la question de savoir quelles **tailles de noyaux de convolution** sont les meilleurs.

- Les réseaux populaires précédents utilisaient des choix **aussi petits que 1x1** et **aussi grands que 11x11**.

Une découverte est qu'il peut parfois être avantageux d'employer une **combinaison de noyaux de différentes tailles**.



Inception block

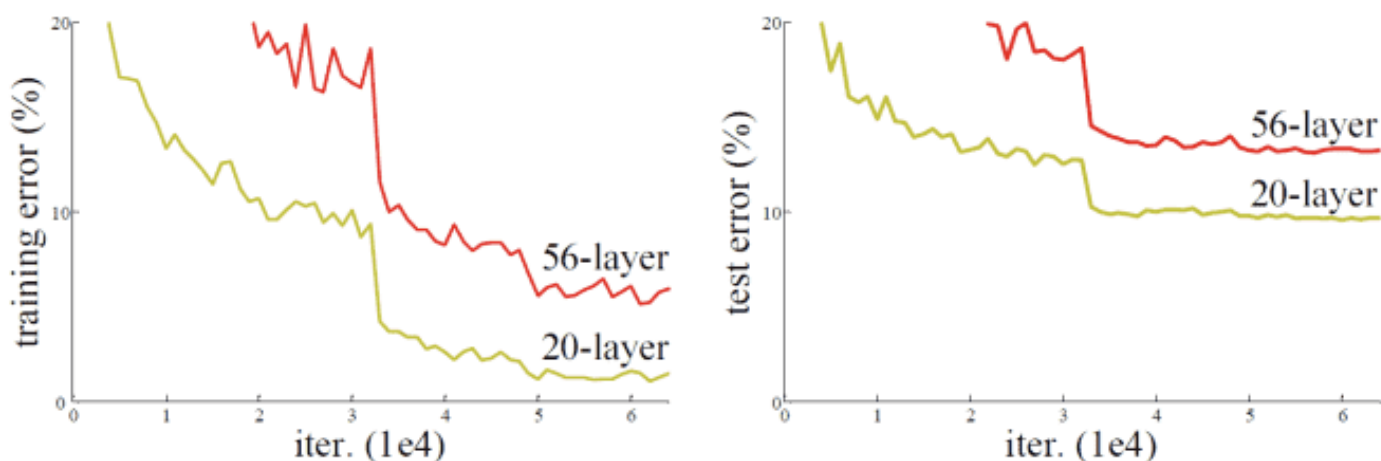


¹ C. Szegedy et al., « Going deeper with convolutions, » Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 1–9, 2015.

Réseau de neurones très profond

Les réseaux de neurones très profonds (plusieurs dizaines de couches) se sont révélés difficiles à entraîner en raison du **problème du gradient de disparition / explosion**.

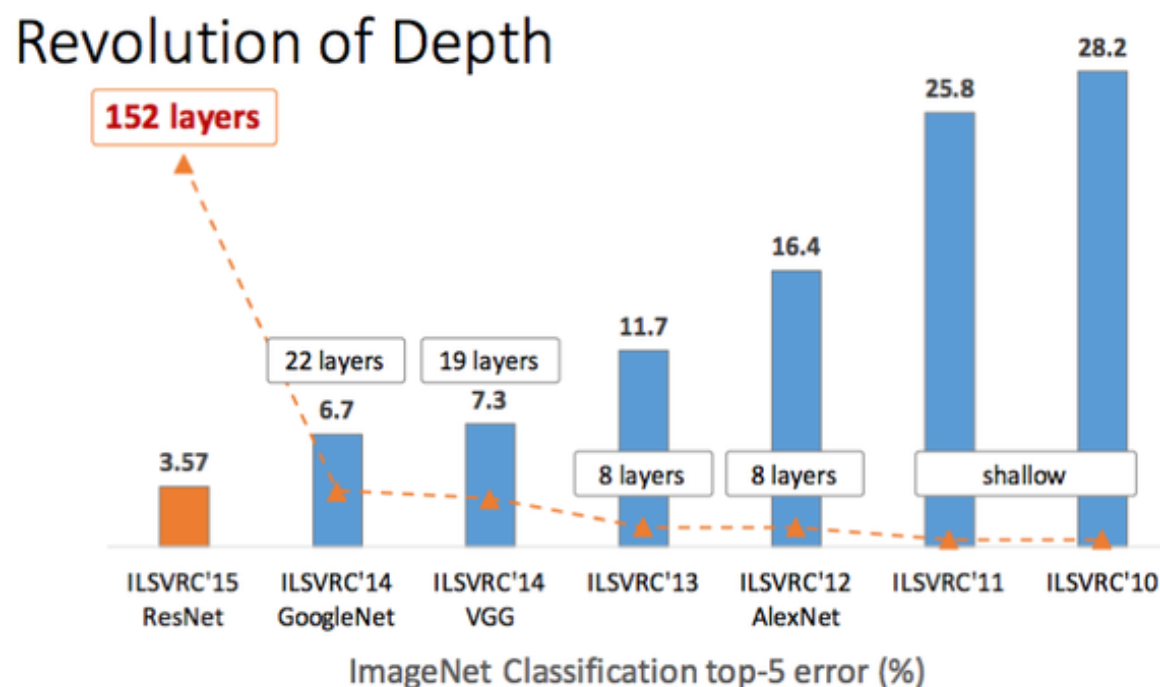
- En empilant plusieurs couches dans un réseau de neurones à convolution, l'erreur d'entraînement devrait en théorie diminuer, mais en pratique, l'ajout de couches supplémentaires dans un CNN (ce qui rend le plus profond) entraîne une augmentation de l'erreur d'entraînement au lieu de la diminution.
- Cela n'est pas causé par le surapprentissage. Le surapprentissage signifie une bonne précision d'entraînement mais une mauvaise précision des tests. Le problème ici est un problème d'optimisation. Les modèles plus profonds sont plus difficiles à optimiser car avec l'augmentation de la profondeur du réseau, la précision est saturée puis se dégrade rapidement (l'erreur d'apprentissage augmente).



He, Kaiming, X. Zhang, S. Ren, and J. Sun. "Deep residual learning for image recognition." In Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 770-778. 2016.

Architectures de CNN : ResNet

ResNet est le gagnant de l'**ILSVRC 2015** en **classification, détection et localisation d'images**, ainsi que le gagnant de **la détection et de la segmentation MS COCO 2015**¹. A été une révolution dans la façon de concevoir et entraîner les réseaux de neurones très profonds.



¹ Microsoft COCO (Common Objects in Context) : <https://cocodataset.org>

Architectures de CNN : ResNet

Les ResNet sont des CNN profonds caractérisés par l'emploi de **connexions saute-couches** (ou de saut) qui s'appuient sur des constructions connues à partir de cellules pyramidales du cortex cérébral.

Les connexions saute-couches sont des raccourcis pour sauter par-dessus certaines couches. Elles sont essentiellement un mappage d'identité où l'entrée de la couche précédente est ajoutée directement à la sortie de la(les) couche(s) suivante(s).

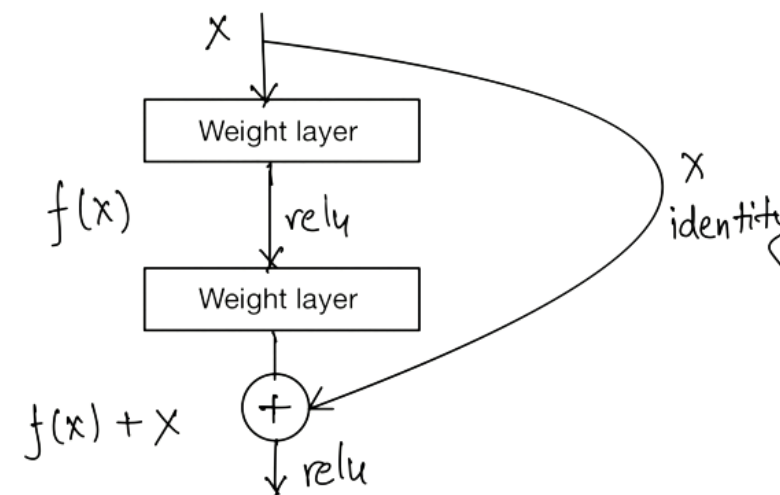
Blocs résiduels

Les blocs résiduels constituant les ResNet ont une connexion directe qui omet certaines couches du modèle

- Cette “**connexion de saut**” est le cœur des blocs résiduels

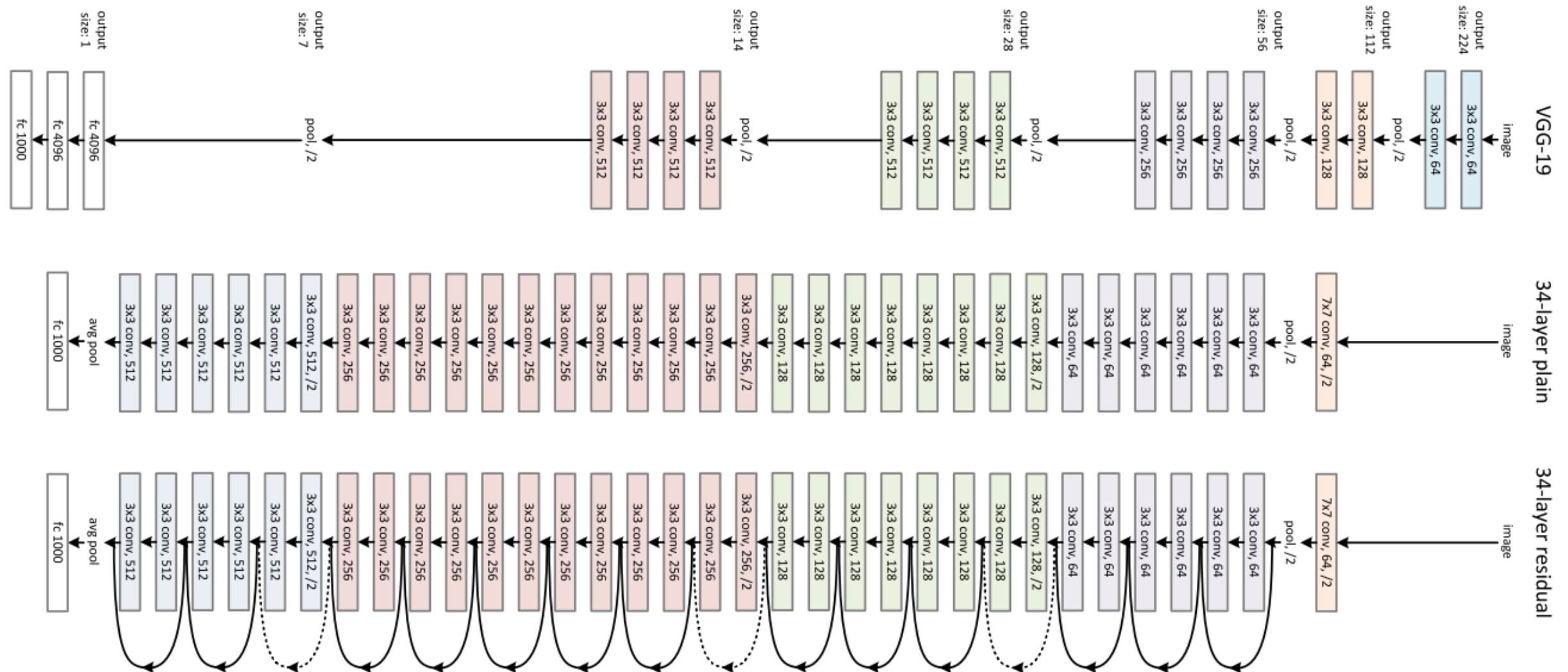
L'introduction des blocs résiduels atténue le problème de **disparition / explosion du gradient** des réseaux très profonds

- La connexion de saut aide si une couche nuit aux performances de l'architecture, alors il sera ignoré par régularisation.
- Permet un **chemin de raccourci alternatif** pour que le gradient s'écoule jusqu'aux couches les plus profondes



- He, Kaiming, X. Zhang, S. Ren, and J. Sun. "Deep residual learning for image recognition." In Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 770-778. 2016.
- A. Zaeemzadeh, N. Rahnavard, and M. Shah, "Norm-preservation: Why residual networks can become extremely deep?" IEEE transactions on pattern analysis and machine intelligence, 2020.

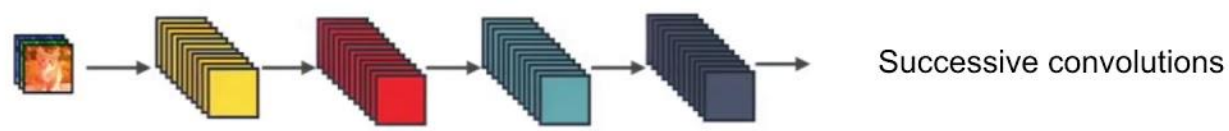
Architectures de CNN : ResNet



Architectures de CNN : DenseNet

DenseNet est le gagnant de l'**ILSVRC 2018**, en exploitant l'idée des connexions sautes-couches du ResNet. **Toutes les couches sont directement reliées entre elles**, assurant un flux maximal de d'informations et de gradient.

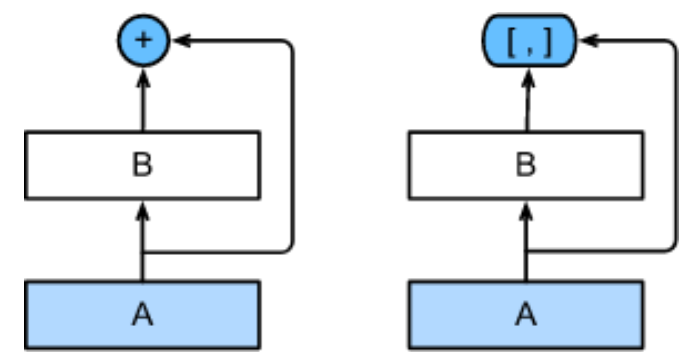
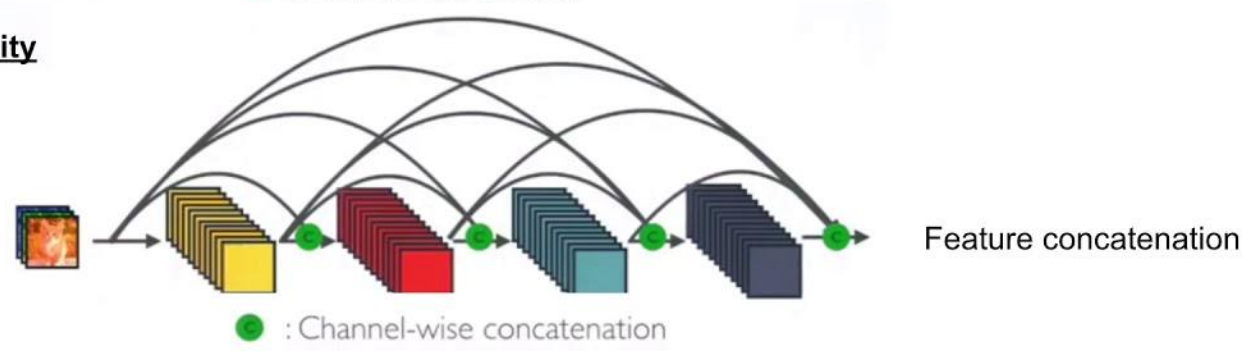
Standard Connectivity



Resnet Connectivity



DenseNet Connectivity



IV. Modélisation supervisée et avancée en Deep Learning



Comprendre et implémenter des architectures profondes adaptées aux différents types de données

4.1

Réseaux de neurones
artificiels

4.2

CNN et vision par
ordinateur

4.3

RNN et traitement des
données séquentielles

4.4

Métriques d'évaluation
des performances

4.5

Optimisation et
régularisation

4.6

IA générative : GAN,
VAE

4.7

Attention, Transformers,
RAG & LLM

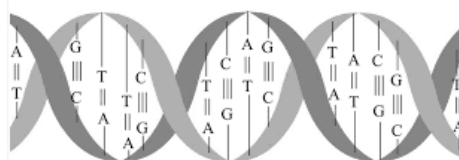
4.8

Modélisation
multimodale

Données séquentielles

Une **donnée séquentielle** est un type de donnée organisée **dans un ordre spécifique**, où **l'ordre des éléments a de l'importance**. Cela signifie que chaque élément dépend souvent des précédents ou doit être traité dans une certaine séquence. La **donnée séquentielle** est souvent **liée à une dimension temporelle ou logique** (comme les mots dans une phrase ou les événements dans le temps).

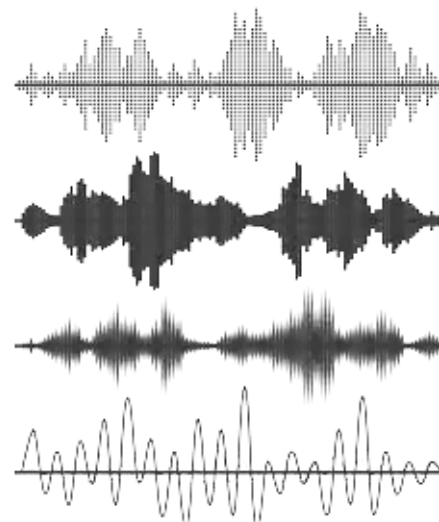
L'ADN



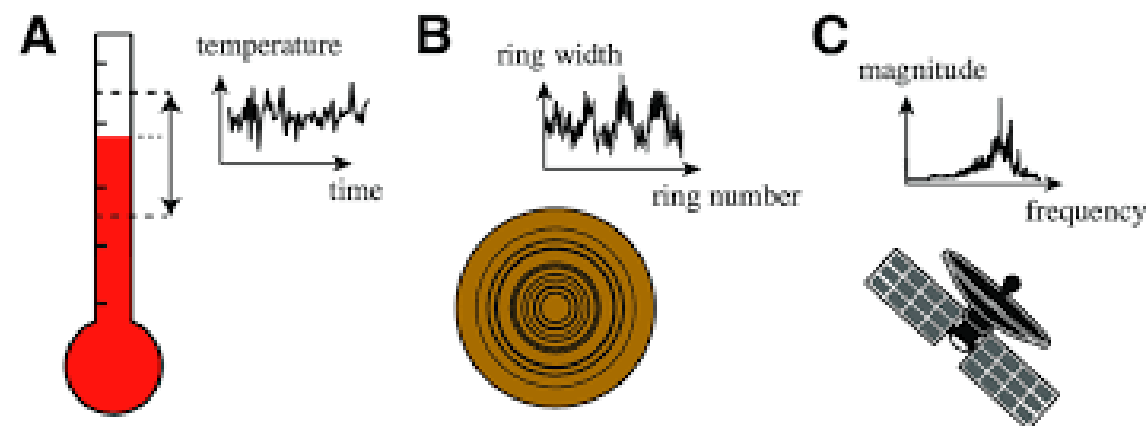
Texte



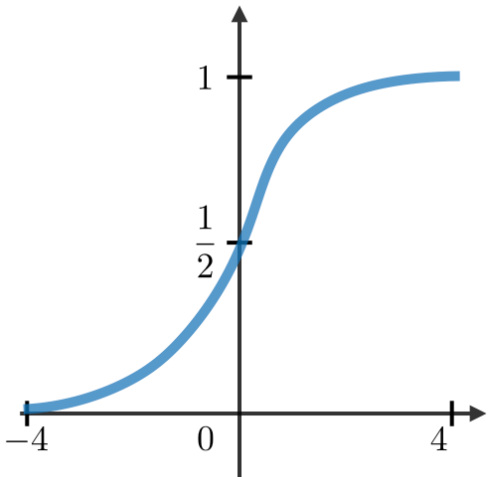
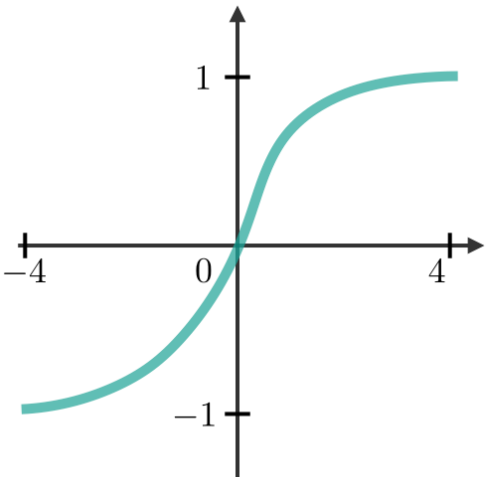
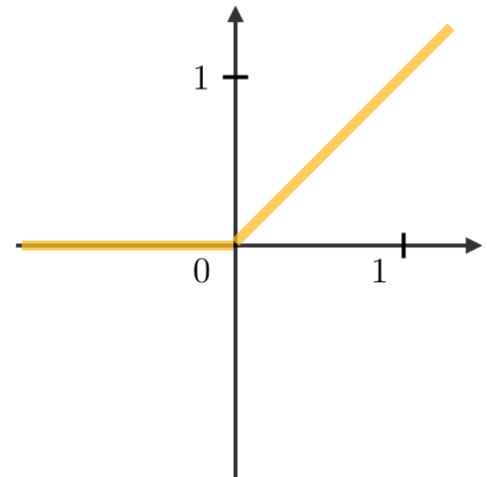
Audio (son)



Données temporelles (time series)



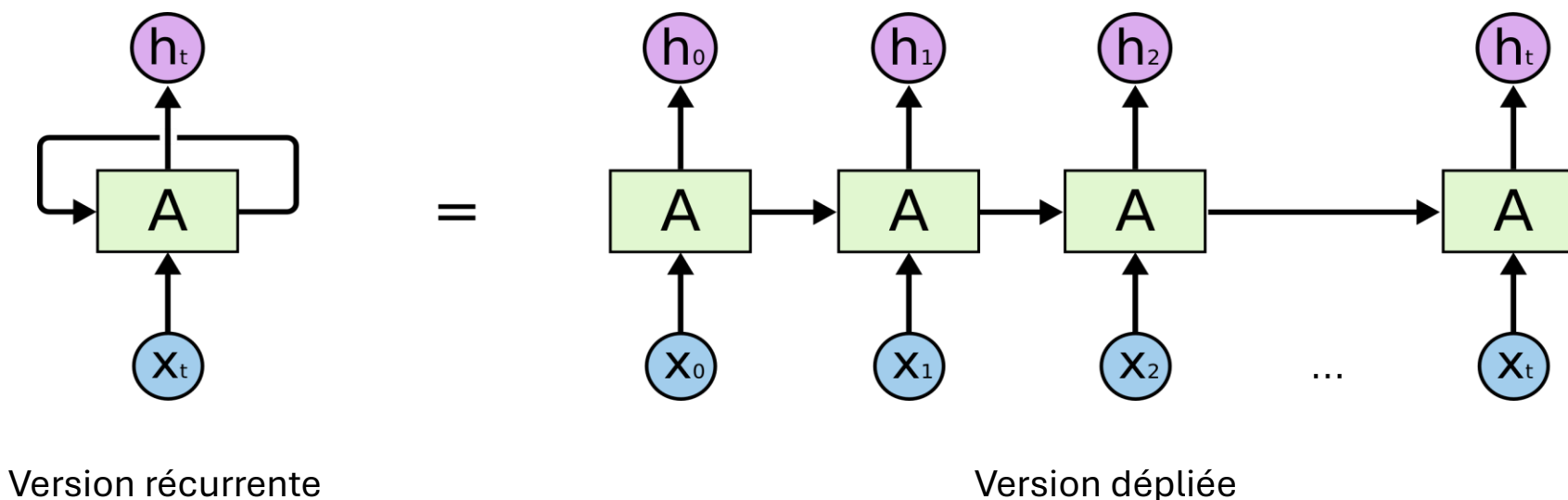
Fonctions d'activation

Sigmoïde	Tanh	ReLU
$g(z) = \frac{1}{1 + e^{-z}}$	$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$g(z) = \max(0, z)$
		
Transforme l'entrée en une valeur entre 0 et 1	Transforme l'entrée en une valeur entre -1 et 1	Renvoie 0 pour les entrées négatives et la valeur d'entrée pour les positives

Recurrent Neural Networks (RNN)

Réseau de neurones artificiels basé sur les principes de fenêtre glissante et [connexions récurrentes](#)

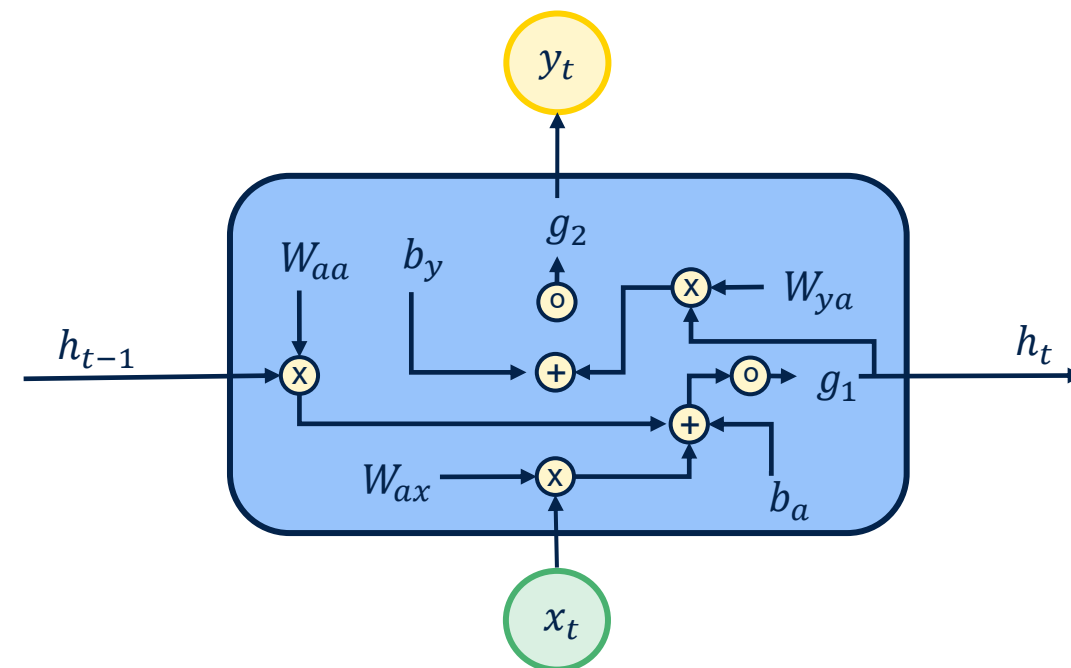
- Réalise des tâches de reconnaissance de formes, traduction automatique directement à partir de données séquentielles
- Deux caractéristiques principales : utilisation des prédictions antérieures via les connexions récurrentes et entrées de taille variable



MACHINE & DEEP LEARNING POUR L'INGÉNIERIE DES DONNÉES

60

- Les prédictions antérieures sont utilisées comme entrées, par le biais d'états cachés
- Les paramètres importants sont l'état caché, les poids synaptiques, et la façon de les ajuster
- Apprendre revient à déterminer les coefficients synaptiques les plus adaptés pour les exemples présentés

 W_{aa}, W_{ax}, W_{ya} : Poids

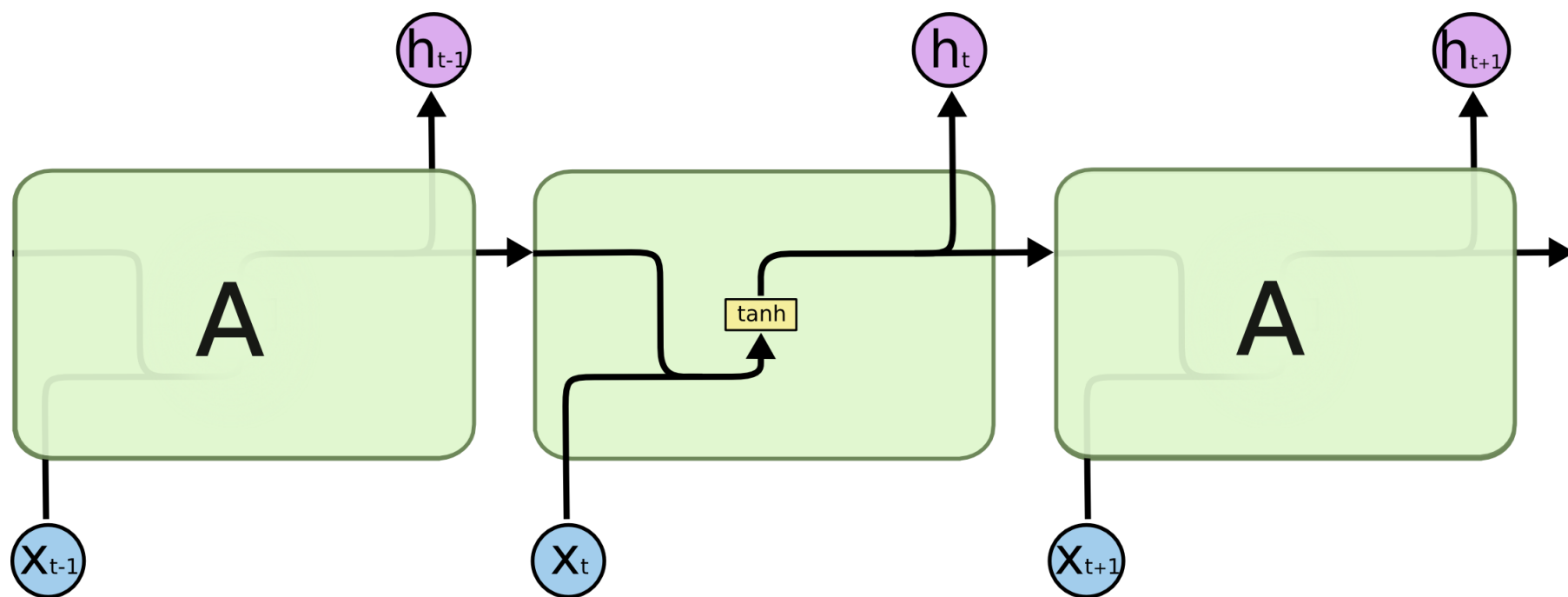
x_t : Entrée y_t : Sortie

 h_t : État caché b_a, b_v : Biaisi

Architecture d'un RNN

Tous les réseaux récurrents ont la forme d'une **chaîne de modules répétitifs** de réseau de neurones

- Dans les RNN standard, le module répétitif a une structure très simple, généralement **une seule couche de tanh**



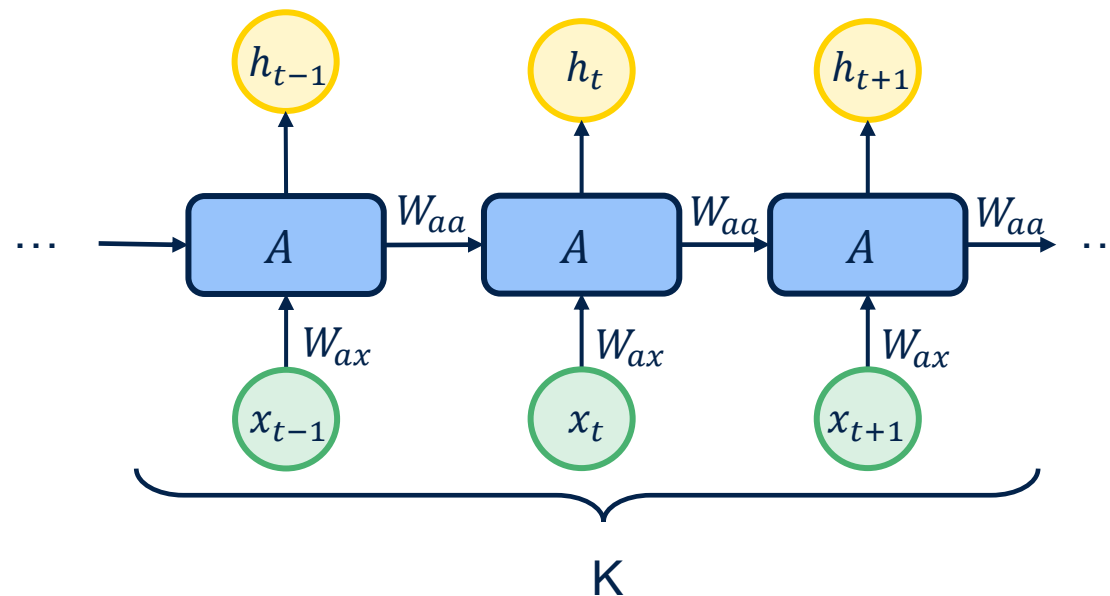
$$h_t = \tanh(W_{aa}h_{t-1} + W_{ax}x_t + b_a)$$

Apprentissage profond et Rétropropagation à travers le temps

L'apprentissage des réseaux de neurones feedforward se fait par rétropropagation du gradient en rétropropageant le gradient de l'erreur à travers les différentes couches de poids du réseau

La **Backpropagation Through Time (BPTT)** applique la rétropropagation standard sur un RNN déplié K fois à travers le temps

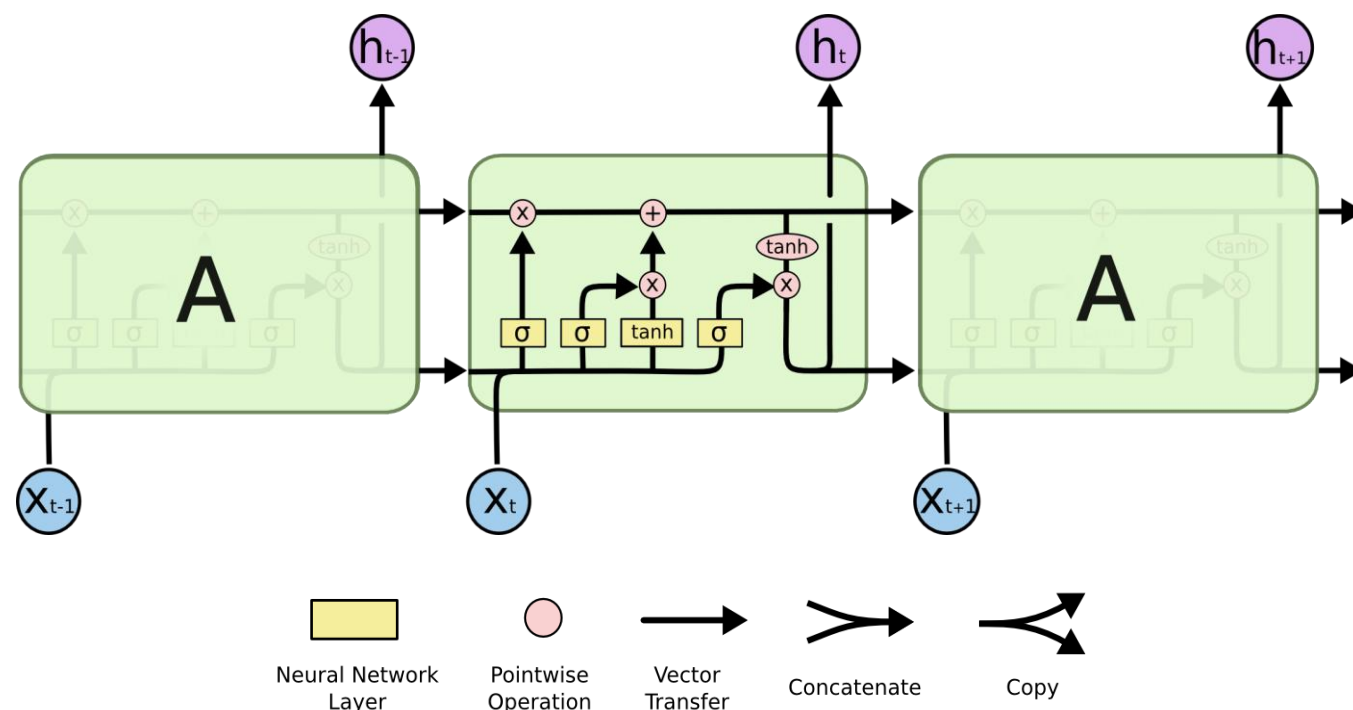
- Un réseau déplié K fois augmente de K la profondeur du réseau et celui-ci ne pourra "mémoriser" qu'un passé limité à K échantillons
- À la différence de la rétropropagation standard, le BPTT force les poids W_{aa} , W_{ax} à être identiques pour chaque pas de temps du réseau déplié



Réseau Long Short Term Memory (LSTM)

Un LSTM est un type de **RNN amélioré** explicitement conçus pour gérer les dépendances à long terme → Répondent au problème de **disparition/explosion du gradient**

- Chaque unité computationnelle est liée à un **état caché h** mais aussi à un **état c de la cellule** qui joue le rôle de **mémoire**
- Le problème de disparition du gradient est surmonté en utilisant des **portes multiplicatives** qui imposent un flux d'erreur constant à travers les états cellules (mémoire)



Réseau Long Short Term Memory (LSTM)

Le LSTM est capable de mémoriser des **relations à long terme** dans une séquence, d'oublier les informations jugées inutiles et de se concentrer sur ce qui est important dans la séquence. Il est donc très adapté aux **phrases longues, discours audio, ou séries temporelles complexes**.

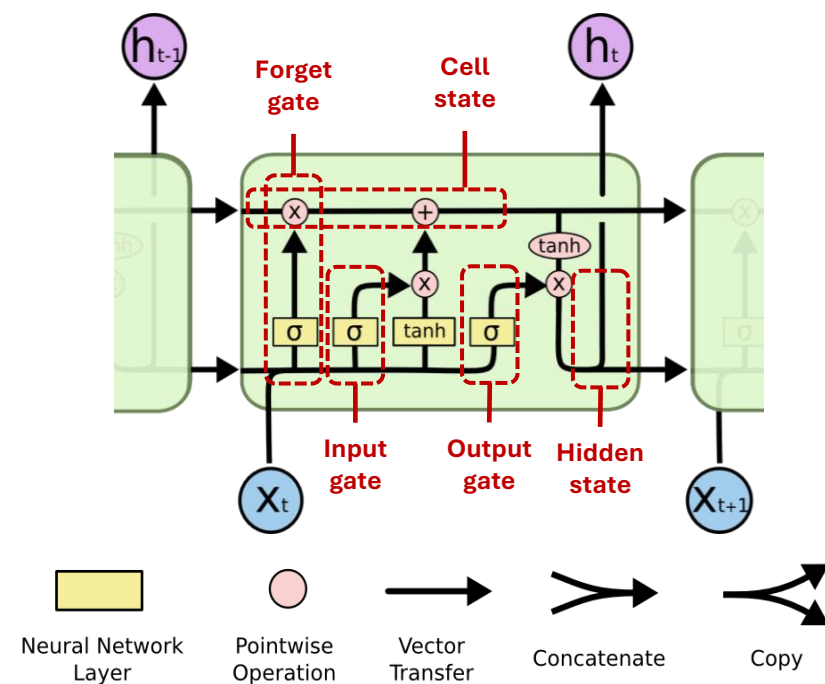
Un LSTM contient :

1. Cell state (mémoire à long terme) → ce qu'il retient de toute la séquence.

2. Hidden state (mémoire de court terme) → ce qu'il utilise à l'instant t .

3. Trois portes :

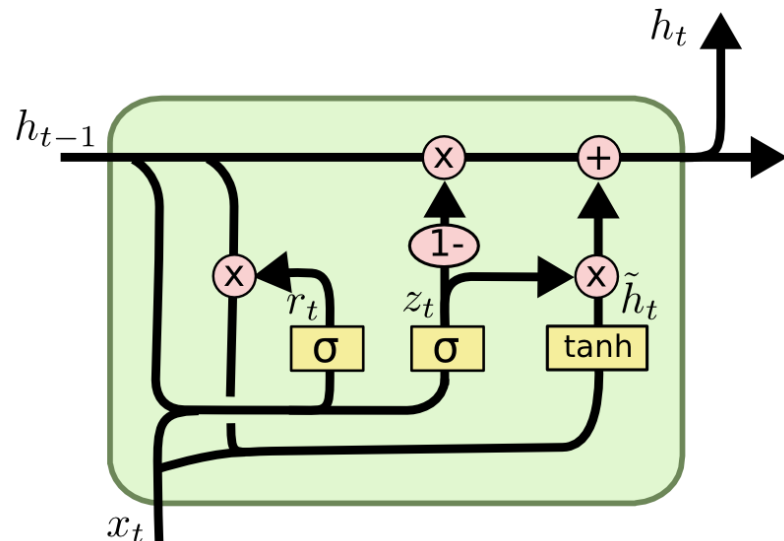
1. **Forget gate** : décide ce qu'on oublie.
2. **Input gate** : décide ce qu'on ajoute à la mémoire.
3. **Output gate** : décide ce qu'on sort comme résultat.



Gated Recurrent Units (GRU)

Variante LSTM : Gated Recurrent Unit (**GRU**) → **modèle plus simple** que les modèles LSTM standard

- Combine les portes d'oubli et d'entrée en une seule **porte de mise à jour** (update gate)
- La porte de sortie est remplacée par une **porte de réinitialisation** (reset gate)
- Fusionne également l'état de la cellule et l'état caché



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

IV. Modélisation supervisée et avancée en Deep Learning



Comprendre et implémenter des architectures profondes adaptées aux différents types de données

4.1

**Réseaux de neurones
artificiels**

4.2

**CNN et vision par
ordinateur**

4.3

**RNN et traitement des
données séquentielles**

4.4

**Métriques d'évaluation
des performances**

4.5

**Optimisation et
régularisation**

4.6

**IA générative : GAN,
VAE**

4.7

**Attention, Transformers,
RAG & LLM**

4.8

**Modélisation
multimodale**

Evaluer pour mieux régner



Un modèle non évalué est comme une voiture sans compteur : vous roulez, mais à quelle Vitesse ?

Pourquoi évaluer un modèle ?

L'évaluation, c'est le **crash-test** d'un modèle et est utile pour :

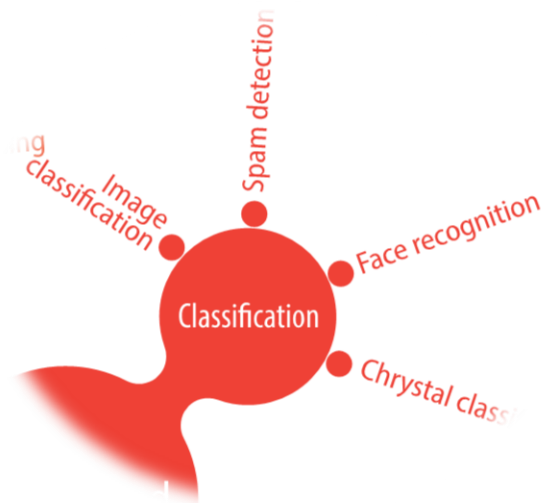
1. Choisir le **bon modèle parmi plusieurs** : comment comparer objectivement différents modèles ?
2. **Améliorer un modèle** en identifiant et ciblant ses faiblesses
3. Éviter le **sur-apprentissage** (overfitting) et le **sous-apprentissage** (underfitting) afin d'avoir des modèles performants en entraînement et en production
4. **Communiquer la** performance aux parties prenantes en s'appuyant sur des bases objectives



Un médecin ne se fierait pas à un thermomètre qui dit que tout le monde a 36.5°C

Les grands types de problèmes

Le choix des métriques se fait toujours en fonction du problème à résoudre et des enjeux métiers.



Prédire une classe (binaire ou multi-classe)
Ex: classification d'images



Prédire des valeurs continues (numériques)
Ex : prédiction de prix

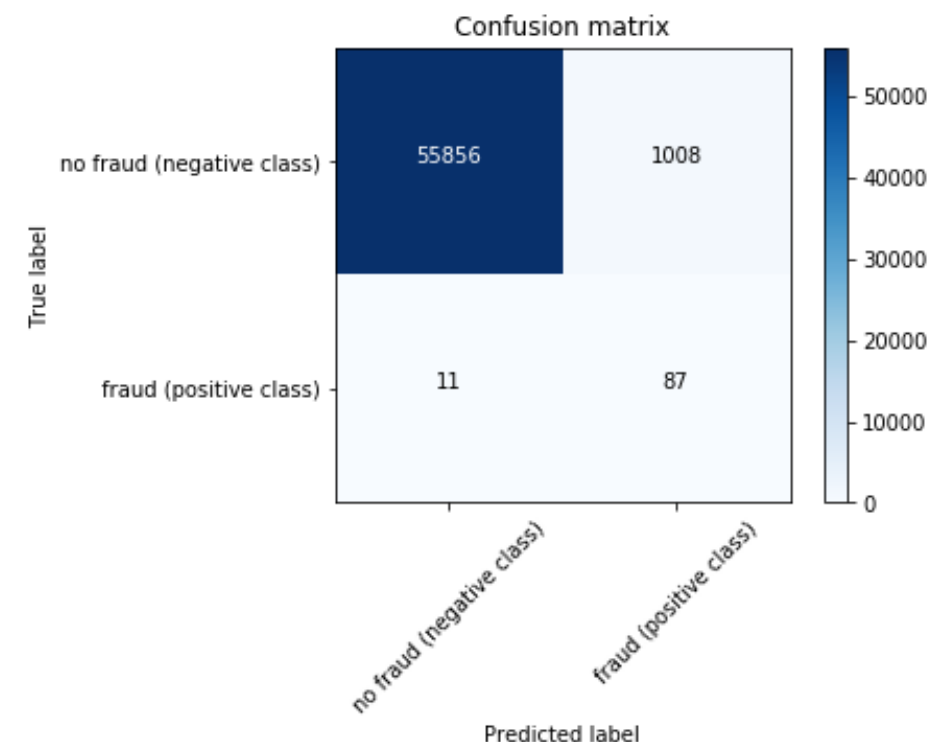


Regrouper des données similaires (non supervisé)
Ex : clustering de clients

Classification : la matrice de confusion

- Tableau qui compare les prédictions aux vraies valeurs
- Vrais Positifs (VP), Faux Positifs (FP), Faux Négatifs (FN), Vrais Négatifs (VN)

		Predicted Class		
		Positive	Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	Sensitivity $\frac{TP}{(TP + FN)}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	Specificity $\frac{TN}{(TN + FP)}$
		Precision $\frac{TP}{(TP + FP)}$	Negative Predictive Value $\frac{TN}{(TN + FN)}$	Accuracy $\frac{TP + TN}{(TP + FP + FN + TN)}$



Exemple : Matrice de confusion d'un modèle de détection de fraudes avec un déséquilibre classe majoritaire (99% de non-fraudes).

Classification : métriques dérivées de la matrice de confusion

$$\textit{Precision} = \frac{VP}{VP + FP}$$

→ Parmi les prédits positifs, combien sont vraiment positifs ?

$$\textit{Rappel} = \frac{VP}{VP + FN}$$

→ Parmi les vrais positifs, combien ont été détectés ?

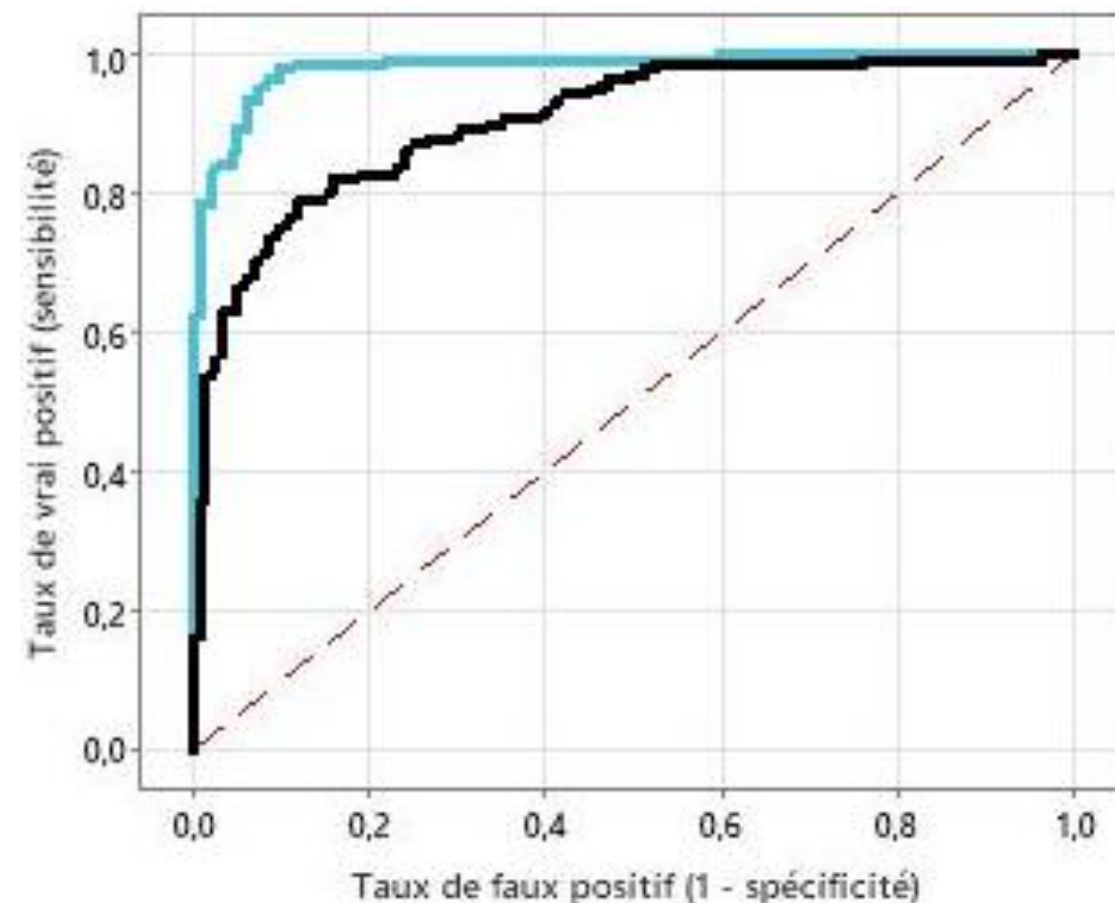
$$F1 = \frac{2}{\frac{1}{\textit{Precision}} + \frac{1}{\textit{Rappel}}} = \frac{2 * \textit{Precision} * \textit{Rappel}}{\textit{Precision} + \textit{Rappel}} \rightarrow \text{Moyenne harmonique pour équilibrer les deux.}$$

$$\textit{Accuracy} = \frac{VP + VN}{VP + VN + FP + FN}$$

→ Quelle est la proportion de prédictions correctes ?

Classification : courbe ROC et AUC

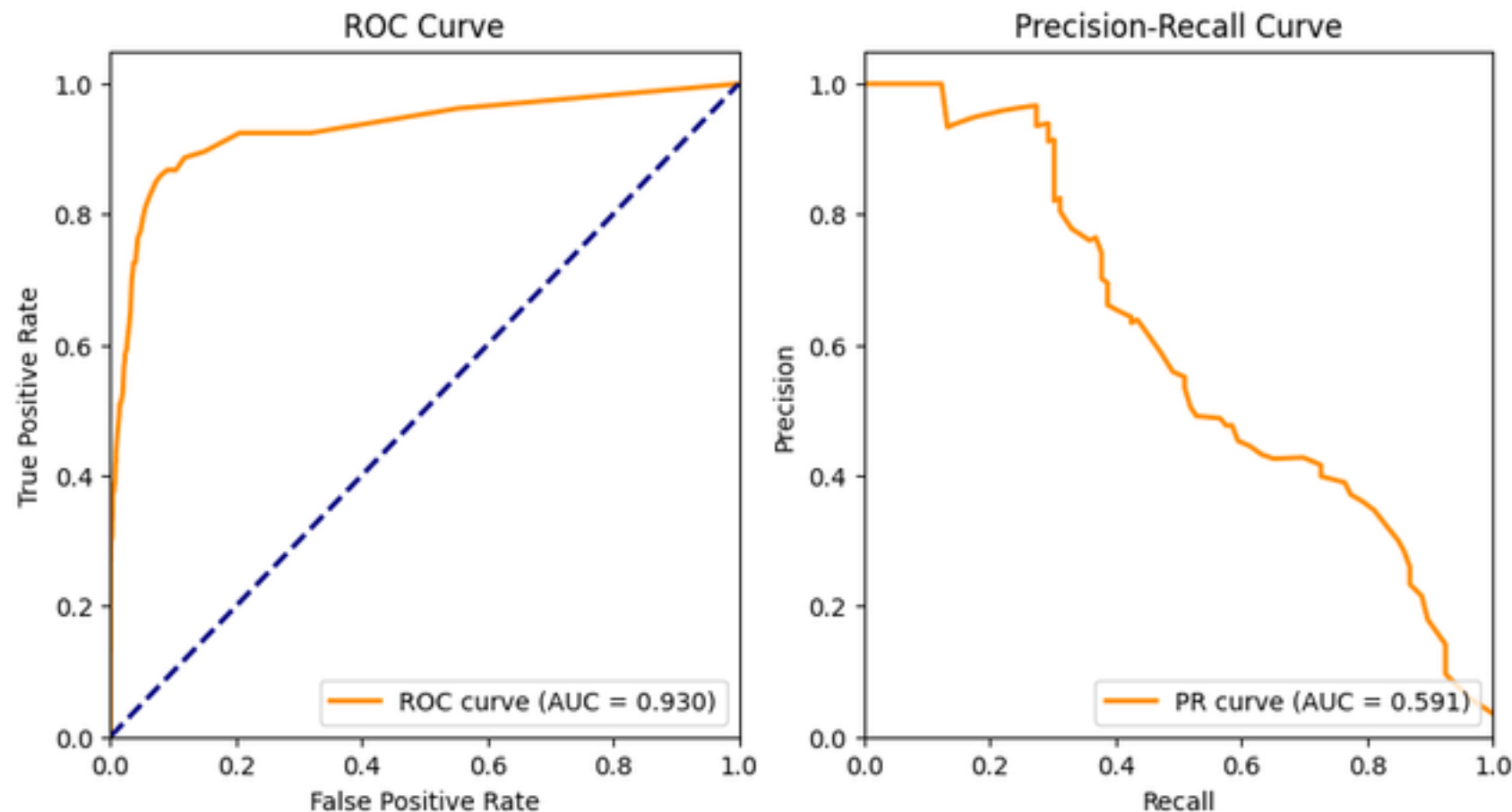
- Courbe ROC (Receiver Operating Characteristic) : courbe qui trace le taux des vrais positifs en fonction du taux de faux positifs pour tous les seuils possibles.
 - Taux de Vrais Positifs (**Sensibilité ou Rappel**)
vs Taux de Faux Positifs ($FPR = \frac{FP}{FP+VN}$)
- AUC (Area Under Curve) : Aire sous la courbe (entre 0 et 1).
 - AUC=0.5 : aléatoire
 - AUC=1 : parfait



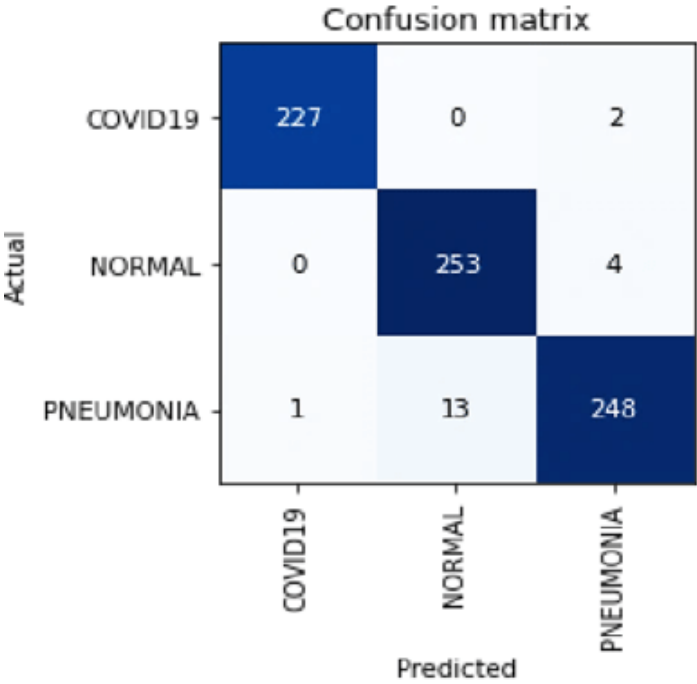
Plus la courbe est proche du coin supérieur gauche, meilleur est le modèle

Classification : courbe précision-rappel

- La courbe précision-rappel (PR-curve) représente la précision en ordonnée et le rappel en abscisse. Elle illustre le compromis entre la valeur prédictive positive (précision) et le taux de vrais positifs (rappel).



Classification : cas multi-classe



Moyenne	Idée	Quand l'utiliser
Micro-average	Performance globale : calcul global en sommant tous les VP, FP, FN, VN	Dataset équilibré ou vision globale
Macro-average	Égalité entre classes : moyenne des métriques par classe (non pondérée)	Classes rares importantes
Weighted-average	Compromis : moyenne des métriques par classe, pondérée par le nombre d'instances	Classes déséquilibrées mais score global souhaité

Les limites des métriques simples

Cas d'école : Détection de Fraude

- 99 % des transactions = non-fraude
- Modèle prédit toujours « non-fraude »

Accuracy = 99 % chiffre flatteur

Détection fraude = 0 % modèle inutile

Sur un jeu déséquilibré, une métrique unique peut masquer un échec total sur la classe d'intérêt.

Ce que le modèle voit vs. Fait

Données réelles

NON FRAUDE	NON FRAUDE	NON FRAUDE	FRAUDE
---------------	---------------	---------------	--------

Prédictions du modèle

NON	NON	NON	NON
-----	-----	-----	-----

3 / 4 corrects sur la classe majoritaire, 0 / 1 sur la classe rare.

Classification : Balanced Accuracy

Donner le même poids à chaque classe

Formule

$$\text{Balanced Acc} = (\text{Recall}_0 + \text{Recall}_1) / 2$$

Exemple chiffré

- Recall non-fraude = **0,95**
- Recall fraude = **0,40**

Balanced Accuracy = 0,675

	Prédit 0
Réel 0	VN
Réel 1	FN

$$\text{Recall}_0 = \text{VN} / (\text{VN} + \text{FP})$$

$$\text{Recall}_1 = \text{VP} / (\text{VP} + \text{FN})$$

On moyenne les deux pour ré-équilibrer.

Classification : Matthews Correlation Coefficient (MCC)

MCC est une mesure globale fiable, même en fort déséquilibre

Intérêt

Le MCC tient compte des quatre cases de la matrice de confusion : il reste informatif quand les classes sont très inégales.

Formule

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$$

Lecture du score

-1 : mauvais

0 : hasard

+1 : parfait

Matrice complète utilisée

	Prédit Oui	Prédit Non
Réel Oui	TP	FN
Réel Non	FP	TN

Contrairement au F1-score, le MCC utilise toutes les cases.

Exemple : fraude

TP = 50

FP = 100

FN = 40

TN = 1000

MCC ≈ 0,45

Classification : Top-K Accuracy

La bonne réponse est-elle dans les K premières propositions ?

Idée

On considère une prédiction correcte si la vraie classe figure dans le top-K du modèle. Utile en classification fine (ImageNet, recherche, retrieval).

Exemple : image de chat

- | | | |
|---|--------|---------|
| 1 | chien | faux |
| 2 | tigre | faux |
| 3 | chat | correct |
| 4 | lion | |
| 5 | renard | |

Lecture

Top-1

échec

La classe « chat » n'est pas en première position.

Top-5

succès

La classe « chat » apparaît en 3^e position : top-5 validé.

Quand l'utiliser ?

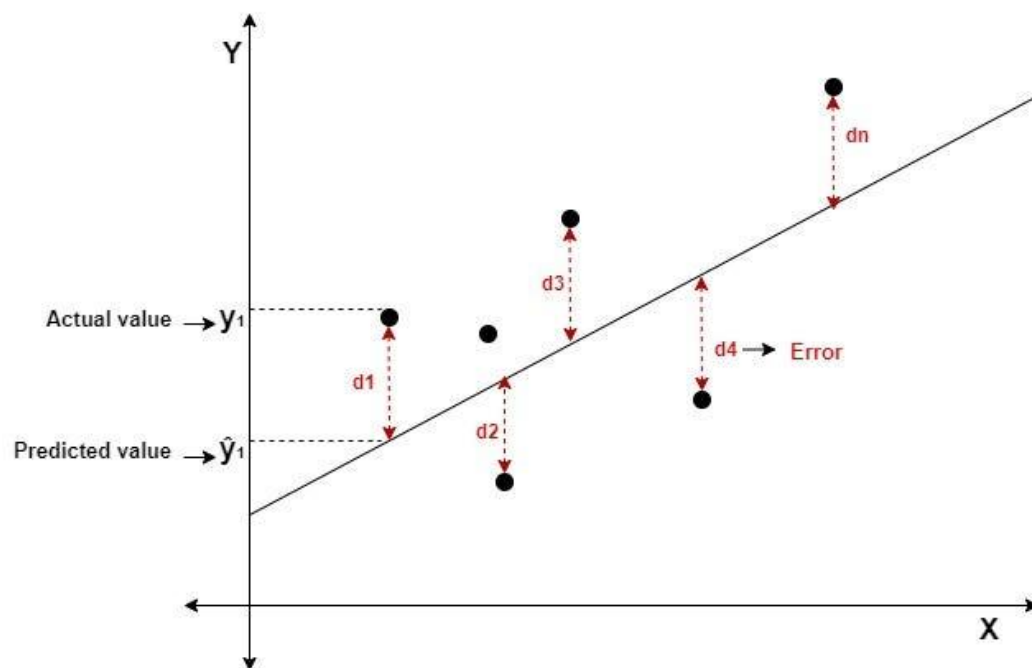
Classification à très nombreuses classes

Systèmes de suggestion (UX tolère plusieurs choix)

Évaluation de modèles de retrieval

Régression : MSE, RMSE, MAE – le trio gagnant

Les métriques de régression ne pénalisent pas les erreurs de la même façon.



$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

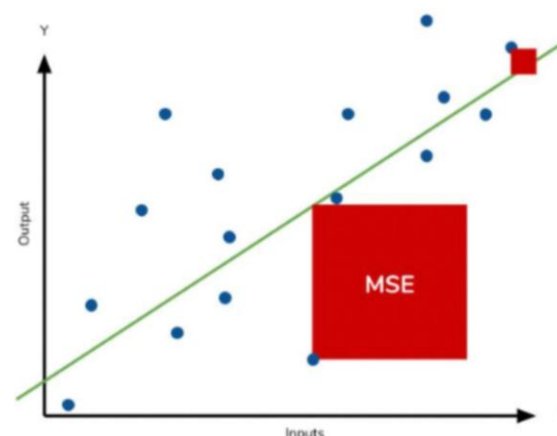
➤ Plus robuste aux valeurs aberrantes (outliers)

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

➤ Pénalisent davantage les grosses erreurs

$$RMSE = \sqrt{MSE}$$

➤ Même unité que la variable cible



Régression : R^2 , MAPE, MPE

Coefficient de
détermination

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y}_i)^2}$$

- Mesure la variance expliquée, mais ne suffit pas seul. Entre 0 et 1. Proche de 1 : bon ajustement

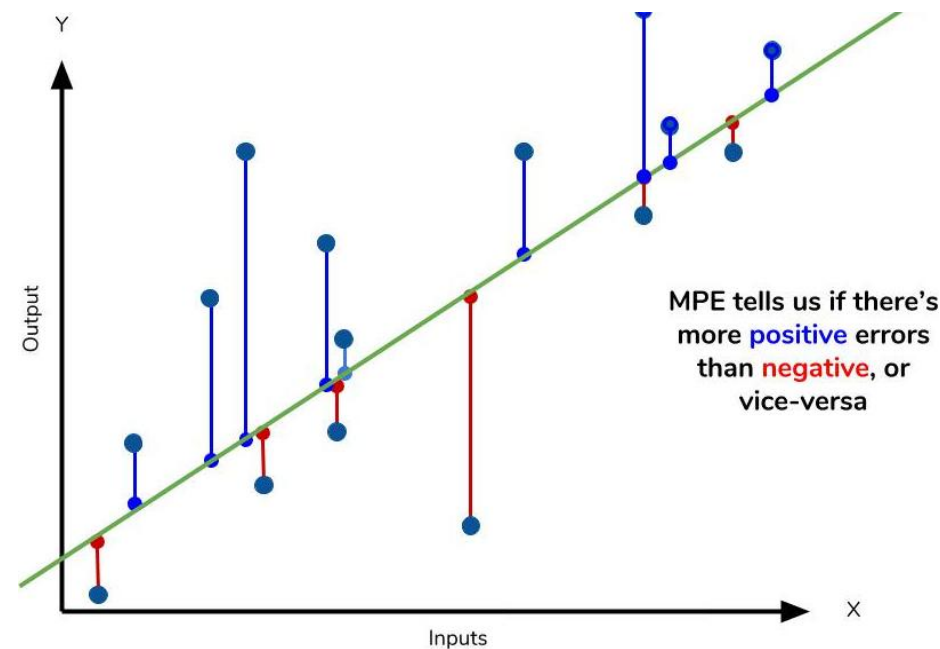
Mean Absolute
Percentage Error

$$MAPE = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

- Mesure l'erreur moyenne en pourcentage. Problème de MAPE : instable si la valeur réelle est proche de 0
- **sMAPE** : variante plus stable

Mean Percentage
Error

$$MPE = \frac{100\%}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{y_i} \right)$$



*<https://sefidian.com/2022/08/18/a-guide-on-regression-error-metrics-with-python-code/>

Régression : Huber Loss

Gérer les erreurs extrêmes sans laisser exploser la loss =
regression robuste

Idée

Petites erreurs → traitées comme du MSE (quadratique, lisse)

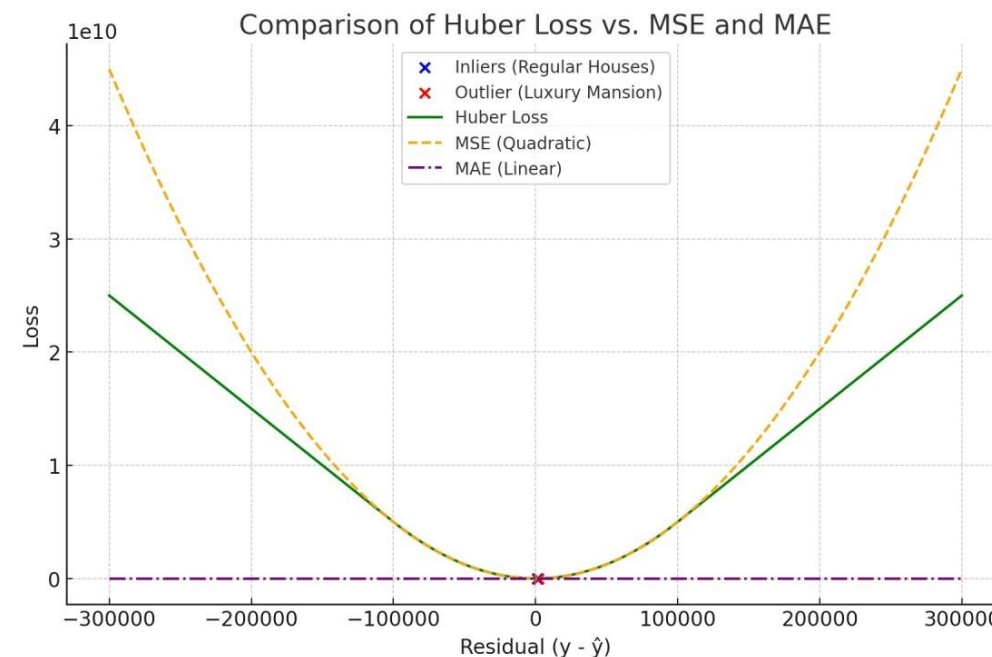
Grandes erreurs → traitées comme du MAE (linéaire, robuste)

Pourquoi ?

Le MSE seul donne trop de poids aux *outliers* ; le MAE seul est moins lisse à optimiser. Huber combine les avantages.

Idéal pour la régression sur données bruitées ou capteurs.

Forme de la fonction

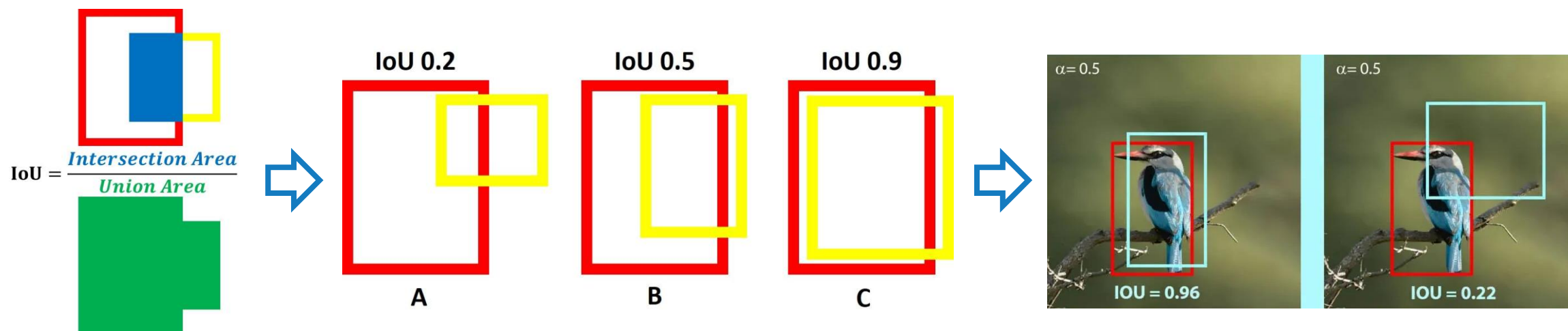


La courbe Huber reste linéaire pour les grosses erreurs.

Métriques spécifiques : vision par ordinateur

- IoU (Intersection over Union)

Pour la détection d'objets : aire d'intersection / aire d'union entre la boîte prédite et la vraie boîte.



Métriques spécifiques : vision par ordinateur

- Dice score

Mesurer le recouvrement entre une zone prédite et une zone réelle

Formule

$$\text{Dice} = 2 \cdot |A \cap B| / (|A| + |B|)$$

Application

Segmentation médicale (tumeurs, organes)

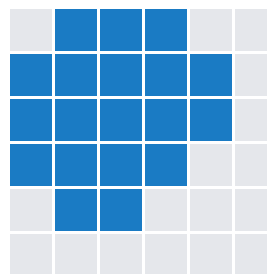
Vision industrielle, satellite, biologie

Vraie zone tumorale vs. prédiction du modèle

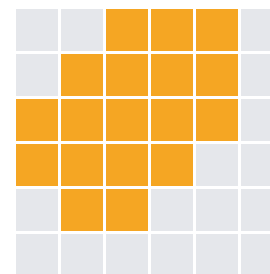
Dice $\approx$ 0,85

Intersection des zones

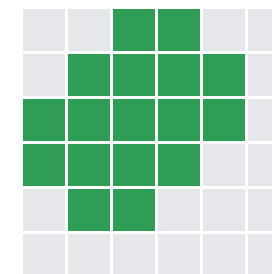
A - RÉEL



B - PRÉDIT

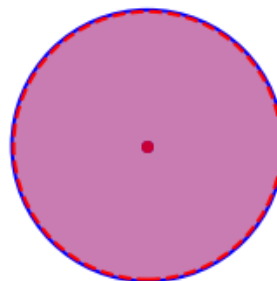


A ∩ B

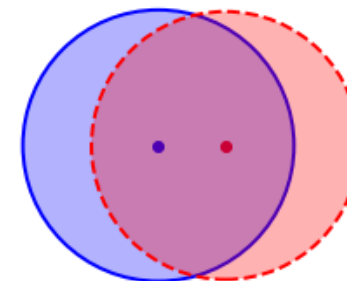


La zone commune (en vert) est le numérateur du score.

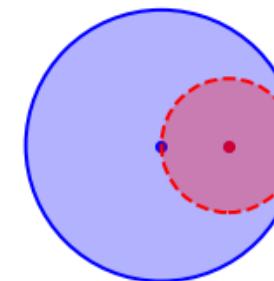
IoU=1.00
Dice=1.00



IoU=0.52
Dice=0.69



IoU=0.25
Dice=0.40



Métriques spécifiques : vision par ordinateur

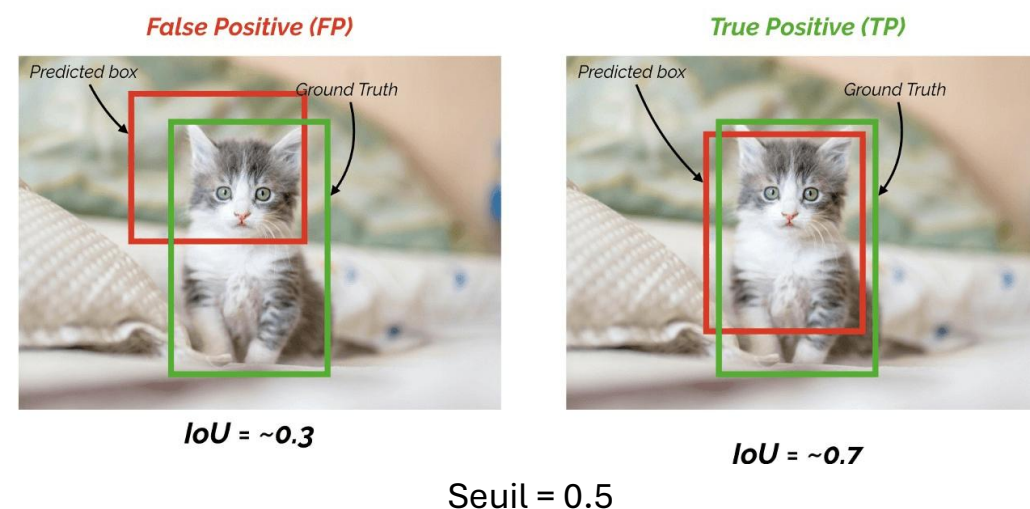
- **mAP (mean Average Precision)**

Pour la détection d'objets : La précision moyenne (mAP) est moyenne des précisions moyennes (AP) par classe à différents seuils de rappel. Cependant, l'interprétation de l'AP et de la mAP varie selon les contextes. Par exemple, dans le document d'évaluation du défi de détection d'objets COCO, l'AP et la mAP sont considérées comme identiques.

Étapes de calcul de l'AP :

1. Générer les scores de prédiction à l'aide du modèle.
2. Convertir les scores de prédiction en étiquettes de classe.
3. Calculer la matrice de confusion (TP, FP, TN, FN).
4. Calculer la précision et le rappel.
5. Calculer l'aire sous la courbe précision-rappel.
6. Mesurer la précision moyenne.

La mAP est calculée en déterminant la précision moyenne (AP) de chaque classe, puis en faisant la moyenne de ces valeurs sur plusieurs classes.



*<https://learnopencv.com/mean-average-precision-map-object-detection-model-evaluation-metric/>

Métriques spécifiques : génération d'image

- **FID - Fréchet Inception Distance**

Évaluer la qualité d'images générées

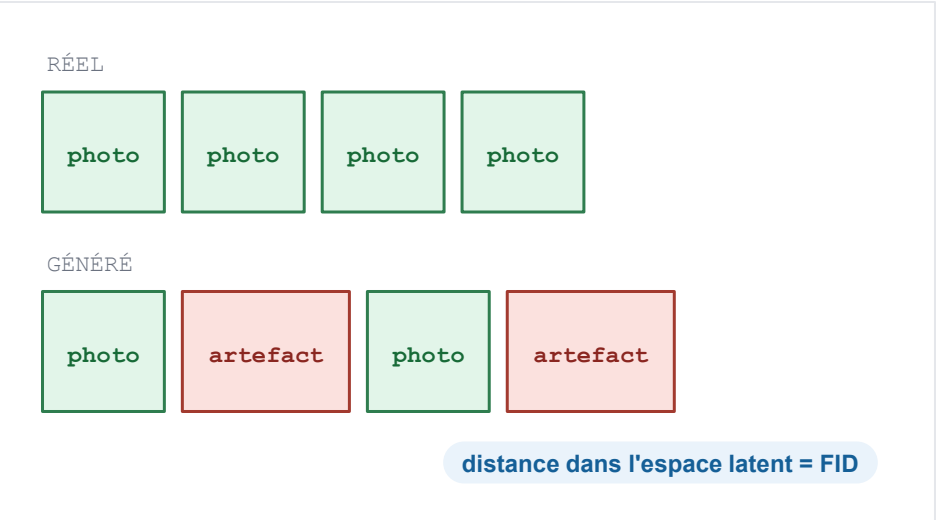
Idée

On extrait les caractéristiques d'un grand nombre d'images réelles et générées via un réseau pré-entraîné, puis on mesure la distance entre les deux distributions.

Interprétation

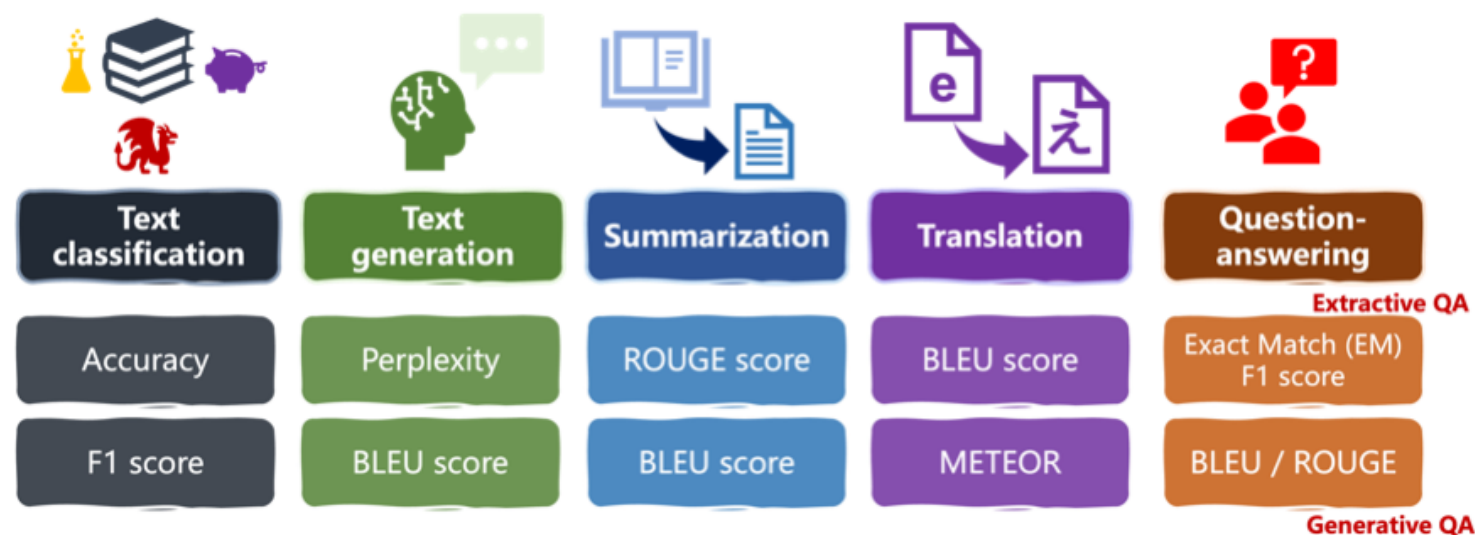
FID faible	générations réalistes
FID élevé	artefacts, mode collapse

Distance entre deux ensembles



Métriques spécifiques : NLP

- **BLEU (Bilingual Evaluation Understudy)** : Pour la traduction automatique → précision n-gramme avec pénalité de longueur.
- **ROUGE (Recall-Oriented Understudy for Gisting Evaluation)** : Pour le résumé automatique → rappel des n-grammes, etc.
- **Perplexité** : Pour les modèles de langue → mesure de l'incertitude de prediction du mot suivant (plus bas = mieux).



*<https://projector-video-pdf-converter.datacamp.com/33893/chapter4.pdf>

*<https://www.youtube.com/watch?v=JicEsC3lXoE>

Métriques spécifiques : NLP

- **BERTScore**

Comparer des phrases par leur sens, pas leurs mots

Idée

On encode chaque mot en vecteur via un modèle de langage, puis on mesure la proximité sémantique entre la phrase prédite et la phrase de référence.

Exemple

RÉFÉRENCE

« Le chat dort »

PRÉDICTION

« Le chat est en train de dormir »

Vecteurs Sémantiques

$$v_{ref} = [0.20, 0.80, 0.50] \quad v_{pred} = [0.21, 0.79, 0.48]$$

Très proches → sens similaire.

Lecture

BLEU

moyen : peu de mots en commun

BERTScore

élevé : sens conservé

Métriques spécifiques : RAG / LLM

- Faithfulness

La réponse reste-t-elle fidèle aux documents fournis ?

Pipeline RAG



Faithfulness

Mesure la cohérence entre la réponse générée et les documents sources. Un score faible = signal d'hallucination.

Vérification automatique par LLM-judge

Détection d'éléments non sourcés

Couplée à la pertinence et à la complétude

Cas : fidèle

DOCUMENT

« Paris est la capitale »

RÉPONSE

« Paris est la capitale »

faithfulness ok

Cas : hallucination

DOCUMENT

« Paris »

RÉPONSE

« Londres est la capitale »

hallucination

Métriques spécifiques : Recommandation

- **Precision@K**

Combien de bonnes recommandations parmi les K affichées ?

Formule

```
Precision@K = nb. d'items pertinents dans le top-K / K
```

Pourquoi ?

Les utilisateurs ne consultent que les premiers résultats : c'est la qualité du *haut de liste* qui compte.

Moteurs de recommandation (films, produits)

Recherche, suggestions, feeds

Exemple : top 5 films recommandés

1. Film A	pertinent
2. Film B	non pertinent
3. Film C	pertinent
4. Film D	pertinent
5. Film E	non pertinent

Precision@5 = 3 / 5 = 0,60

Métriques spécifiques : Recommandation

- **NDCG** (Normalized Discounted Cumulative Gain)

Les bons résultats doivent être en haut de la liste = **Ranking-aware**

Idée

Precision@K ignore l'ordre dans le top-K. Le NDCG (Normalized Discounted Cumulative Gain) pénalise les bons résultats placés trop bas.

Principe

Le gain d'un item est divisé par $\log_2(\text{position} + 1)$.

Position 1 → poids maximum

Position 10 → poids très faible

Comparaison de classements

Cas 1 : bon

1	préféré
2	moyen
3	moyen

L'item le plus utile est en tête.

Cas 2 : mauvais

1	moyen
2	moyen
3	préféré

Même contenu, ordre dégradé : NDCG plus faible.

L'ordre des résultats est essentiel pour mesurer la qualité d'un ranking.

Métriques spécifiques : Production

- **Calibration : Brier score**

Un modèle peut être très confiant... et souvent faux

Le problème

Une probabilité de 0,9 doit correspondre, sur le long terme, à ~90 % de cas positifs réels.

Brier score

$$\text{Brier} = (p - y)^2$$

Exemple

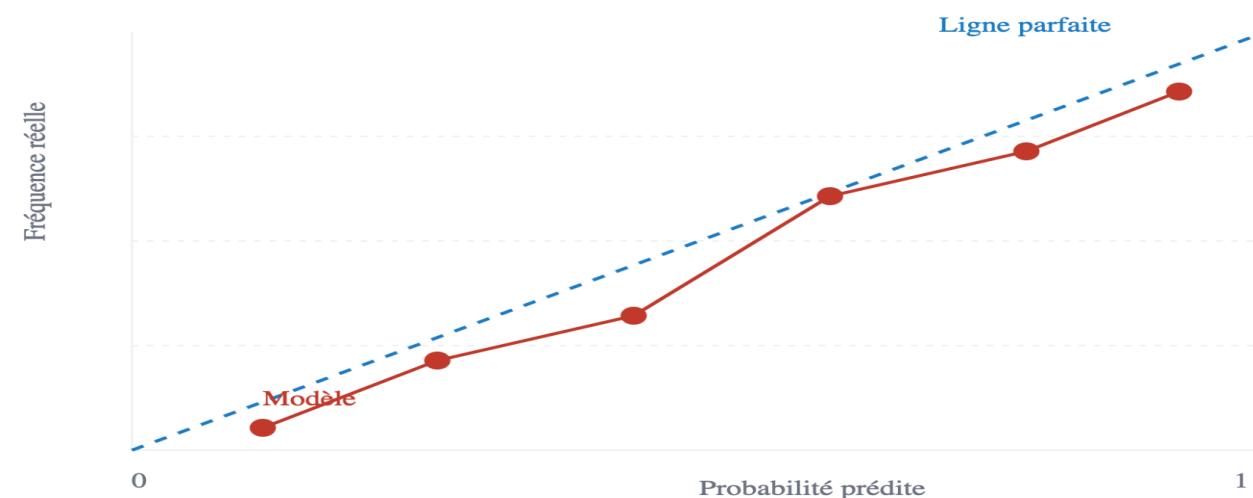
Prédiction = **0,9**

Réalité = **1**

Erreur = $(0,9 - 1)^2 = \mathbf{0,01}$

10 cas annoncés à 0,9, seulement 7 corrects → modèle mal calibré.

Courbe de calibration



Plus la courbe s'écarte de la diagonale, plus le modèle est mal calibré.

Métriques spécifiques : Production

- **Performance en production : Latence**

Un bon modèle qui répond trop lentement n'est pas utilisable

Définition

Temps écoulé entre la requête de l'utilisateur et la réponse renvoyée par le système.

À surveiller

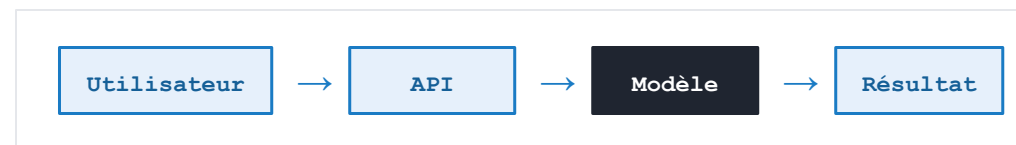
Latence médiane (p50)

Latence p95 / p99 (queue de distribution)

Débit (requêtes / seconde)

Toujours mesurer en conditions réelles, charge incluse.

Flux d'une requête



Ordres de grandeur

10 ms

rapide : UX fluide

2 s

trop lent : UX dégradée

Métriques spécifiques : Production

- **Data Drift**

Les données changent et le modèle vieillit

DÉFINITION

Décalage progressif entre la distribution des données d'entraînement et celle observée en production.

EXEMPLE

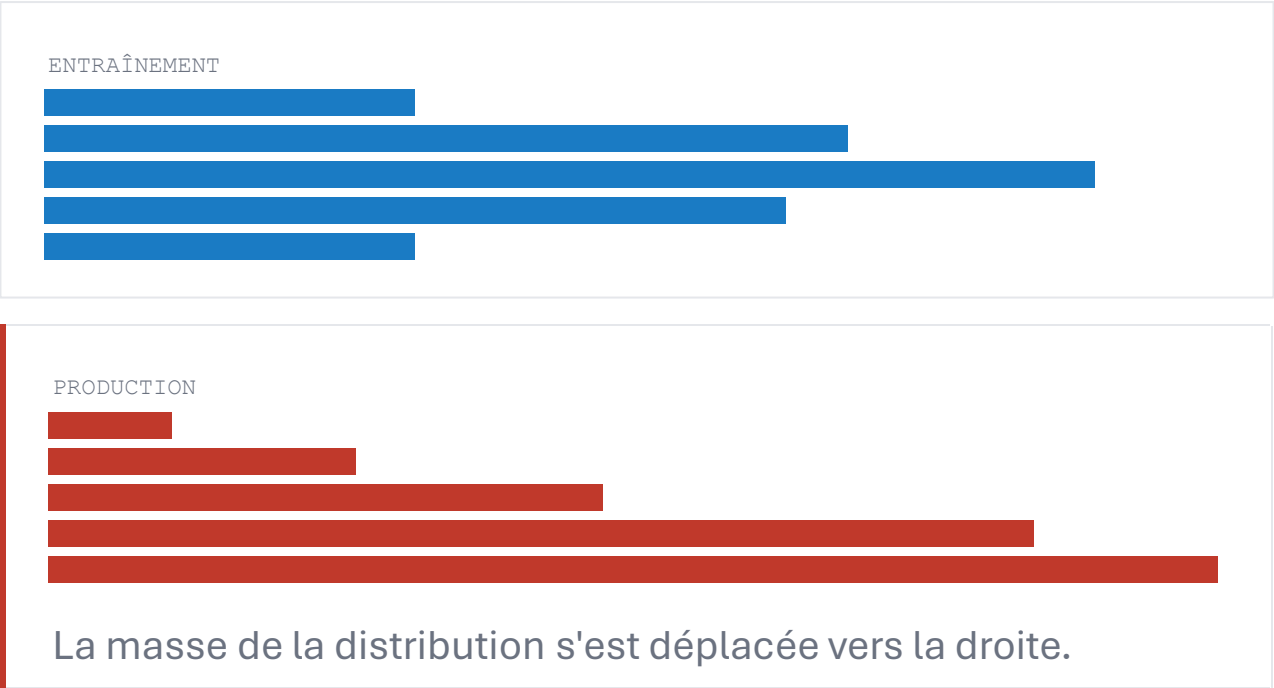
- Modèle entraîné sur une population jeune
- Production déployée sur des seniors

la performance se dégrade

SURVEILLER

- Distribution des entrées (features)
- Distribution des sorties (prédictions)
- Qualité métier mesurée a posteriori

AVANT VS APRÈS : DISTRIBUTION D'UNE



Synthèse : choisir la bonne métrique

Une métrique par problème, jamais une seule en production

DOMAINE	MÉTRIQUES RECOMMANDÉES
Classification	F1 · MCC · Recall · Balanced Accuracy
Régression	MAE · RMSE · Huber Loss
Vision / segmentation	IoU · Dice Score
NLP	BLEU · BERTScore
LLM / RAG	Faithfulness · Hallucination · Top-K
Recommandation	Precision@K · NDCG
Génératif	FID
Production	Latence · Data Drift · Calibration

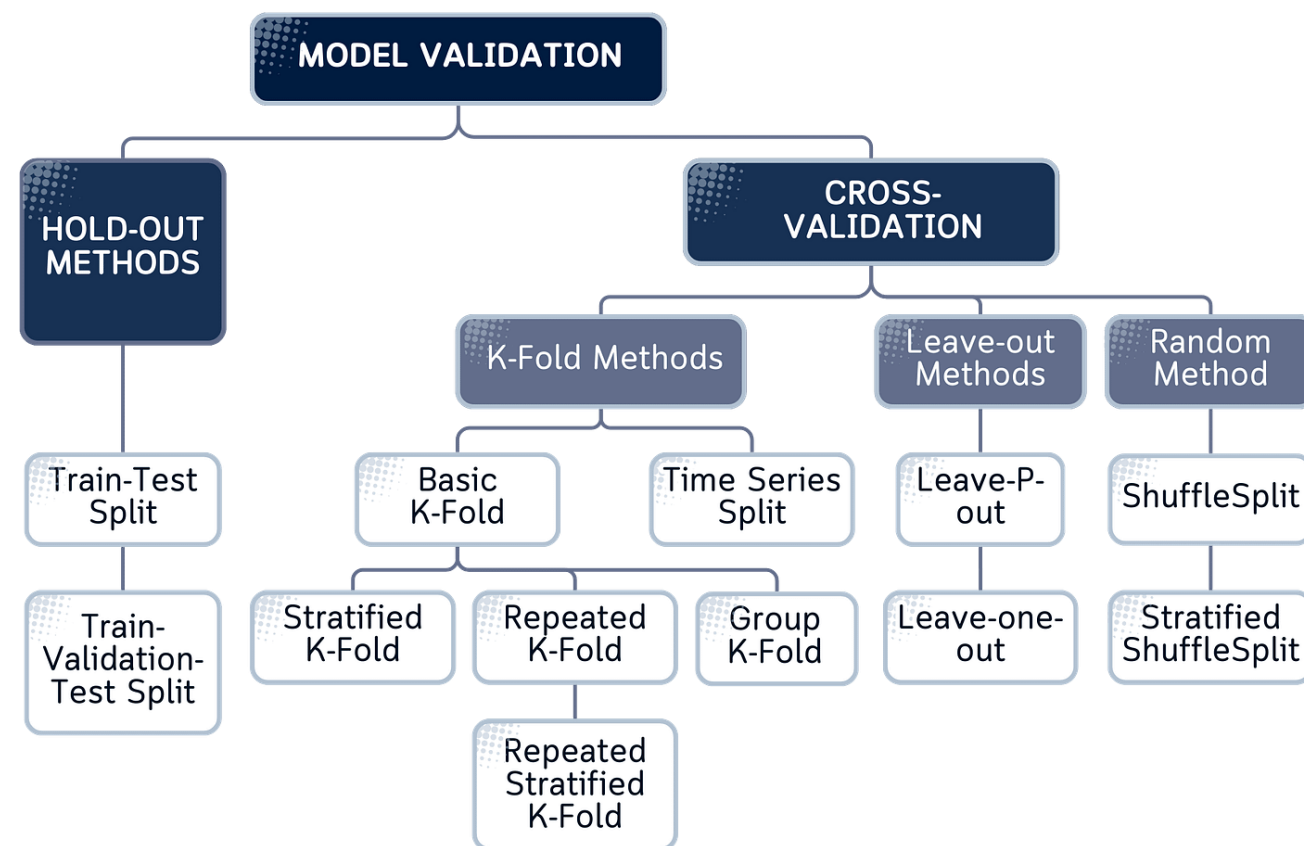
À retenir. Évaluer un modèle, c'est choisir la bonne lentille : l'accuracy n'en est qu'une parmi beaucoup.



Méthodes de validation : Hold-out vs cross-validation

Une bonne métrique sans bonne validation reste trompeuse.

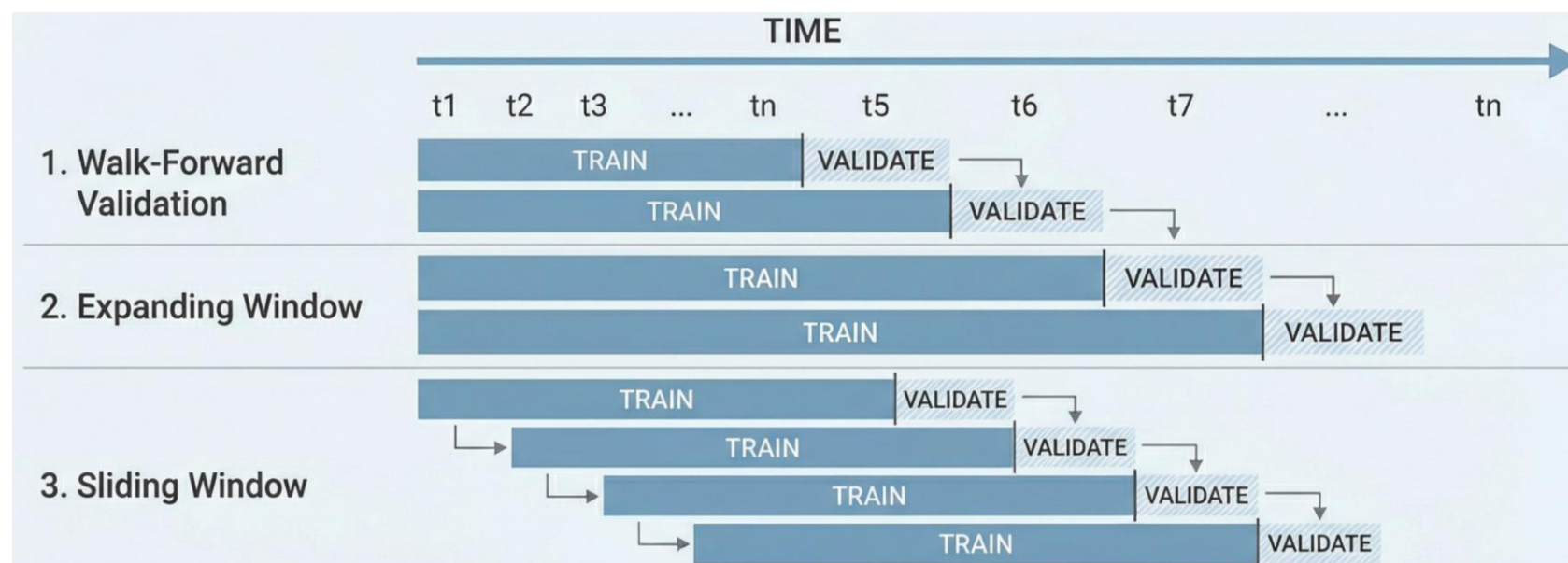
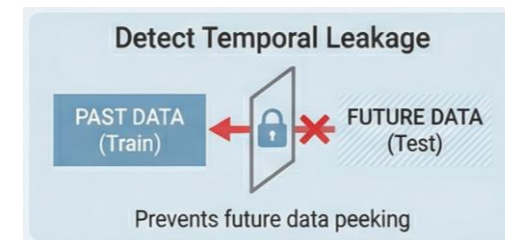
- **Hold-out** : train/test split (ex: 80/20) → simple mais sujet aux problèmes de variance.
- **k-fold Cross-validation** : moyenne sur plusieurs partitions
 - Diviser en k parties (folds) → Entraîner sur k-1 folds, valider sur 1 fold, répéter k fois.
 - **Stratified k-fold** : conserve la distribution des classes (pour les classes déséquilibrées).
- **Leave-P-out** : utiliser p observations comme ensemble de validation et les observations restantes comme ensemble d'entraînement
- **ShuffleSplit** : mélange aléatoirement les données, crée plusieurs splits train/test indépendants, sans contrainte de répartition fixe



Méthodes de validation pour séries temporelles

Pour les séries temporelles : **on ne peut pas mélanger les données !**

- Split chronologique : Train sur $[t_0, t_1]$, validation sur $[t_1, t_2]$, test sur $[t_2, t_3]$.
- Validation glissante (walk-forward).
- Jamais mélange passé/futur.



*<https://machinelearningmastery.com/5-ways-to-use-cross-validation-to-improve-time-series-models/>

Méthodes de validation : résumé

Scénario	Méthode recommandée	Pourquoi	Coût	Exemples
Classification équilibrée	K-Fold (5 ou 10)	Très bon équilibre entre stabilité, biais et variance.	Moyen	Classification d'images, NLP, modèles classiques (SVM, RF).
Classification déséquilibrée	Stratified K-Fold	Préserve la distribution des classes dans chaque fold ; indispensable pour la classe minoritaire.	Moyen	Détection de fraude bancaire, churn client, défaut industriel.
Très grands datasets	Hold-Out ou K-Fold léger (K=3)	Évite des répétitions coûteuses ; la taille des données réduit la variance naturellement.	Faible (Hold) / Moyen (K=3)	Données de logs, big data, modèles de recommandation.
Très petits datasets (< 100-150 échantillons)	Leave-one-out (LOOCV)	Maximise chaque échantillon ; meilleure estimation possible.	Très élevé	Études médicales rares, prototypes, petits datasets scientifiques.
Flexibilité des splits, réplifications multiples	ShuffleSplit / StratifiedShuffleSplit	Contrôle total sur les proportions, idéal si K-Fold trop rigide	Faible à moyen	Mélange aléatoire contrôlé, scoring robuste
Régression (toutes tailles)	K-Fold	Pas de classes → la stratification n'a pas de sens ; stable et robuste.	Moyen	Bourse, estimation de prix, prévisions environnementales.
Données groupées	Group K-Fold	Empêche les fuites de données entre groupes corrélés.	Moyen	Analyses médicales par patient, données utilisateurs.
Séries temporelles	TimeSeriesSplit / Walk-Forward	Respecte la chronologie ; aucune fuite du futur dans le passé.	Moyen à élevé	Prévisions de ventes, prévisions météo,...

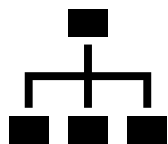
Bonnes pratiques et outils



1. Commencer par des métriques simples et adaptées au problème

Sélectionner les métriques selon :

- le type de problème (classification, régression, ...).
- les enjeux métiers (coût d'une fausse alerte, coût d'un faux négatif, etc.).



4. Tenir compte du déséquilibre des classes

Adapter les métriques. Utiliser des techniques adaptées : Stratified sampling, rééchantillonnage, class weights, ...



2. Utiliser des visualisations pour compléter les métriques

Croiser *métriques* + *visualisations* pour une compréhension plus riche : courbe ROC / PR, matrice de confusion, erreurs résiduelles, QQ-plot, scatter y vs $\hat{y}$, calibration curve, Brier score, etc.



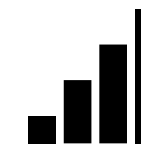
5. Utiliser une validation rigoureuse

Toujours valider via une méthode robuste : Cross-validation (K-Fold, Stratified K-Fold, GroupKFold, etc.), TimeSeriesSplit pour séries temporelles. Éviter le simple train/test quand les données sont limitées.



3. Ne jamais se fier à une seule métrique

Une seule métrique peut être trompeuse (ex. accuracy sur données déséquilibrées). Toujours analyser un **ensemble cohérent** d'indicateurs. Rechercher les divergences (ex. bonne précision mais mauvais rappel).



6. Comparer le modèle à une baseline simple

Modèle trivial (majority class, moyenne, médiane), logistic regression / random forest. Si le modèle complexe ne surpasse pas la baseline → revoir la feature engineering ou le modèle.

IV. Modélisation supervisée et avancée en Deep Learning



Comprendre et implémenter des architectures profondes adaptées aux différents types de données

4.1

**Réseaux de neurones
artificiels**

4.2

**CNN et vision par
ordinateur**

4.3

**RNN et traitement des
données séquentielles**

4.4

**Métriques d'évaluation
des performances**

4.5

**Optimisation et
régularisation**

4.6

**IA générative : GAN,
VAE**

4.7

**Attention, Transformers,
RAG & LLM**

4.8

**Modélisation
multimodale**

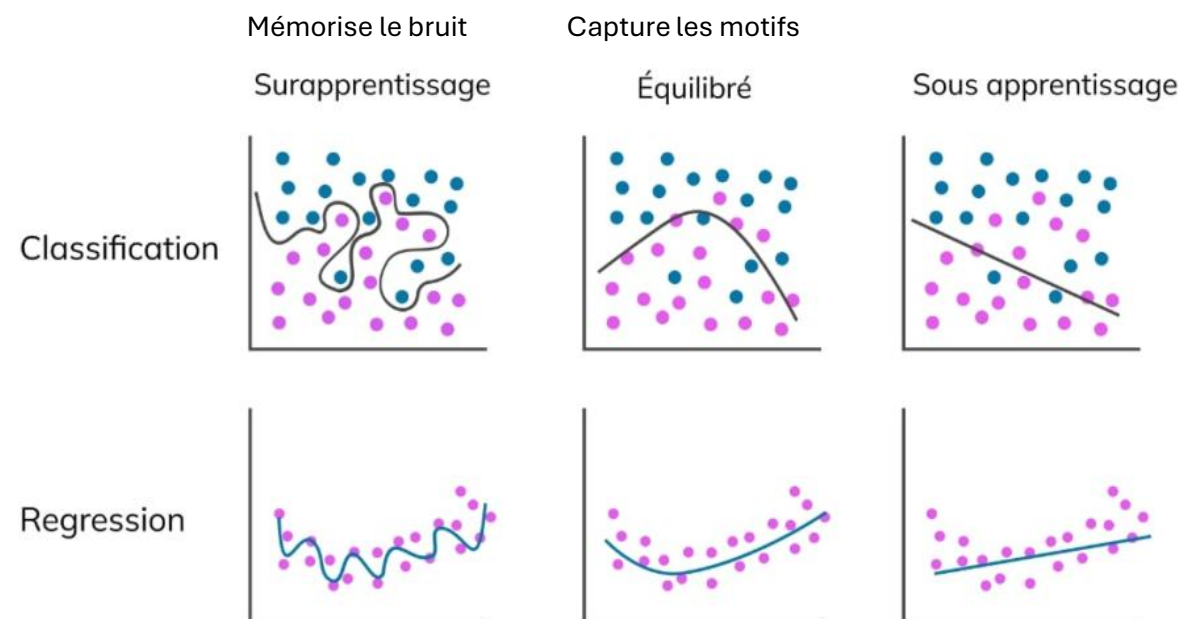
Dompter son modèle : l'art d'optimiser et de régulariser



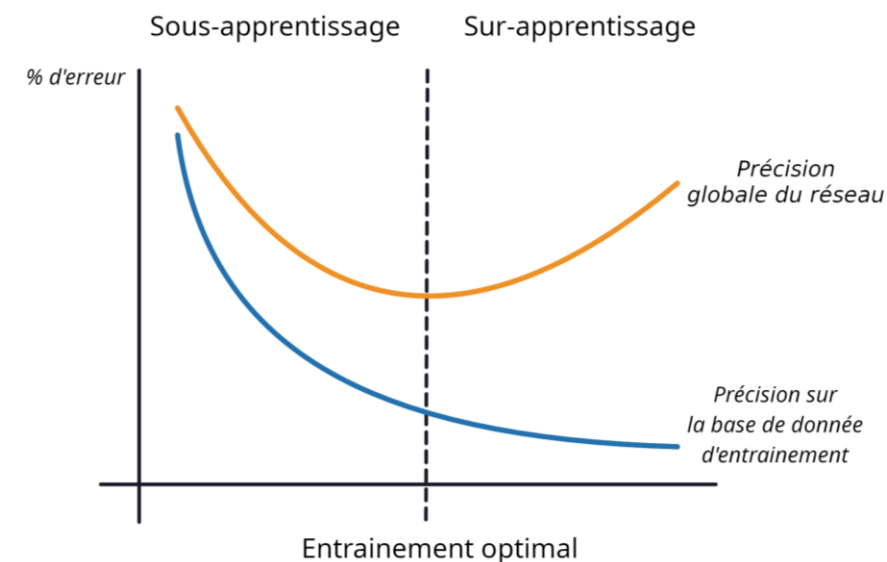
Un modèle non optimisé est comme une formule 1 sans réglage : son plein potentiel n'est pas exploité.
Un modèle non régularisé est comme un moteur sans limiteur : il explose.

Dompter son modèle : l'art d'optimiser et de régulariser

Rappel - Le Fléau du Surapprentissage (Overfitting)



Train loss diminue + Val loss diminue puis remonte
= modèle mémorise le bruit



Apprendre trop longtemps ou avec trop de capacité fait chuter la généralisation.

Conséquences :

- Performance en chute sur des données invisibles.
- Coûts métier élevés (ex: faux négatifs en diagnostic médical).

Pourquoi optimiser et régulariser ?

Optimisation :

1. Trouver des paramètres qui minimisent la fonction de perte.
2. Accélérer la convergence de l'apprentissage.
3. Éviter les plateaux, vallées étroites, gradients explosifs/vanissants.
4. Assurer un apprentissage stable et reproductible.

Régularisation :

1. Réduire le surapprentissage (overfitting).
2. Améliorer la généralisation sur des données nouvelles.
3. Stabiliser l'apprentissage (training plus régulier).
4. Introduire des biais inductifs utiles (e.g., sparsité, lissage).



Optimiser = bien entraîner

Régulariser = empêcher d'apprendre le bruit

Les hyperparamètres clés en Deep Learning

Les performances d'un modèle changent énormément selon les hyperparamètres. Les principaux hyperparamètres à optimiser sont :



Architecture

- Nombre de couches / profondeur
- Nombre de neurones / largeur
- Type de couches (Dense, Conv, LSTM, Attention...)



Optimisation

- Learning rate (le plus critique !)
- Scheduler (step decay, cosine annealing, warmup...)
- Type d'optimiseur (SGD, Adam, AdamW, RMSprop...)



Régularisation

- Dropout rate
- Weight decay (L2)
- Early stopping patience
- Data augmentation configuration



Entraînement

- Batch size
- Nombre d'époques
- Métriques cibles
- Mélange des données (shuffle, stratification)

Les hyperparamètres clés en Deep Learning

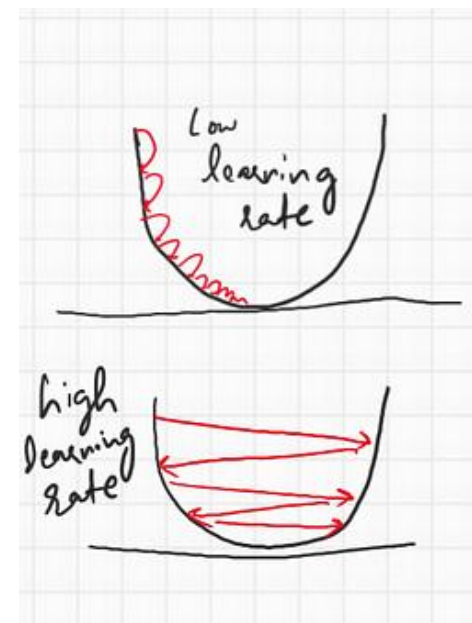
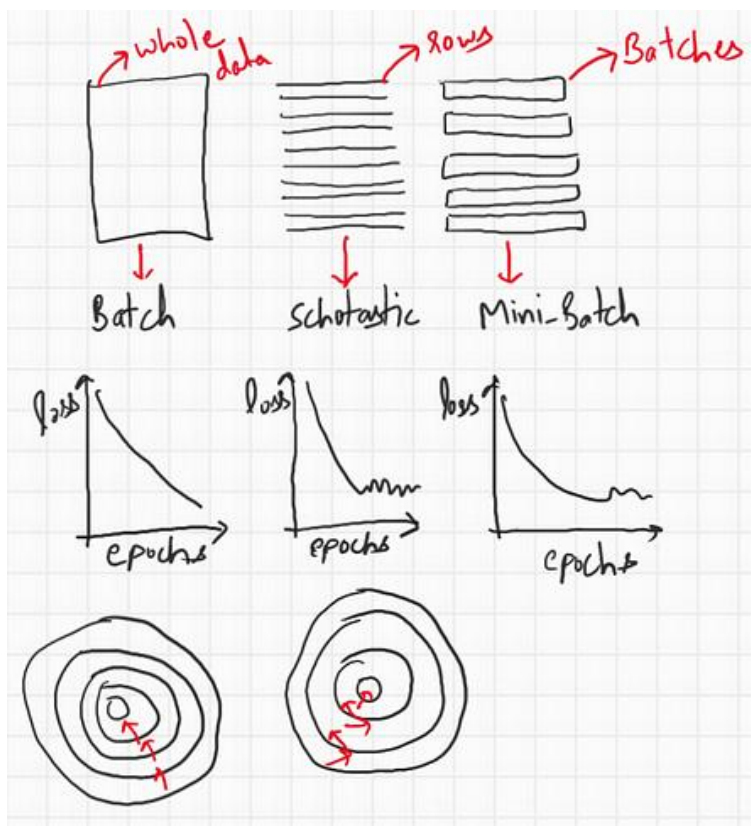
Les performances d'un modèle changent énormément selon les hyperparamètres.

Hyperparamètre	Effet principal	Risque si mal choisi
Learning rate (Optimisation)	vitesse d'apprentissage	divergence / lenteur
Batch size (Entrainement)	stabilité et Vitesse d'apprentissage	bruit trop fort / mémoire
Profondeur / largeur / types de couche (Architecture)	capacité du modèle	surapprentissage / coût
Dropout (Régularisation)	Activation des neurones	sous-apprentissage si trop fort
Weight decay (Régularisation)	contrôle des poids	modèle trop rigide
Epochs (Entrainement)	temps d'apprentissage	surapprentissage

Optimisation des poids : Descente du gradient

Rappel : algorithme d'optimisation capable de trouver des solutions optimales en corrigeant petit à petit les paramètres (poids) dans le but de minimiser une fonction coût.

$$\theta(\text{étape suivante}) = \theta - \eta \nabla_{\theta} \mathcal{L}(\theta)$$



Optimisation des poids : Descente du gradient

Caractéristique	SGD	RMSprop	Adam	AdamW
Mécanisme principal	Descente de gradient + momentum	Moyenne mobile des gradients au carré	Momentum + gradients au carré	Adam avec weight decay découplé
Weight Decay	Optionnel, intégré dans la mise à jour	Inclus dans la mise à jour	Inclus dans la mise à jour	Découplé (implémentation correcte)
Scalabilité	Faible	Modérée	Bonne	Meilleure qu'Adam
Efficacité	Faible	Modérée	Modérée	Modérée
Sensibilité aux hyperparamètres	Très élevée	Élevée	Élevée	Modérée
Brève explication	Simple et robuste mais exige un réglage précis ; très bonne généralisation	LR adaptatif, utile pour séquences non stationnaires	Convergence rapide et stable ; standard par défaut	Version moderne d'Adam ; généralise mieux grâce au vrai weight decay
Quand l'utiliser	Vision classique, grands datasets, recherche de bonne généralisation	RNN, signaux ou environnements instables	Cas généraux, NLP, modèles complexes	Vision moderne, NLP, Transformers, entraînements sensibles au weight decay

*<https://towardsdatascience.com/understanding-deep-learning-optimizers-momentum-adagrad-rmsprop-adam-e311e377e9c2/>

Optimisation d'hyperparamètres : Méthodes naïves

Méthodes simples qui sont faciles à comprendre et implémenter mais coûteuses en calcul.

Méthode	Idée	Avantage	Limite
Réglage manuel	Ajuster « à la main »	Simple, intuitif	Très lent, pas systématique
Grid Search	Essayer toutes les combinaisons	Exhaustif	Explosion combinatoire
Random Search	Tirer au sort les hyperparamètres	Beaucoup plus efficace que Grid Search	Pas adaptatif

Optimisation d'hyperparamètres : Méthodes intelligentes

Méthode	Idée générale	Forces principales	Limites / Contraintes	Quand l'utiliser ?
Optimisation bayésienne	Crée un modèle probabiliste qui estime comment les hyperparamètres influencent la performance, puis choisit les essais les plus prometteurs.	Très efficace quand chaque entraînement est coûteux ; nécessite peu d'essais ; excellent pour trouver des valeurs optimales fines (ex. taux d'apprentissage).	Moins adaptée aux espaces très grands ou bruyants ; nécessite une fonction d'évaluation stable.	Pour des modèles coûteux à entraîner (réseaux profonds, Transformers), ou pour un tuning précis et méthodique.
Optuna	Moteur d'optimisation moderne utilisant une approche bayésienne + arrêt anticipé automatique des essais peu prometteurs.	Rapide, facile à intégrer, réduction drastique du temps d'optimisation grâce au <i>pruning</i> ; excellent support pour PyTorch et TensorFlow.	Peut nécessiter une configuration plus avancée pour des exécutions distribuées ; dépend de la qualité de la fonction objectif.	Pour la plupart des projets actuels : vision, NLP, apprentissage par renforcement, modèles complexes.
Hyperband	Explore un grand nombre de configurations en leur attribuant progressivement plus de ressources si elles semblent prometteuses.	Extrêmement efficace quand on peut arrêter tôt les mauvais modèles ; très adapté aux grandes recherches.	Nécessite beaucoup de configurations pour fonctionner pleinement ; moins pertinent quand le temps d'apprentissage minimal est élevé.	Pour tester très rapidement des centaines ou milliers de configurations.

Optimisation d'hyperparamètres : Méthodes intelligentes

Méthode	Idée générale	Forces principales	Limites / Contraintes	Quand l'utiliser ?
ASHA (Successive Halving Asynchrone)	Version améliorée et asynchrone d'Hyperband : élimine individuellement les modèles faibles dès qu'ils sont identifiés, sans attendre les autres.	Très scalable, performant en cluster, réduit le coût total d'entraînement ; élimination rapide des mauvaises configurations.	Peut parfois éliminer trop rapidement un modèle qui aurait été bon à plus long terme.	Pour des environnements parallélisés, serveurs multi-GPU, ou tuning intensif.
BOHB (Bayesian Optimization + Hyperband)	Combine l'efficacité d'Hyperband avec la précision de l'optimisation bayésienne.	Excellent compromis entre rapidité et qualité ; performant sur des espaces continus ou mixtes.	Mise en œuvre plus complexe ; nécessite une gestion fine des budgets de calcul.	Lorsque l'on veut le meilleur compromis entre exploration massive et précision bayésienne.
Ray Tune	Framework complet qui permet d'exécuter à grande échelle presque toutes les méthodes ci-dessus, sur une ou plusieurs machines.	Très flexible, supporte Hyperband, BOHB, Optuna, recherche distribuée ; idéal pour l'industrie et les gros modèles.	Installation et configuration plus lourdes ; inutile pour les petits projets.	Projets industriels, pipelines MLOps, tuning distribué, modèles très coûteux ou complexes.

Techniques de régularisation : Panorama

Régulariser, c'est contrôler la capacité du modèle.

Méthode	Idée	Effet principal
Early stopping	Arrêter avant le surapprentissage	Empêche l'overfitting
L2 / Weight decay	Punir les poids trop grands	Modèle plus lisse, réduit la variance
Dropout	Déconnecter des neurones au hasard	Rend le réseau robuste, réduit co-adaptation
Batch normalization	Normaliser activations par batch	Stabilise gradient, régularise indirectement
Data augmentation	Créer plus de données artificiellement	Augmente la diversité → meilleure généralisation
Label smoothing	« Adoucir » les labels (ex: 0.9 au lieu de 1)	Moins de surconfiance, meilleur calibrage
Stochastic depth	Skip aléatoirement des couches	Très efficace sur réseaux profonds
Mixup / Cutmix	Mélanger images et labels	Généralisation accrue, réduit l'overfitting, plus tolérant aux bruits

Techniques de régularisation : L2 / Weight decay

On pénalise les poids trop grands.

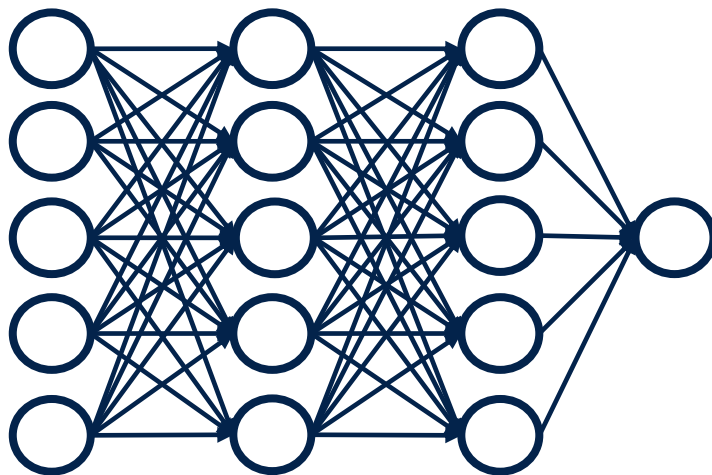
- Consiste à **rajouter au vecteur des poids d'un neurone un vecteur d'erreur** proportionnel à au carré des poids (norme 2 ou euclidienne)
- Ce vecteur d'erreur peut ensuite être multiplié par un **coefficient de proportionnalité** que l'on va augmenter pour pénaliser davantage les vecteurs de poids forts

$$\mathcal{L}_{reg} = \mathcal{L}_{data} - \lambda \sum_j \omega_j^2$$

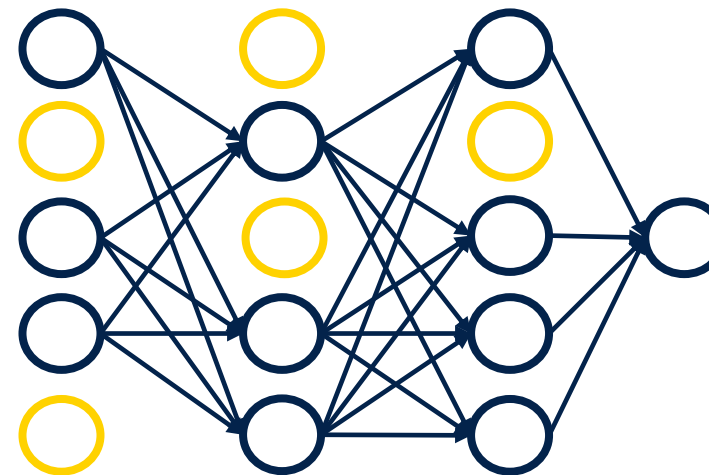
- Limite les solutions trop “agressives” en empêchant les poids d’exploser
- Favorise les solutions simples en réduisant la variance
- Souvent utile avec réseaux profonds

Techniques de régularisation : Dropout

Réseau sans dropout



Réseau avec dropout



- Consiste à « **désactiver** » **des sorties de neurones aléatoirement** (avec une probabilité prédéfinie) pendant la phase d'apprentissage
- Chaque neurone étant possiblement inactif pendant une itération d'apprentissage, cela force chaque unité à « bien apprendre » indépendamment des autres et évite ainsi la « coadaptation »
- Force le réseau à apprendre des représentations plus robustes

Techniques de régularisation : Batch Normalisation

La normalisation par lots **normalise chaque feature indépendamment** dans le mini-lot d'apprentissage. Comme la normalisation des lots dépend de la taille des lots, elle n'est pas efficace pour les petites tailles de lots.

- Normalise les activations intermédiaires
- Permet souvent un apprentissage plus stable → meilleure vitesse
- Agit aussi comme légère régularisation → régularisation implicite

1 Batch with 3 samples mean std_dev

Features	1	3	8	4	2.94
x_1	1	3	8	4	2.94
x_2	3	4	3	3.33	0.471
x_3	5	6	2	4.33	1.69
x_4	7	2	1	3.33	2.62

Normalization across mini-batch,
independently for each feature

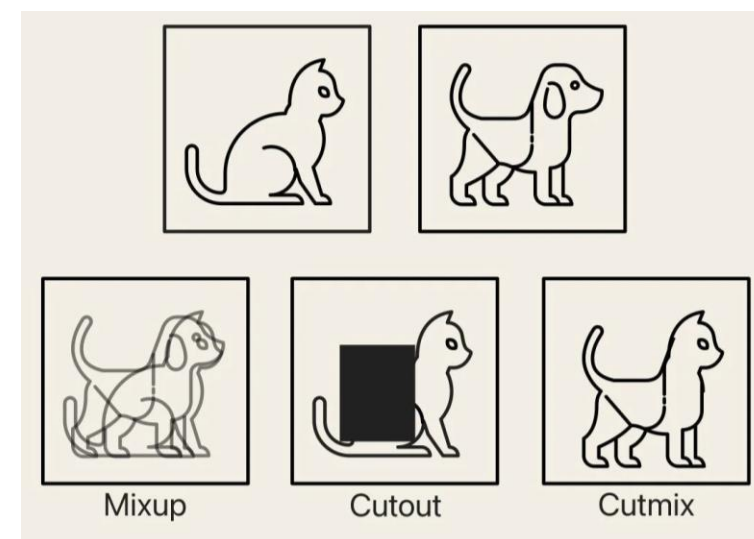
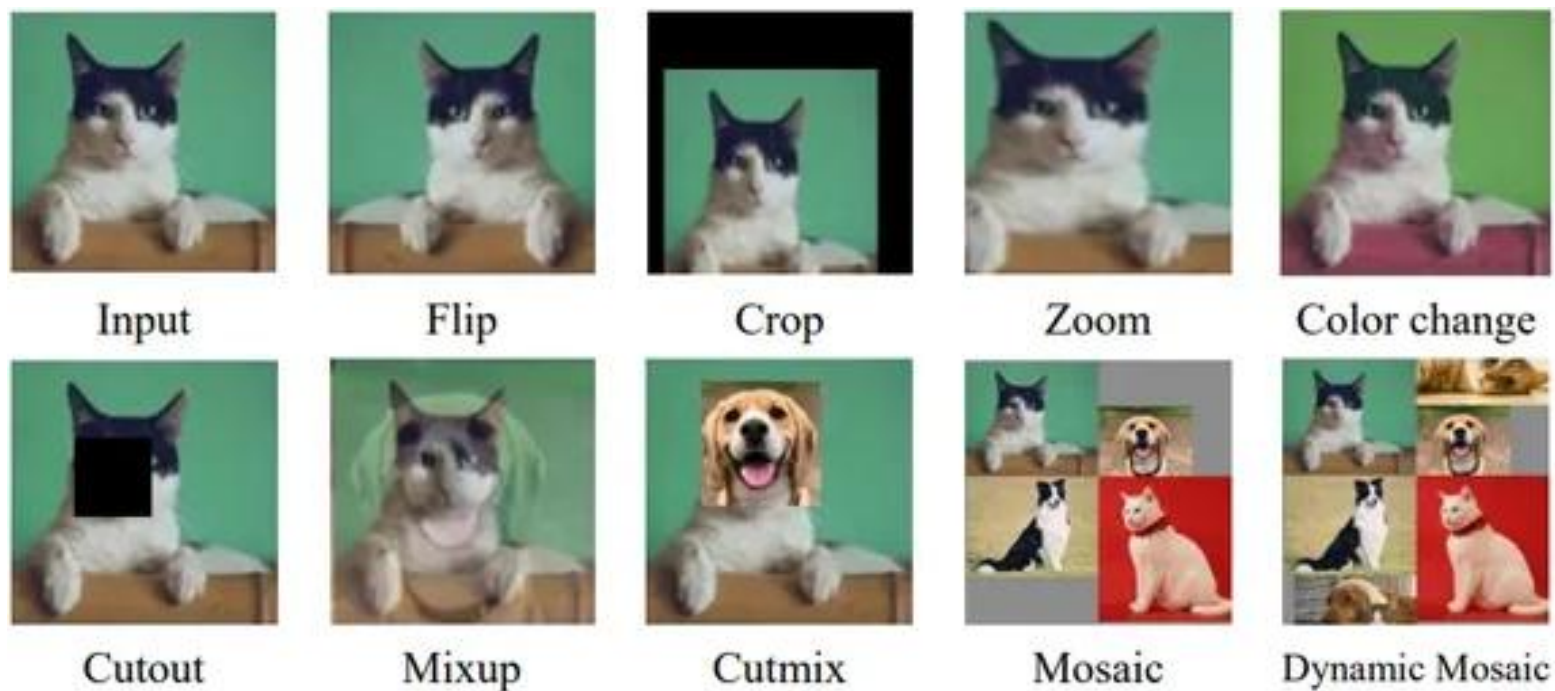
Attention

Pas idéal pour RNN et Transformers

Techniques de régularisation : Data augmentation

On enrichit artificiellement les données d'entraînement.

- Rotation, jitter, crop, flip, noise, zoom, mixup / cutout / cutmix, ...
- Crucial pour la vision, utile en audio et NLP (synthetic noise)



Transfer learning comme régularisation

Le transfer learning **agit comme régularisation** car le modèle commence dans une zone de paramètres déjà « raisonnable ». C'est une régularisation **par biais inductif** : on force le réseau à réutiliser des représentations déjà apprises.

Approche classique :

1. Utiliser un **backbone pré-entraîné** (ImageNet, CLIP, BERT...)
2. Ajouter une **tête** adaptée à la tâche cible
3. Geler le backbone
4. Fine-tuning partiel
5. Fine-tuning complet (optionnel)

Effets bénéfiques :

- Réduit le besoin en données
- Apprentissage plus stable
- Empêche l'overfitting en début de training
- Accélère la convergence

Bonnes pratiques et outils

L'optimisation et la régularisation doivent rester traçables et méthodiques

Principes clés

- L'optimisation et la régularisation doivent être **traçables, répétables, documentées**.
- Utiliser un **experiment tracker** (MLflow, ClearML, ...).
- Garder un **seed** fixe au début pour comparer équitablement.
- Toujours conserver :
 - version du code, résultats et logs
 - dataset utilisé
 - hyperparamètres
- Tester des *hyperparamètre* de manière contrôlée : varier 1 paramètre à la fois.

Outils modernes

- **Optuna** (Bayesian + pruning)
- **Ray Tune** (distribué, scalable)
- **Weights & Biases Sweeps**
- **KerasTuner**
- **MLflow Tracking**



IV. Modélisation supervisée et avancée en Deep Learning



Comprendre et implémenter des architectures profondes adaptées aux différents types de données

4.1

**Réseaux de neurones
artificiels**

4.2

**CNN et vision par
ordinateur**

4.3

**RNN et traitement des
données séquentielles**

4.4

**Métriques d'évaluation
des performances**

4.5

**Optimisation et
régularisation**

4.6

**IA générative : GAN,
VAE**

4.7

**Attention, Transformers,
RAG & LLM**

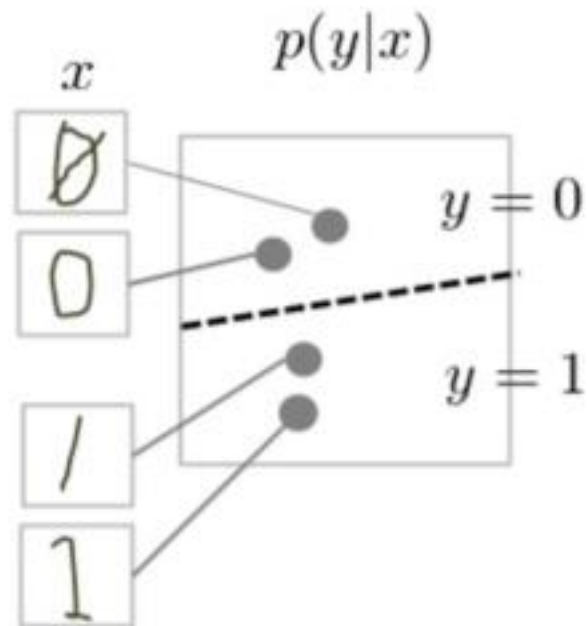
4.8

**Modélisation
multimodale**

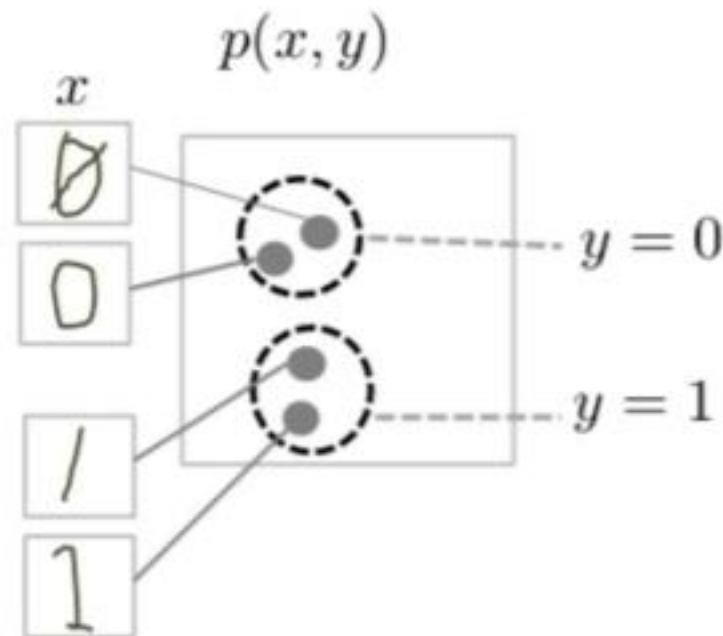
Modèles discriminatifs vs génératifs

Les modèles génératifs brillent dans les tâches nécessitant la génération de données, la détection d'anomalies ou le traitement de données manquantes, tandis que les modèles discriminatifs sont la solution pour des prédictions de haute précision dans les tâches de classification ou de régression.

Discriminatif : modélise $p(y | x)$



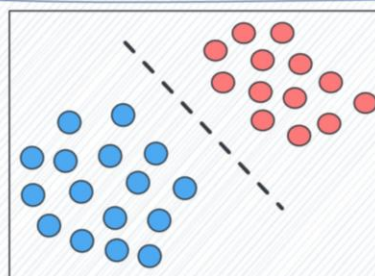
Génératif : modélise $p(x)$ ou $p(x, y)$



*https://www.carmatec.com/fr_fr/blog/modeles-generatifs-et-modeles-discriminatifs-lequel-utiliser/#

Modèles discriminatifs vs génératifs

Modèles discriminatifs



Apprend la frontière de décision entre les classes

Maximise la probabilité conditionnelle : $P(Y|X)$

Estime directement $P(Y|X)$

Ne peut pas générer de nouvelles données

Dédié aux tâches de classification

Ne possède pas de propriétés génératives

Régression logistique

Forêts aléatoires

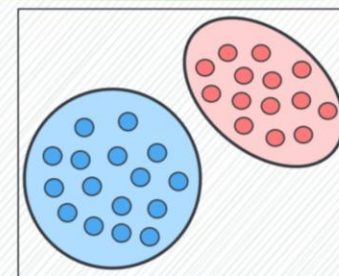
SVM

Réseaux de neurones

Arbre de décision

k-NN

Modèles génératifs



Apprend la distribution des données d'entrée

Maximise la probabilité conjointe : $P(X, Y)$

Estime $P(X|Y)$ pour en déduire $P(Y|X)$ avec la règle de Bayes

Peut générer de nouvelles données

Généralement non utilisé pour les tâches de classification

Possède des propriétés discriminatives

Modèles de Markov cachés

Naïve Bayes

Modèles de mélange gaussien

Analyse discriminante gaussienne

LDA

Réseaux bayésiens

Qu'est-ce qu'un modèle génératif ?

Un **modèle génératif** apprend une distribution $p_{\theta}(x)$ qui **approxime** la distribution des données $p_{data}(x)$. Il sert à **échantillonner** de nouveaux exemples plausibles, à **compléter**, **restaurer** ou **conditionner** des données (texte→image, segmentation→image, etc.).

Deux grandes familles

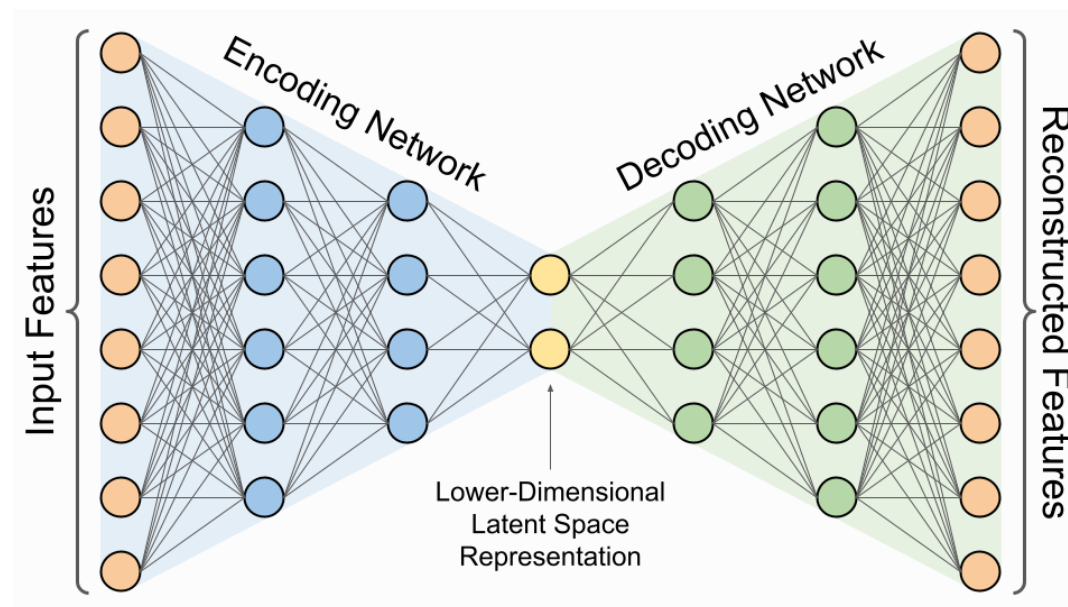
- **Explicites** (likelihood-based) : on modélise directement une **densité** (ou un **borne inférieur**) → **VAE**, **flows**, **diffusion** (score-based).
- **Implicites** (likelihood-free) : on **sait échantillonner** mais pas calculer $p_{\theta}(x)$ → **GAN**.

Notation (images)

- $x \in \mathbb{R}^{H \times W \times C}$: entrée (exemple : image).
- $z \in \mathbb{R}^d$: variable latente.
- θ, ϕ : paramètres (générateur/décodeur, discriminateur/encodeur).
- $\varepsilon \sim \mathcal{N}(0, I)$: bruit gaussien.

Autoencodeur (AE)

Rappel : Les réseaux AE apprennent à **encoder les données en une représentation compacte** puis à les **reconstruire**, capturant ainsi les caractéristiques essentielles. Les autoencodeurs sont **efficaces pour extraire les caractéristiques principales des données**, facilitant ainsi l'analyse et la détection d'anomalies.



$$x \rightarrow [\text{Encodeur } q_{\phi}(z|x)] \rightarrow z \rightarrow [\text{Décodeur } p_{\theta}(x|z)] \rightarrow \hat{x}$$

$$(f_{\phi}) \qquad \qquad \qquad (g_{\theta})$$

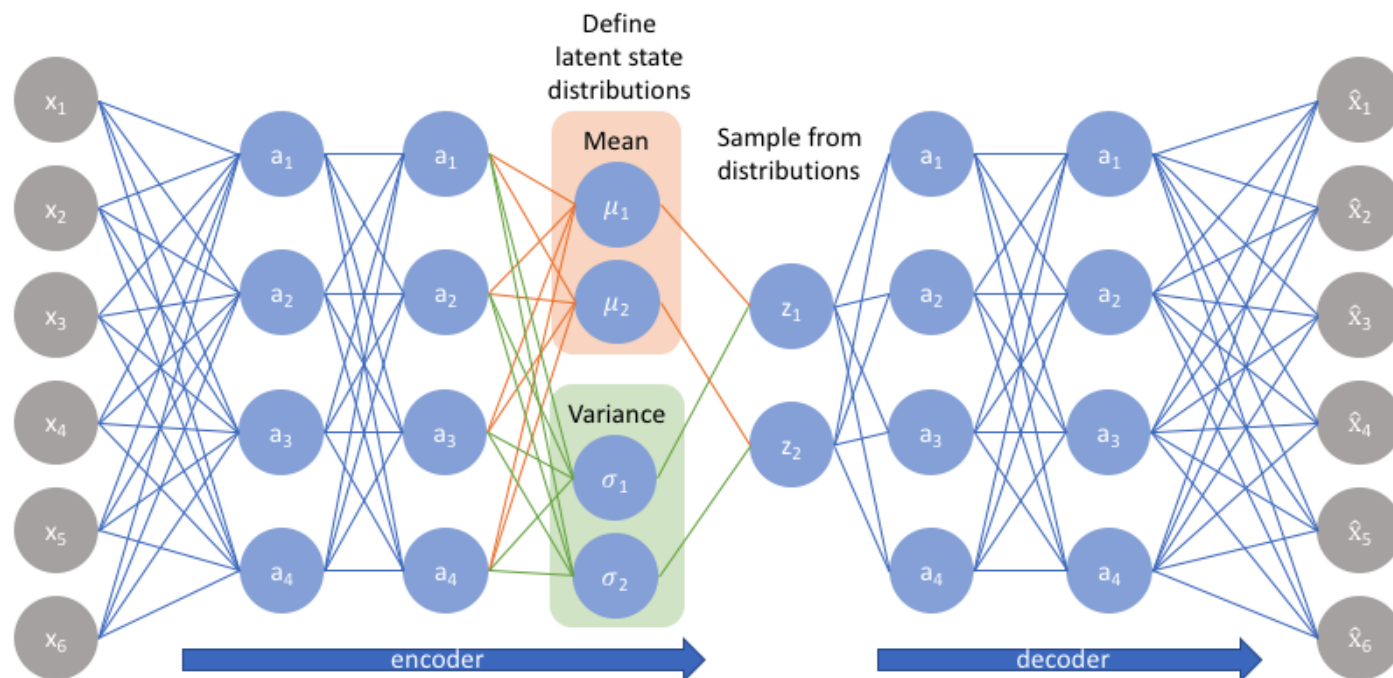
$$\mathcal{L}_{AE} = \|x - \hat{x}\|^2$$

Idée clé : apprendre une **représentation comprimée** z qui permet de reconstruire x .

Variational Autoencoder (VAE)

Les autoencodeurs variationnels (VAE) sont des modèles génératifs utilisés pour générer de nouvelles données sous la forme de variations des données d'entrée sur lesquelles ils sont entraînés. Le VAE est une évolution des autoencodeurs classiques.

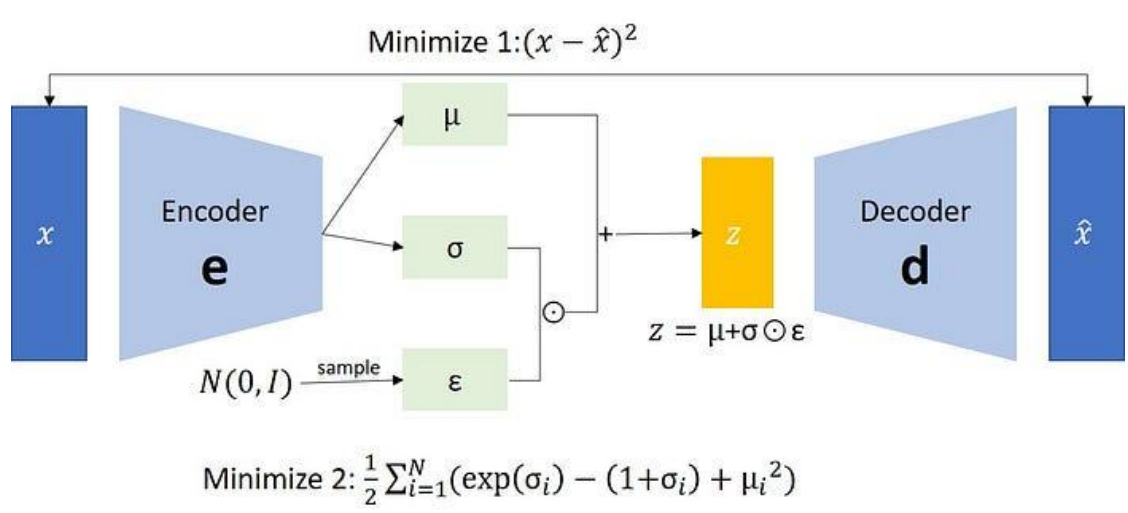
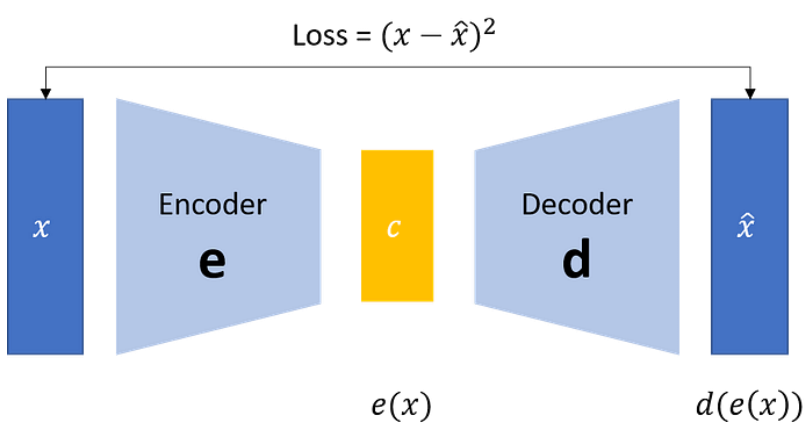
Les VAE encodent les variables latentes des données d'entraînement, non pas sous la forme d'une valeur discrète fixe $\mathbf{z}$, mais sous la forme d'une gamme continue de possibilités exprimée sous forme de distribution de probabilité $p(\mathbf{z})$.



Variational Autoencoder (VAE)

Pour chaque attribut latent des données d'entraînement, les VAE encodent deux vecteurs latents différents : un vecteur de moyenne, « μ », et un vecteur d'écart type, « σ ». En substance, ces deux vecteurs représentent l'éventail des possibilités pour chaque variable latente et la variance attendue à l'intérieur de chaque éventail de possibilités.

En prélevant au hasard des échantillons dans cet éventail de possibilités codées, les VAE peuvent synthétiser de nouveaux échantillons de données qui, tout en étant uniques et originaux, ressemblent aux données d'entraînement d'origine.

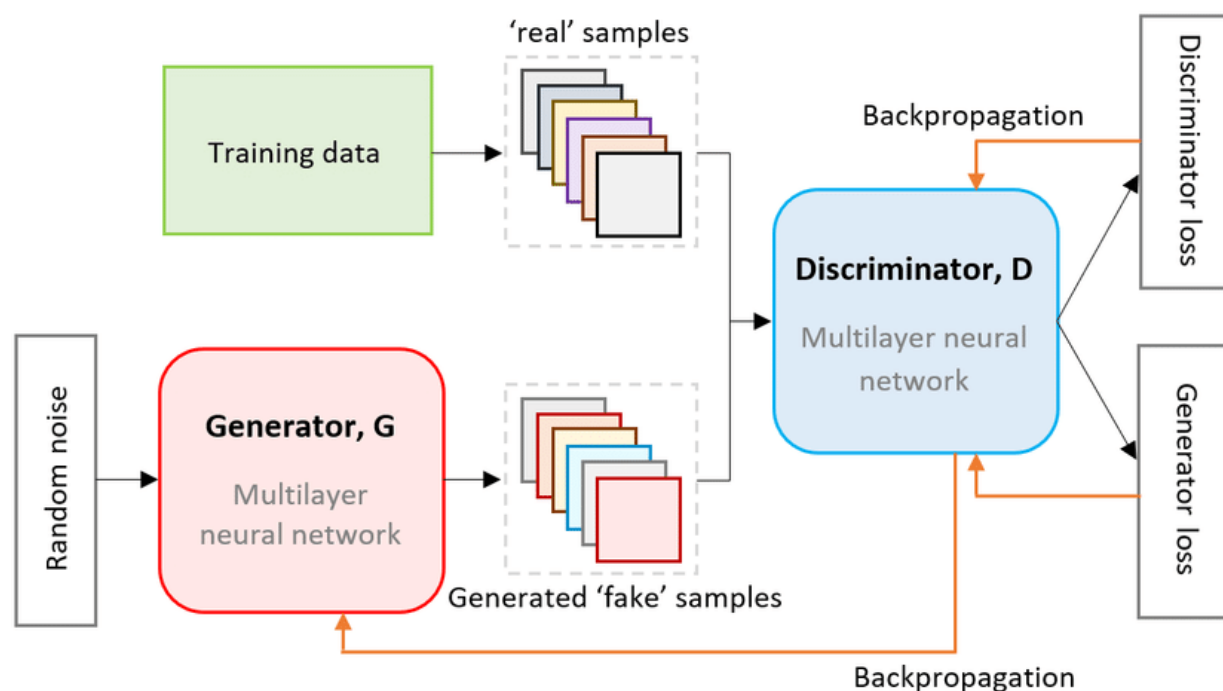


*<https://www.ibm.com/fr-fr/think/topics/variational-autoencoder>

Apprendre un **modèle latent probabiliste**

Generative Adversarial Network (GAN)

Rappel : Un réseau antagoniste génératif (GAN) consiste en **deux réseaux en compétition** : un générateur qui crée des données synthétiques et un discriminateur qui évalue leur authenticité. La capacité des GANs à **générer des données réalistes** les rend intéressants pour l'enrichissement de données et les applications créatives.



Générateur $G_{\theta}(z)$ produit des échantillons.

Discriminateur $D_{\phi}(x)$ distingue réels vs générés.

$$z \sim p(z) \rightarrow G_{\theta} \rightarrow x_{\text{fake}} \rightarrow \neg$$

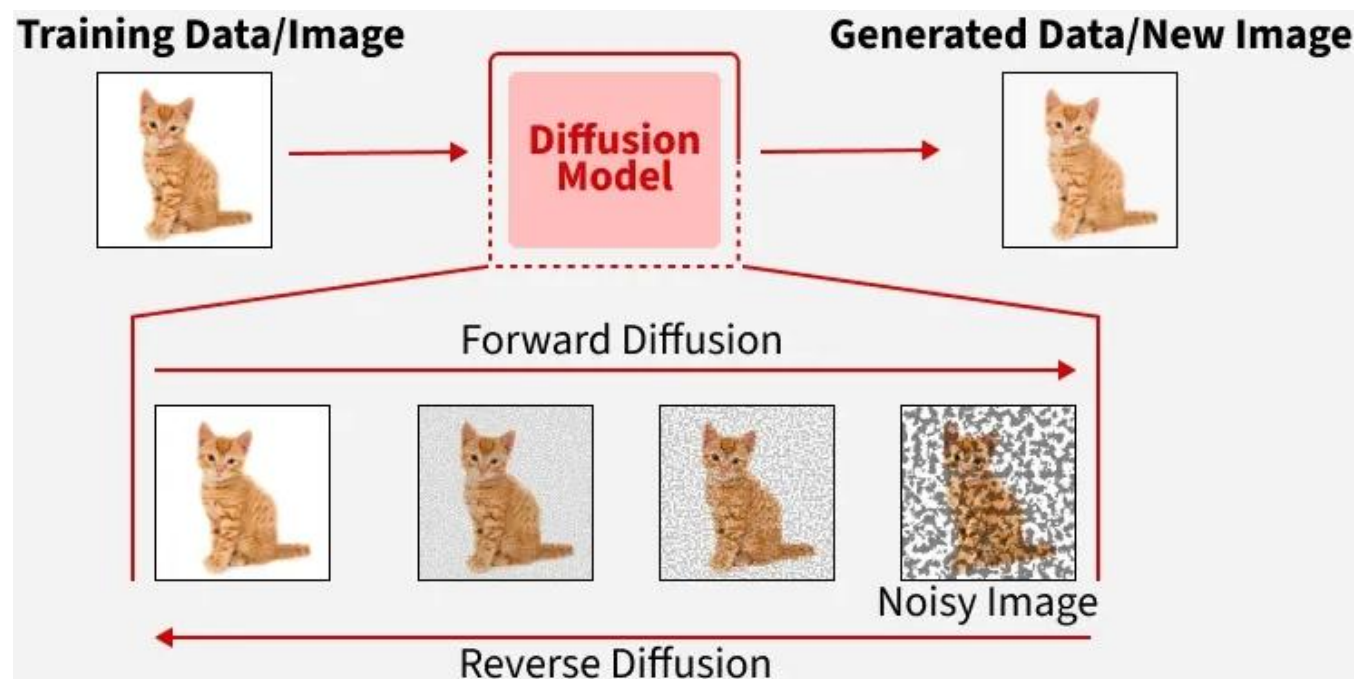
$$x_{\text{real}} \rightarrow D_{\phi} \rightarrow \{\text{réel/faux}\}$$

*<https://developers.google.com/machine-learning/gan?hl=fr>

*<https://machinelearningmastery.com/what-are-generative-adversarial-networks-gans/>

Modèles de diffusion

Les modèles de diffusion sont un type d'IA générative qui crée de nouvelles données (images, audio ou vidéo) à partir d'un bruit aléatoire qu'ils transforment progressivement en informations exploitables. Leur fonctionnement repose sur la **simulation d'un processus de diffusion** : les données sont lentement altérées par du bruit pendant l'entraînement, puis le modèle apprend à inverser ce processus étape par étape. Ainsi, il apprend à générer des échantillons de haute qualité à partir de zéro.



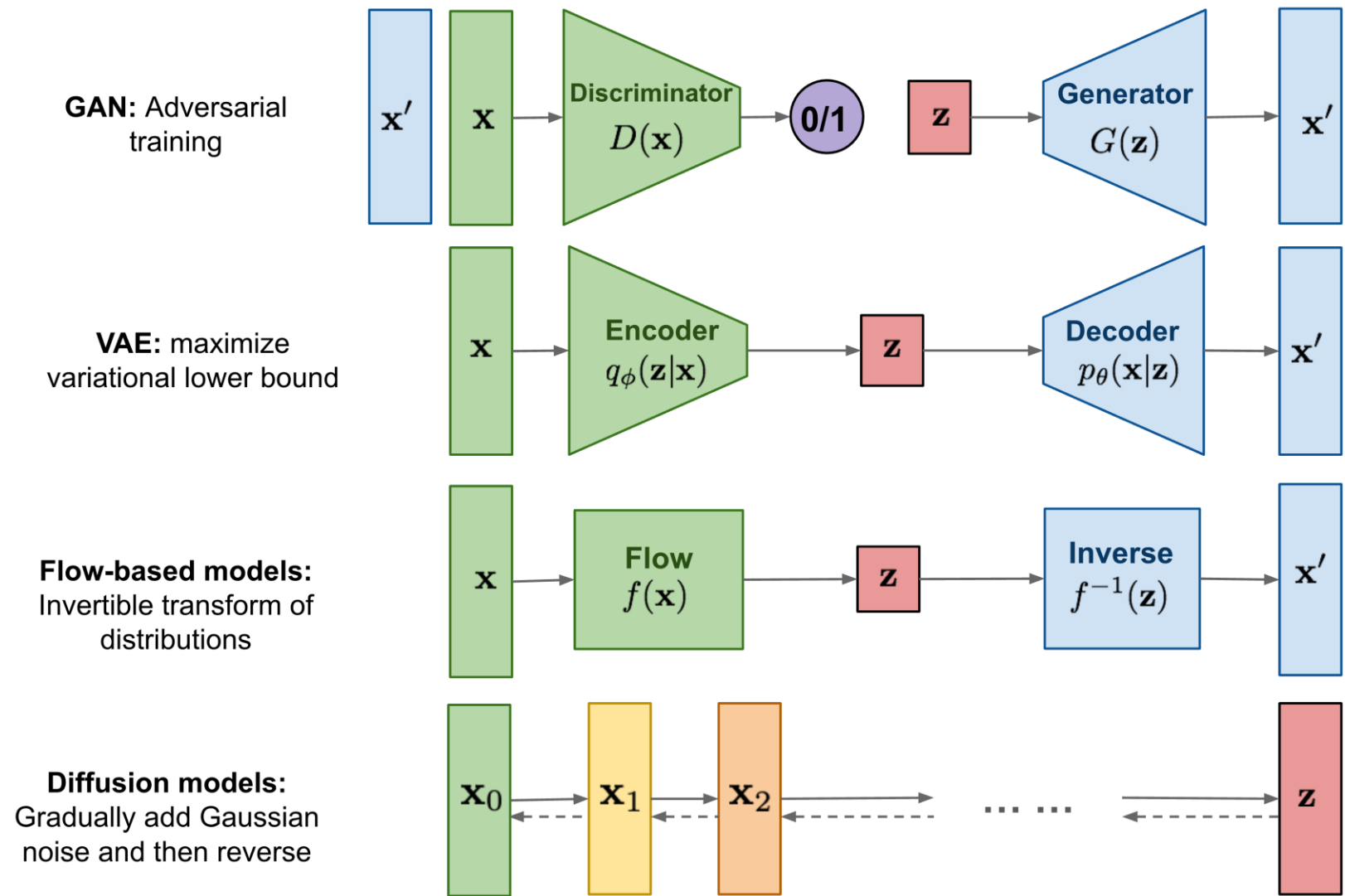
Processus avant (forward) : ajoute du bruit gaussien en T pas.

Processus inverse (reverse) appris par un réseau (U-Net + attention).

*<https://www.superannotate.com/blog/diffusion-models>

*<https://lilianweng.github.io/posts/2021-07-11-diffusion-models/>

Modèles génératifs : résumé



Modèles génératifs : résumé

Critère	AE	VAE	GAN	Diffusion
Paradigme	Reconstruction déterministe	Modèle latent probabiliste (approx. bayésienne)	Modèle implicite adversarial (divergence / IPM)	Modèle de débruitage progressif (score-based / diffusion)
Perte pratique (forme usuelle)	MSE / BCE entre x et $\hat{x}$	Reconstruction + KL (souvent β -VAE)	D : CE binaire / hinge / WGAN-GP ; G : non-saturant ou WGAN	MSE sur le bruit ε (ou prédiction x_0/v , (pondérée par un schedule
Variable latente	Oui (non probabiliste)	Oui, avec prior explicite $p(z) = \mathcal{N}(0, I)$	Optionnelle (conditionné/ non-conditionné)	Optionnelle (LDM : diffusion en latent d'un VAE)
Qualité des échantillons	Correcte (souvent flou)	Bonne (peut être plus lisse)	Très élevée (net, détaillé)	Très élevée (détaillé, peu de défauts)
Stabilité entraînement	Haute	Haute (attention au collapse postérieur)	Difficile (équilibre D/G)	Haute (sensibles aux schedules, mais robustes)
Vitesse d'échantillonnage	Très rapide	Rapide	Très rapide (une passe)	Lente (multi-pas) → accéléralbe (DDIM, DPM-Solver, distillation)
Contrôle / Conditionnement	Limité	Bon (via prior/latent)	Bon (classe, texte, etc.)	Excellent (CFG, cross-attention, ControlNet, etc.)
Coût d'entraînement	Faible	Modéré	Modéré → Élevé (stabilisation)	Élevé (U-Net lourds, longues schedules)

IV. Modélisation supervisée et avancée en Deep Learning



Comprendre et implémenter des architectures profondes adaptées aux différents types de données

4.1

**Réseaux de neurones
artificiels**

4.2

**CNN et vision par
ordinateur**

4.3

**RNN et traitement des
données séquentielles**

4.4

**Métriques d'évaluation
des performances**

4.5

**Optimisation et
régularisation**

4.6

**IA générative : GAN,
VAE**

4.7

**Attention, Transformers,
RAG & LLM**

4.8

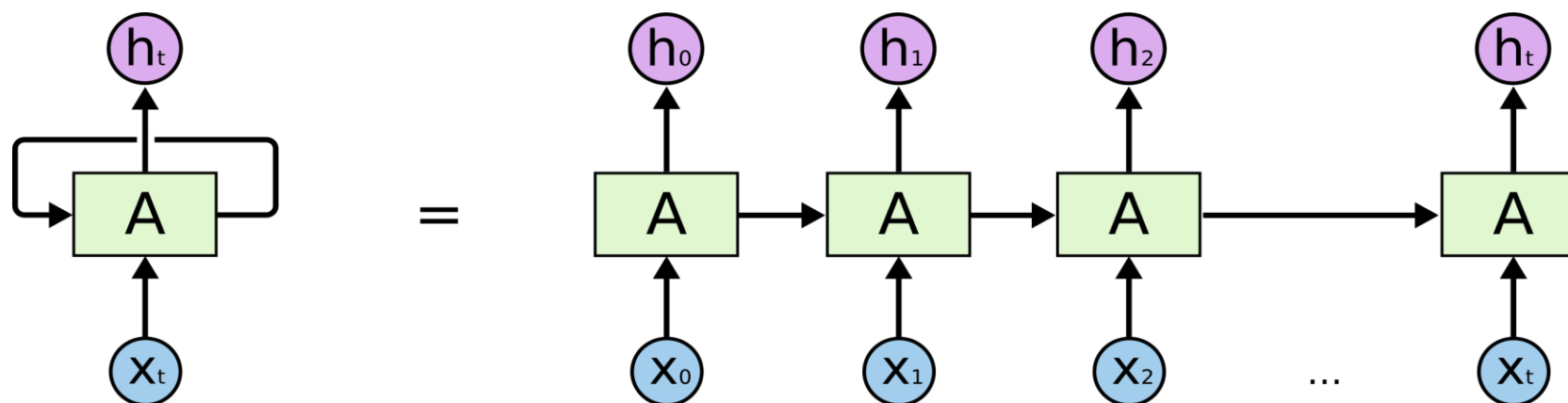
**Modélisation
multimodale**

Pourquoi dépasser les RNN ?



Rappel : ce que font les RNN !

Un RNN traite une séquence **pas à pas**. À l'instant t , l'état caché h_t dépend de l'entrée courante x_t et de l'état précédent h_{t-1} . Cette structure est naturelle pour les séquences, mais elle introduit une **dépendance temporelle forte** dans le calcul.



On peut l'écrire, de manière simplifiée :

$$h_t = f(W_x x_t + W_h h_{t-1} + b)$$

Cette récursion rend l'apprentissage **séquentiel** : on ne peut pas calculer h_t sans avoir calculé h_{t-1}

Pourquoi dépasser les RNN ?



Les limites majeures des RNN

Les RNN souffrent de plusieurs difficultés structurelles :

- **Vanishing gradients / exploding gradients** lors de la rétropropagation à travers le temps (BPTT), ce qui rend difficile l'apprentissage de dépendances longues.
- **Goulot d'étranglement séquentiel** : le calcul n'est pas naturellement parallélisable sur les positions d'une séquence.
- Difficulté à modéliser efficacement des dépendances de long terme, même avec des variantes comme **LSTM** ou **GRU**, qui améliorent la situation sans supprimer totalement le problème.

Pourquoi dépasser les RNN ?



Pourquoi les Transformers changent la donne.

Le Transformer remplace la récurrence par un mécanisme où **chaque position peut regarder directement les autres positions** via l'attention. Cela permet :

- une **meilleure parallélisation** à l'entraînement ;
- des **chemins de dépendance plus courts** entre deux tokens éloignés dans la séquence ;
- une meilleure montée en échelle sur GPU/TPU, ce qui a favorisé l'essor des très grands modèles de langage.

les Transformers n'ont pas “supprimé les séquences”, ils ont remplacé le **traitement récursif** par un **traitement global avec pondération des dépendances**.

Comprendre l'attention



Intuition

L'idée de l'attention est simple : au lieu de traiter tous les mots précédents comme également importants, le modèle apprend à **ponderer** quelles parties de l'entrée sont les plus utiles pour représenter ou générer un token donné.

Exemple de phrase :

« Le chat que le voisin a adopté dort. »

Pour comprendre ou générer « **dort** », le modèle doit surtout relier ce verbe au sujet « **chat** », et non à tous les mots avec la même intensité. L'attention permet précisément ce mécanisme de focalisation.



Comprendre l'attention



Query, Key, Value

Le mécanisme d'attention revient à **transformer chaque vecteur X représentant un token de la séquence d'entrée en un vecteur de contexte Z qui prend en compte une relation particulière avec les autres tokens de la séquence.**

À partir de l'entrée X , on calcule trois projections linéaires :

$$Q = XW_Q, K = XW_K, V = XW_V$$

Chaque token est projeté dans trois espaces :

- **Query (Q)** : ce que cherche ce token ;
- **Key (K)** : comment chaque token peut être retrouvé ;
- **Value (V)** : l'information transportée par le token.

On peut résumer l'idée par la question :

“Pour ce token, à quels autres tokens faut-il prêter attention, et avec quel poids ?”

Comprendre l'attention



Scaled dot-product attention

Le cœur du mécanisme est la formule suivante :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

où d_k est la dimension des vecteurs de clés. Cette formulation est explicitement introduite dans *Attention Is All You Need*.

Lecture pas à pas de la formule

Étape 1 — Similarité QK^T

Le produit QK^T calcule à quel point une requête est alignée avec chaque clé. Plus le score est grand, plus la clé est jugée pertinente pour la requête.

Étape 2 — Mise à l'échelle par $\sqrt{d_k}$

On divise par $\sqrt{d_k}$ pour éviter que les produits scalaires deviennent trop grands lorsque la dimension augmente, ce qui saturerait la softmax et dégraderait les gradients.

Étape 3 — Softmax

La softmax transforme les scores en **poids positifs** qui somment à 1. On obtient donc une distribution d'attention.

Étape 4 — Somme pondérée des valeurs

La sortie est une combinaison linéaire des vecteurs V , pondérée par ces coefficients d'attention.

Comprendre l'attention



Multi-head attention

Au lieu d'une seule attention, on utilise plusieurs "têtes" d'attention en parallèle. Chaque tête peut apprendre des relations différentes : accord sujet-verbe, co-référence, dépendances syntaxiques, indices sémantiques, etc. Les sorties des têtes sont ensuite concaténées puis reprojettées.

Si l'on note $head_i$ la sortie de la tête i , alors :

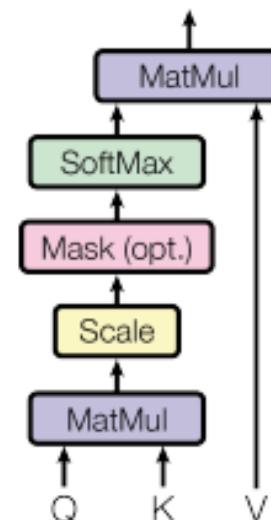
$$head_i = Attention(Q_i, K_i, V_i)$$

et la sortie multi-tête s'écrit :

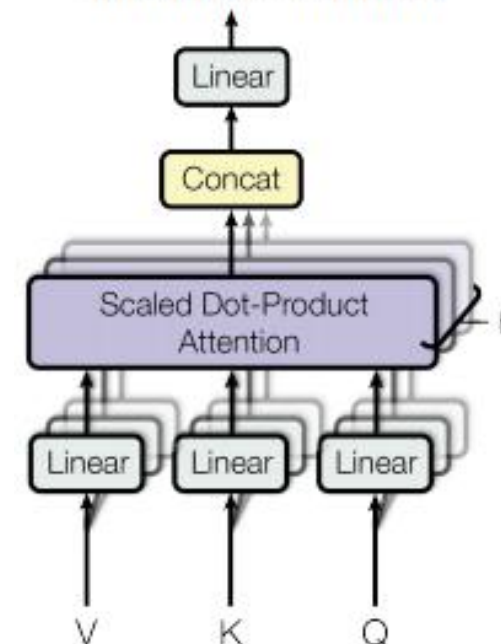
$$MultiHead(Q, K, V) = Concat(head_1, \dots, head_h)W_O$$

où h est le nombre de têtes.

Scaled Dot-Product Attention



Multi-Head Attention



Transformers : bloc de base



Idée générale

Le **Transformer** est une architecture fondée principalement sur des mécanismes d'**attention**, sans récurrence ni convolution dans sa formulation de base. Dans l'article fondateur *Attention Is All You Need*, l'idée centrale est que l'on peut traiter une séquence en permettant à chaque position de regarder directement les autres positions pertinentes, plutôt que de propager l'information pas à pas comme dans un RNN.

Le **bloc de base** du Transformer est l'unité modulaire que l'on empile plusieurs fois pour construire un encodeur, un décodeur, ou des variantes plus modernes. C'est donc le “brique élémentaire” qu'il faut absolument comprendre avant d'étudier BERT, GPT, T5 ou les LLM.

Transformers : bloc de base



Vue d'ensemble du bloc

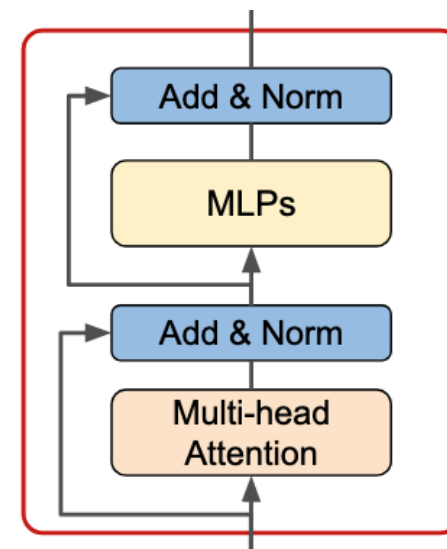
Dans sa forme la plus classique, un bloc Transformer contient les éléments suivants :

1. une **multi-head self-attention** ;
2. une **connexion résiduelle** suivie d'une **normalisation** ;
3. un **réseau feed-forward positionnel** (souvent appelé MLP ou FFN) ;
4. une seconde **connexion résiduelle** suivie d'une **normalisation**.

On peut le résumer ainsi :

$$X \rightarrow \text{Self-Attention} \rightarrow \text{Add \& Norm} \rightarrow \text{Feed-Forward} \rightarrow \text{Add \& Norm}$$

Cette structure apparaît dans le Transformer original et constitue encore aujourd'hui la base conceptuelle de très nombreux modèles.



Transformers : bloc de base



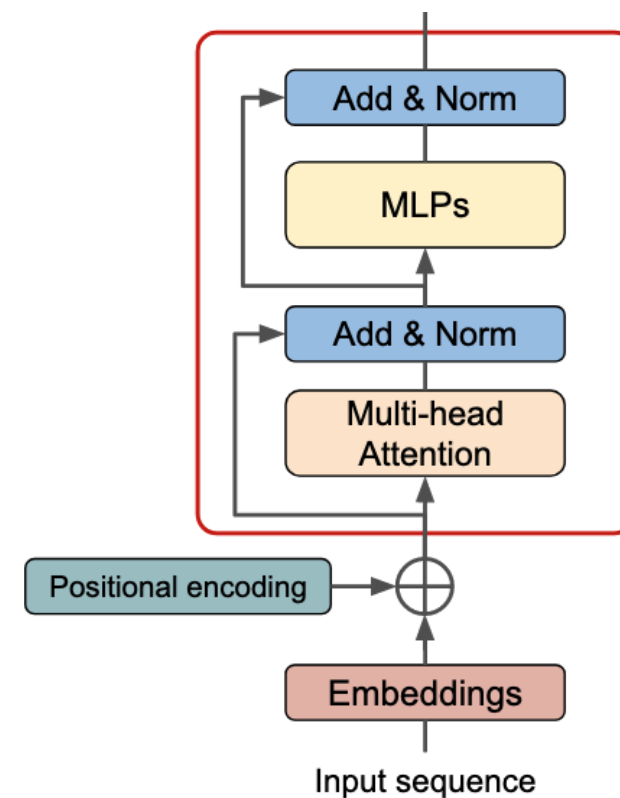
Entrée du bloc : embeddings + information de position

Avant d'entrer dans un bloc Transformer, chaque token de la séquence est représenté par un **vecteur d'embedding**. Comme le Transformer ne traite pas la séquence récursivement, il faut aussi lui fournir une information sur l'**ordre** des tokens, via des **encodages positionnels**. Dans le papier original, ces encodages sont sinusoïdaux, mais beaucoup de variantes modernes utilisent des positions apprises ou d'autres schémas plus avancés.

On peut donc voir l'entrée du bloc comme :

$$X = E + P$$

où E représente les embeddings des tokens et P l'information de position. Sans cette composante, le modèle saurait quelles sont les unités présentes, mais pas dans quel ordre elles apparaissent.



Transformers : bloc de base



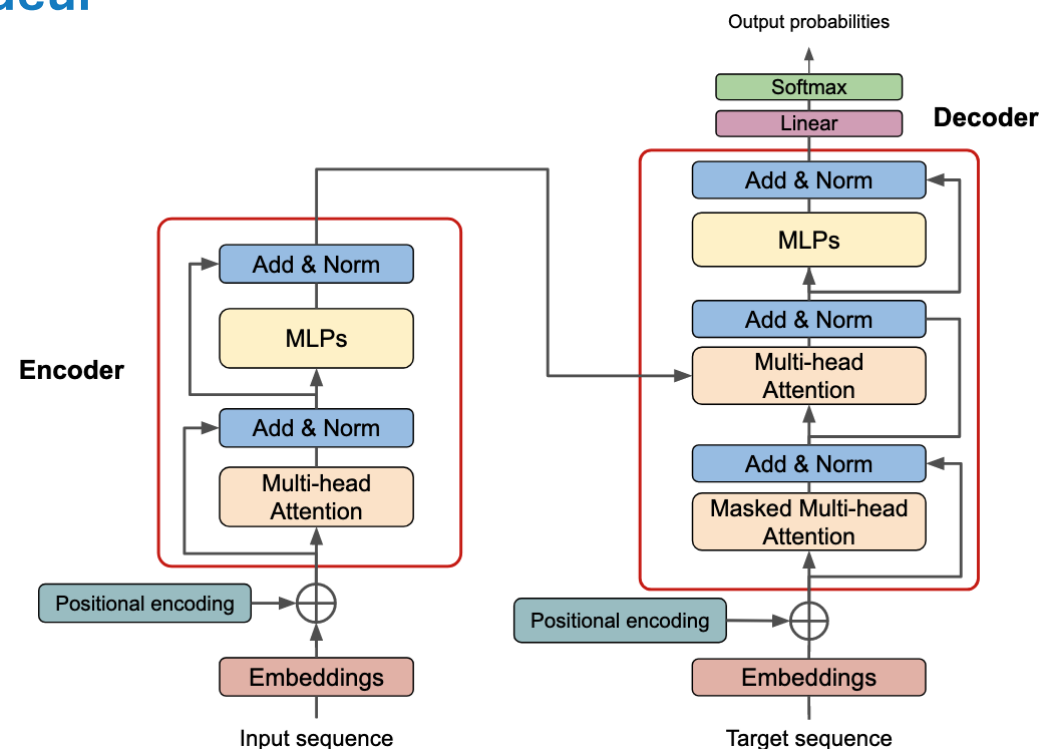
Différence entre bloc d'encodeur et bloc de décodeur

Dans un **encodeur**, la self-attention est généralement **non masquée** : chaque token peut regarder tous les autres tokens de la séquence. C'est adapté aux tâches de compréhension, où l'on veut exploiter tout le contexte disponible.

Dans un **décodeur**, la self-attention est **masquée causalement** : un token ne peut pas consulter les tokens futurs. Cela garantit que la génération autoregressive reste cohérente. Dans les architectures encodeur-décodeur, le décodeur comporte en plus une **cross-attention** vers les sorties de l'encodeur.

Le décodeur contient en général :

- une **self-attention masquée** sur la sortie déjà produite ;
- une **cross-attention** vers les représentations de l'encodeur .



Transformers : bloc de base



Tableau de synthèse

Composant	Rôle principal
Embeddings + positions	Représenter les tokens et leur ordre dans la séquence
Self-attention	Faire interagir les positions entre elles
Multi-head	Capturer plusieurs types de relations en parallèle
Résidu	Préserver l'information utile et faciliter l'apprentissage profond
LayerNorm	Stabiliser l'entraînement
Feed-forward	Transformer non linéairement chaque position

Encoder-only, decoder-only, encoder-decoder



Pourquoi distinguer plusieurs familles de Transformers ?

Le Transformer initial a été proposé dans un cadre **encodeur-décodeur** pour la traduction automatique, mais la même brique fondamentale (attention + MLP + résidus + normalisation) a ensuite donné naissance à plusieurs familles spécialisées selon le type de tâche visée.

L'idée centrale est la suivante :

- si l'on veut surtout **comprendre** un texte, on privilégie souvent un **encodeur** ;
- si l'on veut surtout **générer** du texte de gauche à droite, on privilégie souvent un **décodeur causal** ;
- si l'on veut **transformer une séquence d'entrée en une séquence de sortie** (traduction, résumé, reformulation), le schéma **encodeur-décodeur** est particulièrement naturel.

Encoder-only, decoder-only, encoder-decoder



Encoder-only

Un modèle **encoder-only** prend une séquence en entrée et calcule, pour chaque token, une représentation contextuelle en exploitant l'information **des deux côtés** du token (contexte gauche et contexte droit). Cette idée est particulièrement emblématique de **BERT**.

BERT (*Bidirectional Encoder Representations from Transformers*) introduit un préentraînement bidirectionnel profond, conçu pour produire des représentations de haute qualité facilement réutilisables sur de nombreuses tâches en ajoutant une petite tête de sortie.

Les architectures encoder-only sont particulièrement adaptées aux tâches de : **(1) Classification de texte ; (2) Reconnaissance d'entités nommées (NER) ; (3) Inférence en langage naturel (NLI) ; (4) Question answering extractif.**

Encoder-only, decoder-only, encoder-decoder



Decoder-only

Un modèle **decoder-only** génère la séquence **autoregressivement**, c'est-à-dire qu'il prédit le token suivant à partir des tokens précédents. Pour garantir ce comportement, l'attention est **masquée causalement** : un token ne peut pas “voir” les tokens futurs.

Pourquoi un masque causal ?

Lorsqu'on veut prédire le mot x_t , il serait incohérent d'autoriser le modèle à consulter x_{t+1} , puisque ce mot appartient au futur de la génération. Le masque causal impose donc une structure triangulaire qui interdit l'accès aux positions futures.

Les architectures decoder-only sont particulièrement adaptées à : (1) La **génération de texte libre** ; (2) Les **assistants conversationnels** ; (3) La **génération de code** ; (4) Les **LLM génératifs modernes**.

Exemples emblématiques

- **GPT-3** est un modèle autoregressif très grand qui a montré la puissance du scaling et du few-shot prompting.
- **LLaMA** a démontré qu'il était possible d'obtenir de très bonnes performances avec des modèles efficaces entraînés sur de grands volumes de données publiques.

Encoder-only, decoder-only, encoder-decoder



Encoder-decoder

Dans une architecture **encoder-decoder**, l'encodeur lit et représente la séquence d'entrée, puis le décodeur génère la séquence de sortie en s'appuyant à la fois sur les tokens déjà générés **et** sur la représentation fournie par l'encodeur. C'est l'architecture du Transformer original pour la traduction.

Pourquoi ce format est-il naturel ?

Parce qu'il correspond exactement à une situation de **transduction de séquence** : on reçoit une séquence source x , puis on génère une séquence cible y . C'est très adapté à : (1) la **traduction** ; (2) le **résumé automatique** ; (3) la **reformulation / paraphrase** ; (4) certaines formes de **question answering génératif**.

Exemple emblématique : T5

T5 (*Text-To-Text Transfer Transformer*) pousse cette logique très loin : **toutes les tâches NLP sont reformulées comme des tâches texte-vers-texte**. Une classification, par exemple, devient une génération d'étiquette textuelle ; une traduction devient une génération conditionnelle classique ; un résumé devient la génération d'un texte plus court.

Encoder-only, decoder-only, encoder-decoder



Tableau comparatif

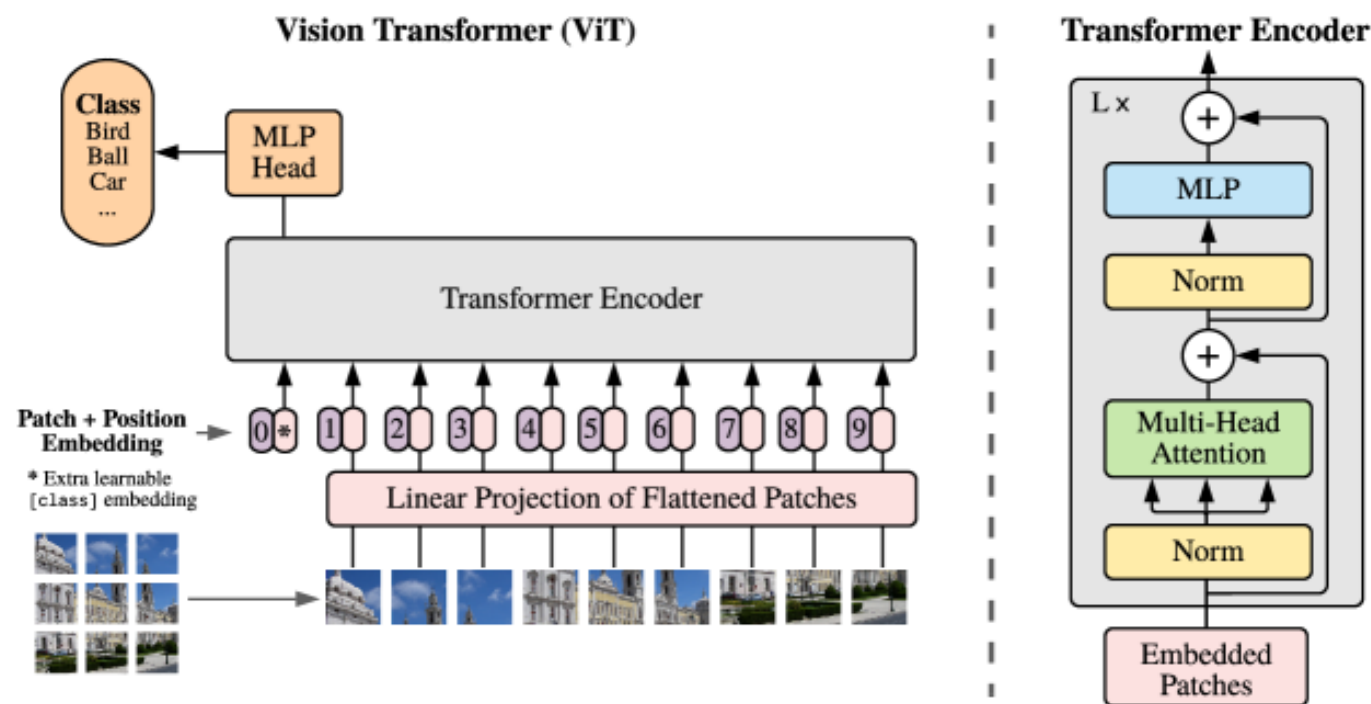
Famille	Vue sur le contexte	Type de tâche dominant	Exemple
Encoder-only	Bidirectionnelle	Compréhension / représentation	BERT
Decoder-only	Unidirectionnelle causale	Génération autoregressive	GPT-3, LLaMA
Encoder-decoder	Entrée complète + génération causale en sortie	Transduction séquence-à-séquence	T5

Vision Transformer (ViT)



Idée générale

Le ViT traite une image comme une **séquence de patches**. Au lieu de lire des mots, il lit de petits blocs d'image. Chaque patch est converti en vecteur (embedding), puis la séquence de patches est envoyée dans un **encodeur Transformer standard**.



Vision Transformer (ViT)



Idée générale

image $\rightarrow$ *patches* $\rightarrow$ *embeddings* $\rightarrow$ *Transformer encoder*

Supposons une $x \in \mathbb{R}^{H \times W \times C}$. On la découpe en patches de taille $P \times P$. Le nombre total de patches est alors :

$$N = \frac{HW}{P^2}$$

Chaque patch est aplati puis projeté linéairement dans un espace de dimension D , exactement comme on projette un token de texte dans un embedding.

Exemple simple

Si l'image est de taille 224×224 et que l'on prend des patches 16×16 , alors on obtient :

$$N = \frac{224 \times 224}{16^2} = 196$$

L'image est donc transformée en une séquence de **196 tokens visuels**.

Vision Transformer (ViT)



En quoi le ViT diffère-t-il d'un CNN ?

Les **CNN** incorporent naturellement des biais inductifs utiles à la vision, comme la **localité** et une certaine **équivalence aux translations**. Le ViT, lui, part d'un cadre plus général et apprend davantage à partir des données, ce qui devient très compétitif quand le préentraînement est suffisamment large.

Pourquoi le ViT est intéressant conceptuellement ?

Le ViT montre que l'attention permet d'agréger **très tôt** de l'information globale dans l'image, ce qui différencie son comportement des CNN classiques et explique certaines de ses propriétés de représentation.

Il constitue aussi un excellent pont vers les **modèles multimodaux**, où l'on traite conjointement texte, image, son ou vidéo sous forme de séquences ou de tokens spécialisés.

LLM : définition et pipeline



Qu'est-ce qu'un LLM ?

Un **Large Language Model** est un modèle de langage de très grande taille, généralement fondé sur un Transformer, entraîné sur de très grands corpus textuels pour modéliser la probabilité des séquences de tokens. Dans la pratique actuelle, beaucoup de LLM performants sont **decoder-only** et entraînés par prédiction du prochain token.

Pipeline simplifié d'un LLM

On peut présenter le pipeline en trois étapes :

1

Préentraînement sur un très grand corpus

2

Adaptation / instruction tuning pour mieux suivre les consignes

3

Éventuellement **alignement par feedback humain** (RLHF ou variantes) pour rendre les réponses plus utiles, plus fidèles aux consignes et moins toxiques

LLM : définition et pipeline



Pourquoi les LLM sont-ils impressionnants ?

La montée en échelle a montré qu'un grand modèle de langage peut exhiber de fortes capacités en **génération**, **raisonnement en contexte**, **adaptation few-shot** et **transfert** sur de nombreuses tâches, même sans fine-tuning spécifique. GPT-3 en est un exemple marquant, avec 175B paramètres et de solides performances en few-shot sur de nombreux benchmarks.

Mais “plus gros” ne veut pas dire “mieux aligné”

Le travail sur **InstructGPT** montre qu'augmenter uniquement la taille ne suffit pas pour bien suivre l'intention de l'utilisateur. Le fine-tuning avec feedback humain peut rendre un modèle **plus utile, plus honnête et moins toxique**, même à taille bien inférieure.

LLM : Modèles emblématiques



Tableau de synthèse

Modèle	Type	Contribution principale
Transformer (2017)	Architecture générale	Remplace récurrence et convolution par l'attention ; base des modèles modernes
BERT (2018/2019)	Encoder-only	Représentations bidirectionnelles puissantes pour la compréhension
T5 (2019/2020)	Encoder-decoder	Unifie les tâches NLP en format texte-vers-texte
GPT-3 (2020)	Decoder-only	Démontre la force du scaling et du few-shot prompting
RAG (2020)	Architecture hybride	Combine mémoire paramétrique et mémoire externe récupérée
ViT (2020/2021)	Transformer pour la vision	Traite les images comme des séquences de patches
InstructGPT (2022)	Alignement	Montre l'intérêt du RLHF pour suivre les instructions humaines
LLaMA / Llama 2 (2023)	Decoder-only open-weight	Modèles efficaces et très influents dans l'open source

RAG : Retrieval-Augmented Generation



Idée générale

Le **RAG** combine : (1) une **mémoire paramétrique** (les poids du modèle) et (2) une **mémoire non paramétrique** externe (documents, base vectorielle, Wikipédia, base métier, etc.) récupérée à la volée.

L'idée fondatrice est que les LLM stockent des connaissances dans leurs paramètres, mais que cette mémoire est limitée, difficile à mettre à jour et peu traçable ; une mémoire externe vient compenser cela.

Pipeline RAG simplifié

1. On encode la requête utilisateur ;
2. On recherche les passages les plus pertinents dans une base ;
3. On injecte ces passages dans le prompt ;
4. Le LLM génère la réponse à partir de la question + du contexte récupéré.

RAG : Retrieval-Augmented Generation



Idée générale

Le RAG apporte plusieurs bénéfices :

- **actualisation** des connaissances sans réentraîner entièrement le modèle ;
- **traçabilité / provenance** des réponses si les passages récupérés sont cités ;
- meilleur comportement sur les tâches **knowledge-intensive** (QA ouverte, documentation métier, support interne, FAQ, etc.).

Limites du RAG

Le RAG n'est **pas** une solution magique. Ses performances dépendent fortement de :

- la qualité du **retriever** ;
- le **chunking** et l'indexation ;
- la façon d'injecter le contexte dans le prompt ;
- la capacité du LLM à réellement **utiliser** les passages récupérés au lieu d'halluciner malgré eux.

RAG : Retrieval-Augmented Generation



Enjeux des LLM / RAG

Hallucinations

Les LLM peuvent générer des contenus "**fluent**" mais faux, phénomène largement étudié sous le nom d'**hallucination**. Ce problème demeure central, y compris dans les systèmes RAG lorsque le contexte récupéré est insuffisant, ambigu, erroné, ou mal exploité.

Fraîcheur et mise à jour des connaissances

Les connaissances apprises paramétriquement deviennent vite obsolètes. Le RAG répond partiellement à cet enjeu en branchant le LLM sur une mémoire externe actualisable.

Explicabilité et provenance

Dans les usages professionnels et scientifiques, il est souvent insuffisant de donner une réponse "convaincante" ; il faut pouvoir **justifier** la réponse, afficher les passages sources, voire noter la confiance ou la couverture documentaire. Le RAG offre ici un avantage structurel par rapport à un LLM "nu".

RAG : Retrieval-Augmented Generation



Enjeux des LLM / RAG

Coût, latence et ingénierie

Les LLM sont coûteux à entraîner et parfois coûteux à servir. Le RAG ajoute une couche système (indexation, base vectorielle, retrieval, orchestration) qui améliore la factualité mais augmente aussi la **complexité d'ingénierie** et parfois la **latence**.

Données privées, sécurité, gouvernance

Dans les environnements métier, brancher un LLM sur des documents internes pose des questions de **confidentialité**, de **droit d'accès**, de **qualité documentaire** et de **gouvernance**. Le RAG n'est utile que si la chaîne documentaire est maîtrisée.

RAG : Retrieval-Augmented Generation



Enjeux des LLM / RAG

Limites actuelles

Parmi les limites importantes à enseigner :

- les Transformers standards ont un coût d'attention quadratique en longueur de séquence dans leur forme de base ;
- les LLM restent sensibles aux hallucinations, à la qualité du prompt et aux biais du corpus d'entraînement ;
- les systèmes RAG dépendent d'un bon retrieval, sans quoi ils peuvent récupérer des documents non pertinents et dégrader la génération .

IV. Modélisation supervisée et avancée en Deep Learning



Comprendre et implémenter des architectures profondes adaptées aux différents types de données

4.1

**Réseaux de neurones
artificiels**

4.2

**CNN et vision par
ordinateur**

4.3

**RNN et traitement des
données séquentielles**

4.4

**Métriques d'évaluation
des performances**

4.5

**Optimisation et
régularisation**

4.6

**IA générative : GAN,
VAE**

4.7

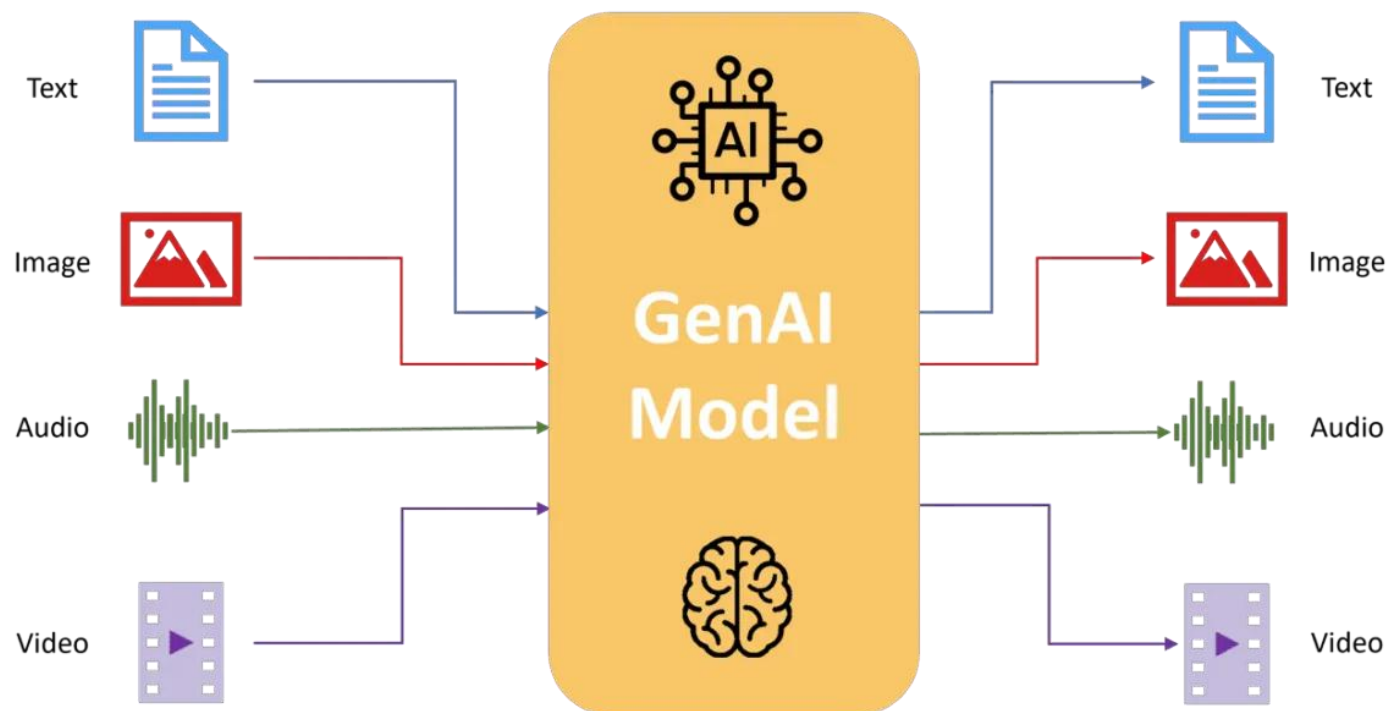
**Attention, Transformers,
RAG & LLM**

4.8

**Modélisation
multimodale**

Qu'est-ce qu'un modèle multimodal ?

Un **modèle multimodal** est un modèle d'apprentissage capable de **traiter, relier et exploiter plusieurs types de données (modalités)** au sein d'un même système : par exemple **texte + image, audio + vidéo**, ou encore **capteurs + langage naturel**. L'idée centrale est que différentes modalités apportent des points de vue complémentaires sur un même phénomène.



Qu'est-ce qu'un modèle multimodal ?

La modélisation multimodale ne se limite pas à “concaténer des données”. Elle implique souvent plusieurs sous-problèmes :

- **Représentation** : comment encoder chaque modalité ?
- **Alignement** : comment mettre en correspondance les éléments d'une modalité avec ceux d'une autre ?
- **Fusion** : comment combiner les informations pour produire une prédiction, une représentation commune ou une génération ?
- **Co-apprentissage** : comment une modalité peut aider une autre, en particulier quand l'une est peu annotée ?

Pourquoi le multimodal ?

Parce que le monde réel est multimodal

Dans la plupart des situations réelles, l'information ne vient pas d'une seule source. Une scène peut comporter des **objets visibles**, des **sons**, un **contexte textuel**, et parfois des **signaux temporels**. Une IA qui ne voit qu'une seule modalité risque donc de perdre une partie du sens global.

Parce que les modalités sont complémentaires

Chaque modalité capture une facette différente d'un même phénomène. Le texte peut expliciter une intention, l'image peut apporter la structure spatiale, l'audio peut révéler un événement ou un état du contexte. Fusionner ces signaux peut donc améliorer la performance du modèle.

Par exemple, en **reconnaissance audio-visuelle de la parole**, les mouvements labiaux peuvent aider lorsque l'audio est bruité ; inversement, l'audio aide lorsque la vidéo est floue ou partiellement masquée.

Pourquoi le multimodal ?

Parce que le multimodal améliore souvent la robustesse

Une modalité peut être incomplète, bruitée ou absente. Le fait d'en disposer de plusieurs peut rendre le système plus robuste, à condition que l'architecture gère correctement les données manquantes et l'équilibre entre les modalités. Les enquêtes récentes sur l'alignement et la fusion multimodales insistent fortement sur cette idée.

Parce qu'il permet de nouvelles tâches

Sans multimodal, certaines tâches n'existent tout simplement pas ou deviennent artificielles :

- **captioning d'image** (générer un texte à partir d'une image) ;
- **VQA** (*Visual Question Answering*) : répondre à une question en utilisant une image ;
- **recherche image ↔ texte** ;
- **agents conversationnels visuels** ;
- **raisonnement sur vidéo + langage** ;
- **applications médicales** combinant imagerie, texte clinique et signaux biologiques.

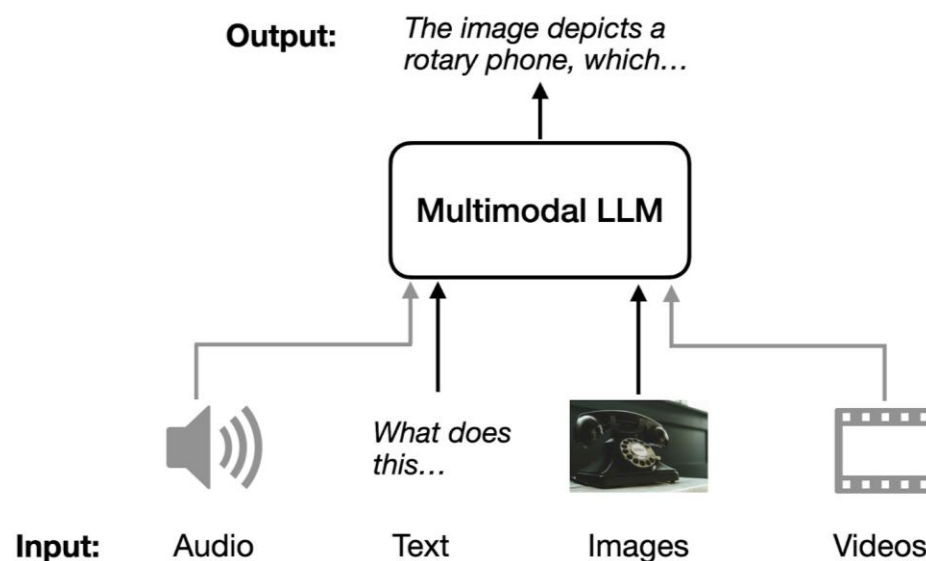
Stratégies de fusion



Les trois gringos

La question centrale en modélisation multimodale est : **à quel moment et comment combine-t-on les modalités ?**

Les trois stratégies les plus classiques sont la **fusion précoce**, la **fusion tardive** et la **fusion intermédiaire**. Les surveys récents les décrivent aussi comme **data-level / feature-level / output-level fusion** ou comme une variante plus hybride lorsque la fusion intervient au milieu du réseau.



Stratégies de fusion



Fusion précoce (early fusion)

Dans la **fusion précoce**, on combine les modalités **très tôt**, généralement au niveau des données transformées en features ou embeddings, avant le cœur du modèle de décision. C'est une stratégie simple et intuitive. Une écriture typique est :

$$z = \text{concat}(x^{(\text{texte})}, x^{(\text{image})}, x^{(\text{audio})})$$

où z devient l'entrée d'un modèle unique.

Avantages

- simple à implémenter ;
- permet d'apprendre tôt certaines corrélations inter-modales ;
- utile lorsque les modalités sont déjà bien synchronisées ou représentées dans des espaces compatibles.

Limites

- sensible à la qualité des représentations initiales ;
- suppose souvent que les modalités soient alignées ou comparables assez tôt ;
- gère moins bien les modalités manquantes ou de nature très hétérogène.

Stratégies de fusion



Fusion tardive (late fusion)

Dans la **fusion tardive**, chaque modalité est d'abord traitée **par son propre modèle**, puis on combine les sorties finales ou intermédiaires de haut niveau : scores, probabilités, décisions ou embeddings agrégés. Un schéma mathématique simple est :

$$\hat{y} = \sum_{m=1}^M \alpha_m \hat{y}^{(m)}$$

où $\hat{y}^{(m)}$ est la prédiction du modèle de la modalité m , et α_m un poids de fusion.

Avantages

- plus modulaire ; plus facile à maintenir ou à faire évoluer modalité par modalité ;
- parfois plus robuste quand les modalités sont très différentes ou pas toujours disponibles.

Limites

- capte moins bien les interactions fines entre modalités ;
- peut manquer des signaux croisés utiles qui n'apparaissent que lorsqu'on fait interagir les modalités plus tôt.

Stratégies de fusion



Fusion intermédiaire (intermediate fusion)

La **fusion intermédiaire** combine d'abord les modalités **partiellement encodées séparément**, puis les fait interagir **au milieu du réseau** via une couche de fusion, d'attention croisée, de gating ou un bloc spécialisé. C'est souvent la stratégie la plus riche et la plus utilisée dans les architectures modernes. Une forme simple en vision-langage est la **cross-attention** :

$$\text{CrossAttn}(Q_{\text{texte}}, K_{\text{image}}, V_{\text{image}}) = \text{softmax}\left(\frac{Q_{\text{texte}} K_{\text{image}}^{\top}}{\sqrt{d}}\right) V_{\text{image}}$$

Ici, le texte “interroge” l'image pour récupérer l'information visuelle pertinente. On peut évidemment faire l'inverse ou utiliser plusieurs blocs croisés.

Avantages

- capte mieux les interactions sémantiques et structurelles fines ;
- particulièrement adaptée aux architectures modernes à base de Transformers et d'attention ;

Limites

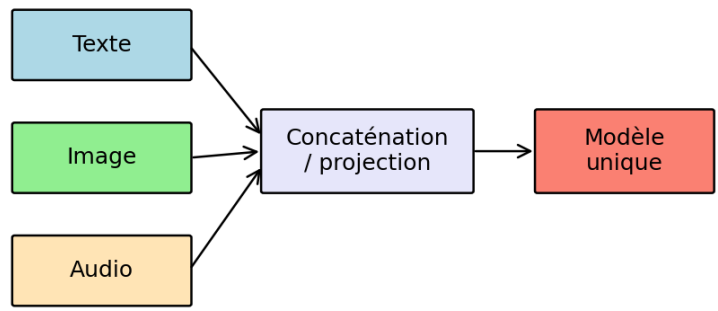
- plus coûteuse à entraîner ; plus sensible aux problèmes d'alignement ; plus difficile à interpréter et à déboguer.

Stratégies de fusion



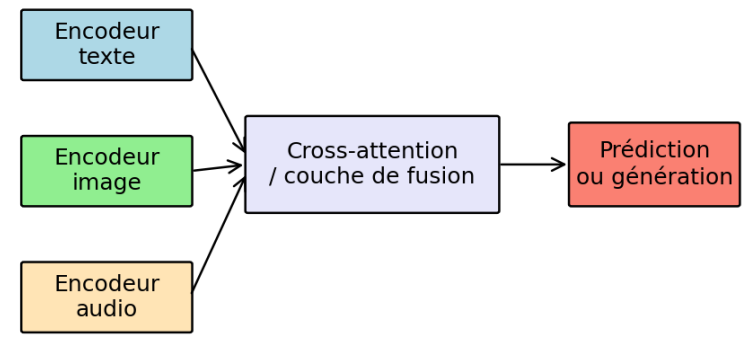
Les trois gringos

Fusion précoce



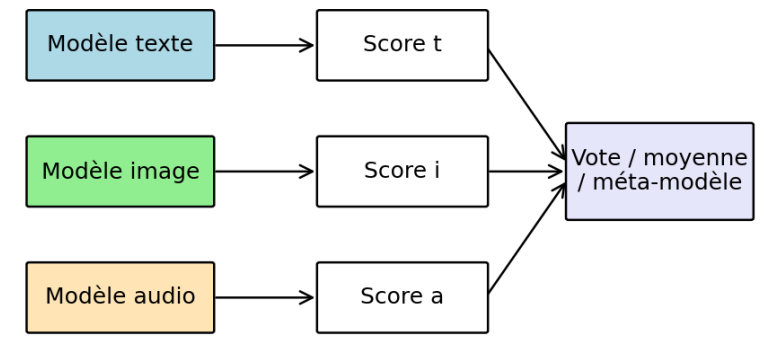
Fusion des modalités avant le cœur du modèle

Fusion intermédiaire



Chaque modalité est d'abord encodée séparément

Fusion tardive



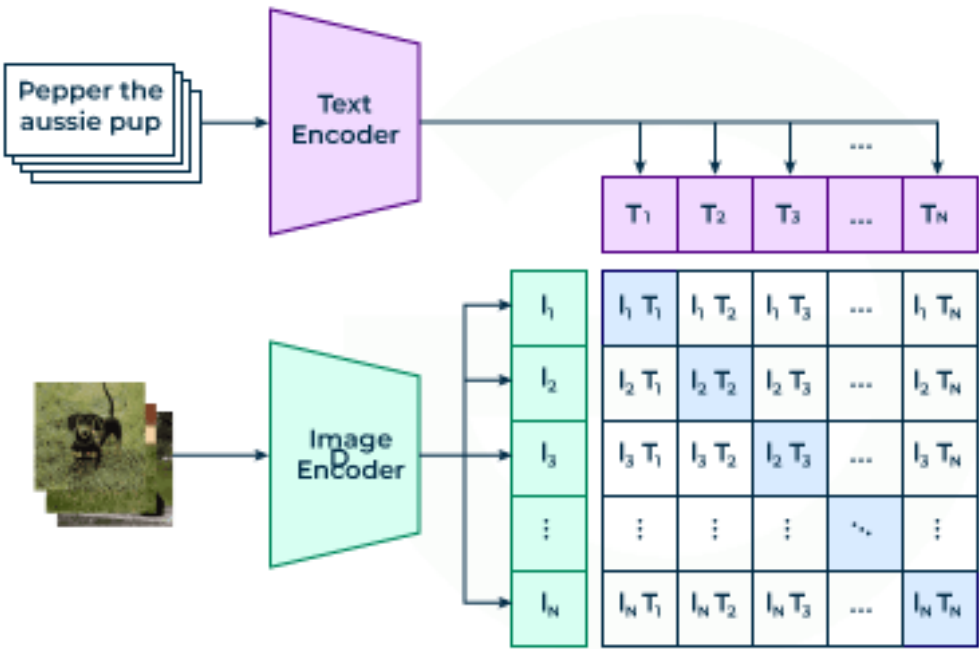
Chaque modalité décide d'abord, puis on combine les sorties

Modèles emblématiques

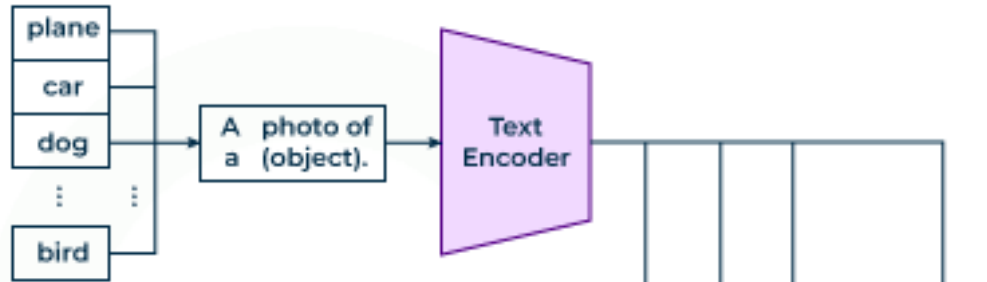


CLIP (*Contrastive Language-Image Pretraining*) est un modèle fondamental pour l'apprentissage image-texte. Il apprend un espace commun où images et textes appariés sont proches, ce qui lui donne de fortes capacités de **zero-shot transfer** sur des tâches visuelles. Il est entraîné sur de très grands volumes de paires image-texte issues du web.

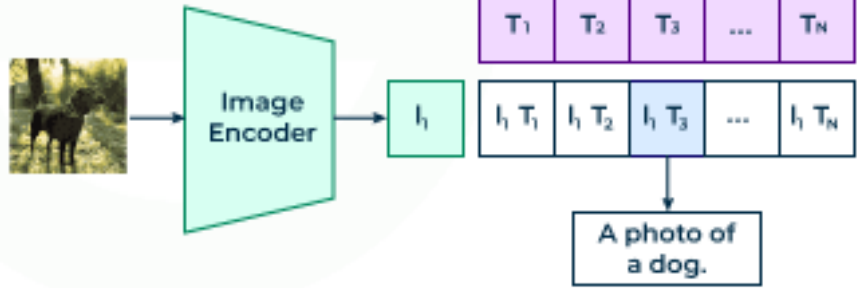
(1) Contrastive pre-training



(2) Create dataset classifier from label text



(3) Use for zero-shot prediction

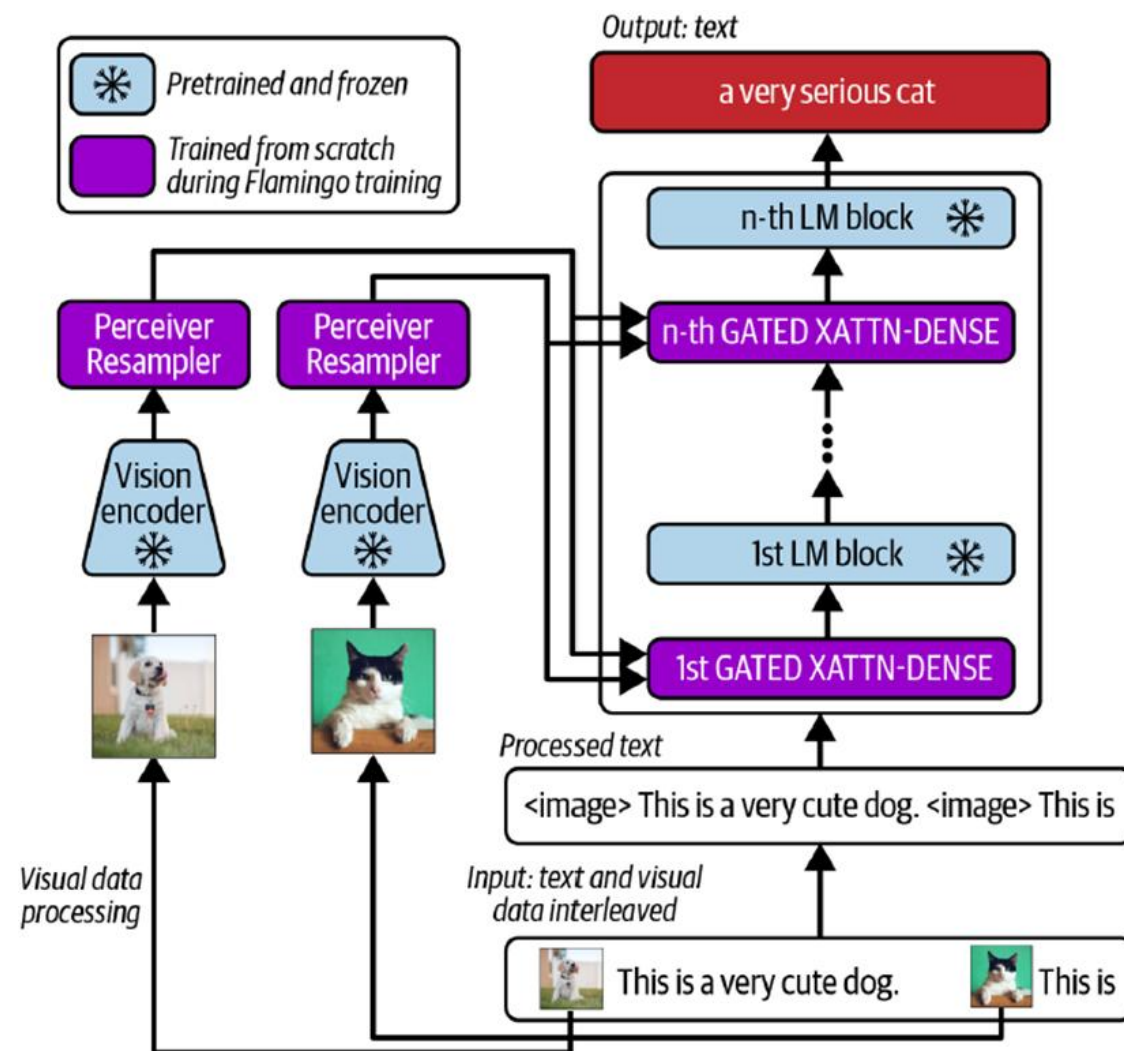


Modèles emblématiques



Flamingo

Flamingo est un **visual language model** conçu pour le **few-shot learning multimodal**. Son idée clé est de **relier un grand modèle de vision et un grand modèle de langage** via une architecture qui accepte des séquences intercalant texte et images, ce qui lui permet de résoudre plusieurs tâches par prompting.

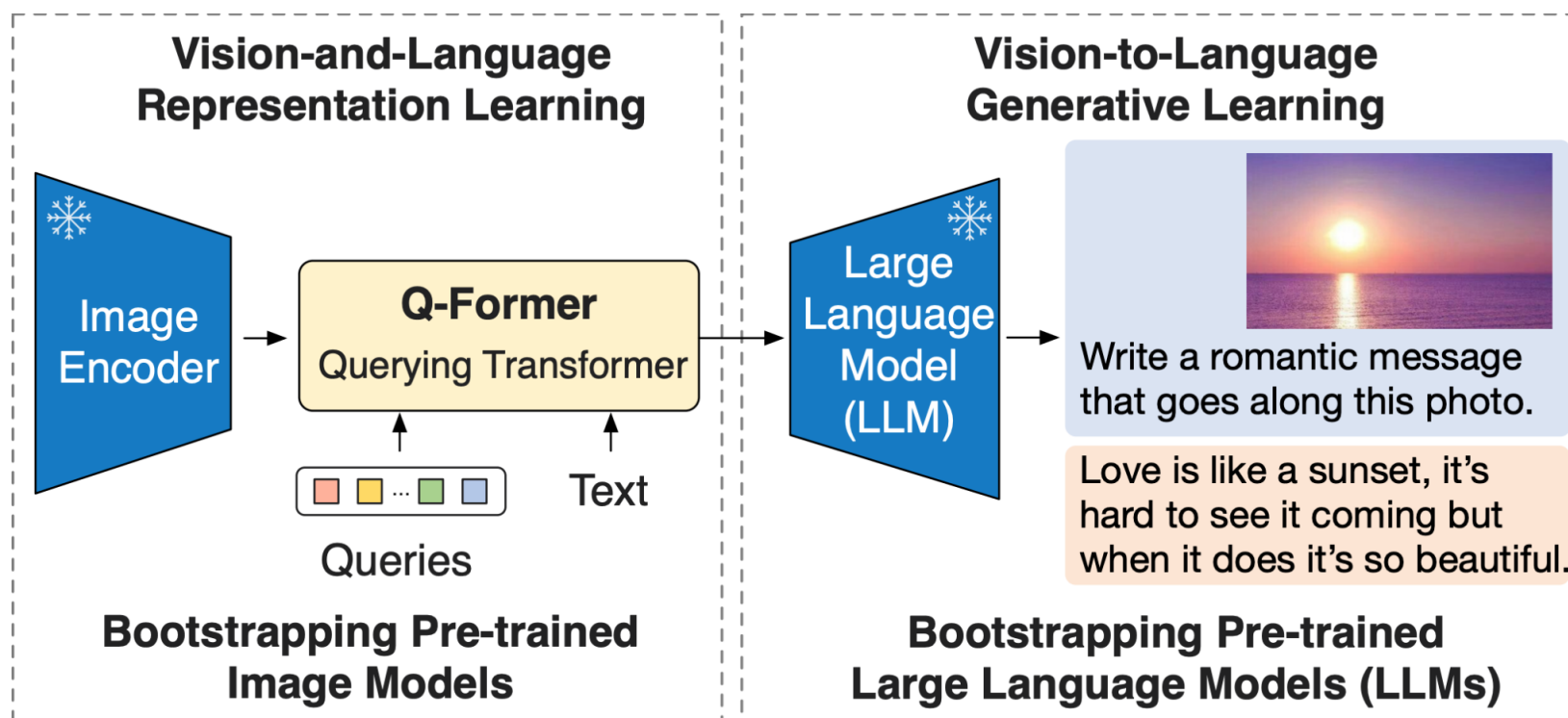


Modèles emblématiques



BLIP-2

BLIP-2 propose une stratégie très influente : garder **gelés** un encodeur visuel pré-entraîné et un grand modèle de langage, puis apprendre un petit module de pont, appelé **Q-Former**, pour réduire le coût du pré-entraînement multimodal. Le modèle obtient d'excellents résultats tout en ayant beaucoup moins de paramètres entraînaibles que d'autres approches.

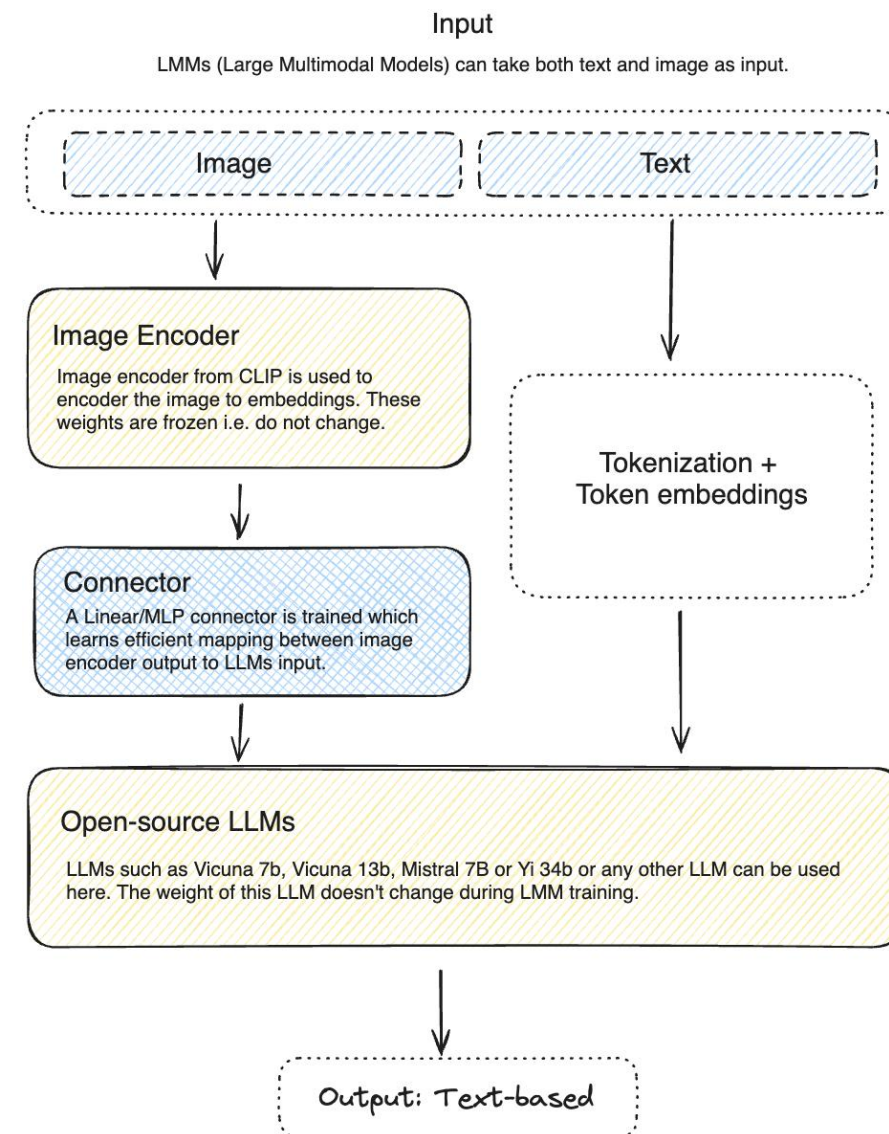


Modèles emblématiques



LLaVA

LLaVA (*Large Language and Vision Assistant*) a popularisé l'idée de **visual instruction tuning** : relier un encodeur visuel et un LLM, puis entraîner le système sur des données d'instructions image-texte pour obtenir un assistant capable de dialogue visuel généraliste.



Avantages de la modélisation multimodale

Meilleure compréhension contextuelle

Le principal avantage est une **meilleure compréhension du contexte** grâce à l'intégration de signaux complémentaires. Une image seule ou un texte seul peuvent être ambigus, alors que leur combinaison lève souvent cette ambiguïté.

Robustesse au bruit et aux données incomplètes

Quand une modalité est bruitée, une autre peut compenser. C'est un bénéfice fort dans les scénarios réels, notamment audio-visuels, robotiques ou médicaux.

Meilleur transfert et nouvelles capacités

Les grands modèles multimodaux héritent parfois des forces de plusieurs mondes : représentations visuelles robustes, capacités génératives des LLM, prompting, zero-shot ou few-shot. C'est exactement ce qu'illustrent Flamingo, BLIP-2 et LLaVA.

Limites et défis

Alignement entre modalités

Le problème le plus difficile est souvent l'**alignement** : comment faire correspondre correctement un mot à une région d'image, un segment audio à une vidéo, ou un événement visuel à une annotation textuelle ? Si l'alignement est mauvais, la fusion peut devenir contre-productive.

Hétérogénéité des données

Les modalités n'ont pas le même format, la même granularité ni la même temporalité. Le texte est symbolique, l'image est spatiale, l'audio est temporel, la vidéo est spatio-temporelle. Cette hétérogénéité complique la représentation et la fusion.

Données manquantes, bruit et déséquilibre modal

Dans la pratique, certaines modalités peuvent être absentes, de mauvaise qualité, ou dominer excessivement les autres. Les surveys récents soulignent notamment le problème de **modality gap** et les difficultés liées au déséquilibre entre modalités.

Limites et défis

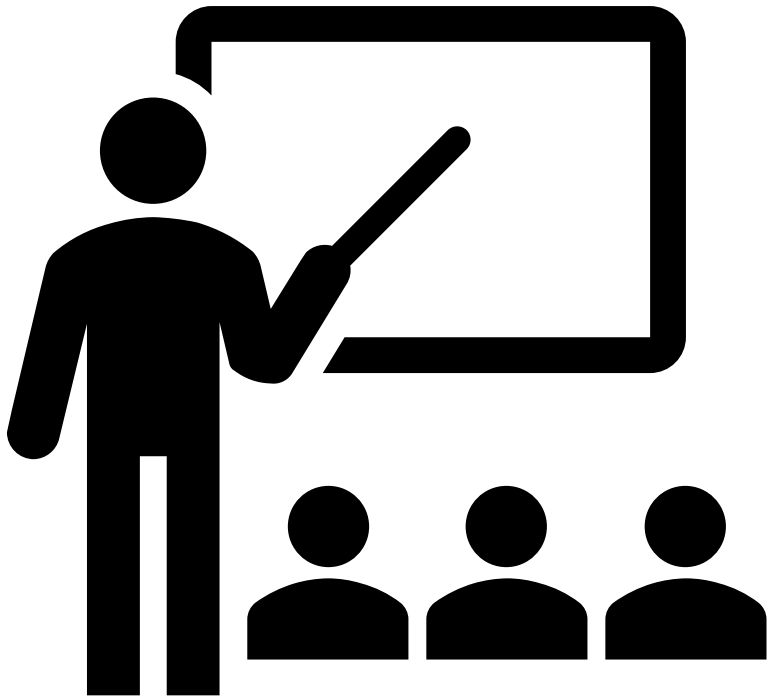
Coût computationnel

Les modèles multimodaux modernes, surtout lorsqu'ils combinent grands encodeurs visuels, LLM et blocs d'attention croisée, sont coûteux à entraîner et à servir. BLIP-2 a précisément gagné en popularité parce qu'il cherchait à réduire ce coût via des modules gelés et un connecteur léger.

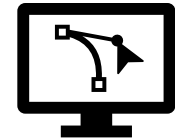
Évaluation plus complexe

Évaluer un système multimodal est souvent plus difficile qu'évaluer un modèle unimodal : il faut vérifier non seulement la qualité de la sortie, mais aussi si le modèle s'appuie réellement sur la bonne modalité, s'il hallucine, et s'il reste robuste quand certaines entrées changent.

Contenu du cours



Fondamentaux du
Machine & Deep
Learning



Extraction, préparation,
visualisation et
communication des
données



Modélisation
supervisée et non-
supervisée en Machine
Learning



Modélisation
supervisée et avancée
en Deep Learning

Vos avis sur le cours



Modalités d'évaluation



Evaluation
écrite
25 %



Projet Machine
Learning
35 %



Projet Deep
Learning
40 %



Bonus de
participation
😊